

第 4 章：Introducing Python Objects (Python 对象入门)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：本章正式开启 Python 语言之旅，用一次“快速巡游”(tour)预览 Python 的核心对象类型——数字、字符串、列表、字典、元组、文件、集合——以及可以对它们施加的操作，并首次定义序列、映射、不可变性、多态、迭代协议、垃圾回收等贯穿全书的基石概念。它相当于全书的地图册：后续章节(第 5–9 章)会对这些对象逐一做第二遍深入讲解，而本章只要求你“先看见全貌”。

4.1 开篇引言

英文原文

This chapter begins our tour of the Python language. In an informal sense, in Python we do things with stuff. "Things" take the form of operations like addition and concatenation, and "stuff" refers to the objects on which we perform those operations. In this part of the book, our focus is on that stuff, and the things our programs can do with it. Somewhat more formally, in

Python, data takes the form of objects—either built-in objects that Python provides, such as strings and lists, or add-on objects we create with Python classes or external-language tools. As you'll find, these objects are essentially just pieces of memory, with values and associated operations. Moreover, everything is an object in a Python script. Even simple numbers qualify, with values (e.g., 99) and supported operations (+, -, and so on). Because objects are also the most fundamental notion in Python programming, this chapter gets us started with a survey that previews Python's built-in object types. Later chapters in this part provide a second pass that fills in details we'll gloss over in this survey. Here, our goal is a brief tour to introduce the basics.

中文翻译

本章开始我们对 Python 语言的巡游。用一个不太正式的说法：在 Python 中，我们用"事情" (things) 对"材料" (stuff) 做操作。"事情"表现为加法、拼接 (concatenation) 之类的操作，"材料"则指我们执行这些操作所作用的对象 (object)。在本部分的内容里，我们的焦点就是那些"材料"，以及我们程序能对它们做的"事情"。稍微正式一点地说：在 Python 中，数据以对象 (object) 的形式存在——要么是 Python 自带的内置对象，如字符串 (string) 和列表 (list)；要么是我们用 Python 的类 (class) 或外部语言工具创建的自定义对象。你会发现，这些对象本质上就是一块块内存，带有值 (value) 和与之关联的操作。而且

，在 Python 脚本里，一切都是对象（everything is an object）。即便是简单的数字也符合——数字带有值（例如 99）和受支持的操作（+、- 等等）。因为对象也是 Python 编程中最基本的概念，本章先用一个总览（survey）带你预览 Python 的内置对象类型。本部分后面的章节会做第二遍讲解，补上我们在这次总览中略过的细节。本章的目标就是一次简短巡游，介绍基础。

深度理解

·核心概念：Python 的世界观是“一切皆对象”。数据不是赤裸裸的“值”，而是内存中的“对象”——值是对象的属性之一，操作（operation）绑定在对象的类型上。作者用 things/stuff 打比方：操作是“事情”，对象是“材料”，对象 = 值 + 操作。本书最核心的心智模型就此确立：对象是值的载体，变量只是指向对象的名字（reference）。

·底层实现：在 CPython（标准 Python 解释器）中，每个对象都是堆（heap）上的一块内存，以一个 C 结构体表示，头部是“对象头”，记录引用计数（reference count）和类型指针（obtype）。"99"这个整数在内存里是一个 PyLongObject 结构体；"加法"不是写死在数字内部的，它是绑定在 int 类型上的操作（C 层对应 PyNumberAdd）。

·设计原因：“一切皆对象”让整门语言只有一套规则——赋值、传参、返回、垃圾回收对数字、字符串、函数、模块、类、甚至类型本身全部一视同仁。规则统一，学习成本就低，这是 Python“最小惊讶原则”（Principle of Least Astonishment）的根源。

·实际问题：以后每当你看到 $a = b$ 、函数传参、列表嵌套等代码时，都可以用这套模型推导它的行为。先接受"对象在内存中独立存在"，后面理解共享引用（shared reference）和垃圾回收（garbage collection）就水到渠成。

·初学者误区：最常见的错误想法是"变量装着值"（C 的思维）。实际上 $a = 99$ 的意思是"把名字 a 绑定到整数对象 99"，而不是"把 99 存进变量 a 这个盒子里"。这也是后面 $a = b$ 时两个名字指向同一对象、改一个"影响"另一个的原因。

4.2 Python 概念层级 (The Python Conceptual Hierarchy)

英文原文

Before we get to the code, let's first establish a clear picture of how this chapter fits into the overall Python picture. From a more concrete perspective, Python programs can be decomposed into modules, statements, expressions, and objects, as follows: 1. Programs are composed of modules. 2. Modules contain statements. 3. Statements contain expressions. 4. Expressions create and process objects. The discussion of modules in Chapter 3 introduced the highest level of this hierarchy. This part's chapters begin at the bottom—exploring both built-in objects and the expressions you can code to use them. We'll move on to statements

nts in the next part of the book, though you will find that they largely exist to manage the objects you'll meet here. Furthermore, by the time we reach classes in the OOP part of this book, you'll discover that they allow you to define new object types of your own, by both using and emulating the object types you will explore here. Because of all this, built-in objects are a mandatory point of embarkation for all Python journeys.

NOTE (术语时刻 Terminology moment) : Traditional introductions to programming often stress its three pillars of sequence ("Do this, then that"), selection ("Do this if that is true"), and repetition ("Do this many times"). Python has tools in all three categories, and these terms might help you organize your thinking early on. But they are also artificial and simplistic, and prone to confuse. For example, tools such as comprehensions are both repetition and selection; these terms have other, more specific meanings in Python; and many later concepts won't seem to fit this mold at all. In Python, the more strongly unifying principle is objects and what we can do with them. To see why, read on.

中文翻译

在写代码之前，我们先把本章在 Python 整体图景中的位置讲清楚。从一个更具体的视角看，Python 程序可以分解为模块 (module)、语句 (statement)、表达式 (expressions)

sion) 和对象 (object) , 层级如下: 1. 程序由模块组成。2. 模块包含语句。3. 语句包含表达式。4. 表达式创建并处理对象。第 3 章对模块的讨论介绍了这一层级的最顶端。本部分的章节从最底层开始——既探索内置对象, 也探索你能写出来使用它们的表达式。我们会在本书的下一部分继续讨论语句, 不过你会发现, 语句在很大程度上就是为了管理你在这里遇到的对象而存在的。此外, 等我们学到 OOP (面向对象编程) 部分的类时, 你会发现类允许你定义自己的新对象类型——既使用、又模拟你在这里将要探索的对象类型。正因如此, 内置对象是所有 Python 旅程的必经起点。

NOTE (术语时刻): 传统的编程入门往往强调编程的三大支柱: 顺序 (sequence, "先做这个, 再做那个")、选择 (selection, "如果这是真的, 就做这个")、重复 (repetition, "这件事做很多次")。Python 在三大类里都有工具, 这些术语也许能帮你早期整理思路。但它们也是人为制造的、过于简化的, 而且容易造成混淆。比如推导式 (comprehension) 既是重复又是选择; 这些术语在 Python 中还有更具体的含义; 而且很多后面的概念根本套不进这个框。在 Python 中, 更强的统一原则是对象 (object) 以及我们能对对象做什么。想知道为什么? 接着往下读。

深度理解

·核心概念: Python 程序的四层解剖: 程序 = 模块的集合, 模块 = 语句的集合, 语句 = 表达式的集合, 表达式 = 对象的"制造车间"。这是一张自上而下的地图。注意层级的排布顺序: 表达式在最底层, 它负责创建对象——这暗示了对象是 Python 的"基本粒子"。

·底层实现：CPython 执行一个 .py 文件时，先把源码编译成字节码（bytecode），再交给虚拟机（VM）逐条执行。语句是字节码的分组单位，而表达式求值后会在求值栈（eval stack）上留下一个指向对象的引用——"表达式创建对象"在字节码层面就是 LOADCONST、BINARYADD 之类的指令在栈上搬运对象引用。

·设计原因：把"表达式产生对象"放在最底层，是因为对象是基本粒子；语句只是"调度器"，控制对象的创建、流转与销毁。先学对象、后学语句，符合从底层到上层的认知顺序，也符合本书"先有材料、再有动作"的教学节奏。

·实际问题：这个层级回答了"为什么 import 要写在文件顶部"（模块级语句）、"为什么 def 是语句而不是表达式"（它创建函数对象并绑定名字）这类疑问。有了这张地图，很多困惑都能立刻定位到正确的层级。

·初学者误区：把"变量"当作对象。在 Python 里变量只是名字，不存在 C 语言的"变量盒子"；另外，模块也不只是"代码的集合"，它本身是一个对象（模块对象），这正是 import 机制的基础。

·关于术语时刻（NOTE）：作者提醒"顺序/选择/重复"三支柱只是人为划分，会误导人（比如推导式同时是重复+选择）。真正的统一原则是对象与操作——所以本书的组织方式不是按语句类型，而是按对象类型（数字、字符串、列表.....）。这决定了整本书的章节结构。

4.3 为什么使用内置对象？ (Why Use Built-in Objects?)

英文原文

If you've used lower-level programming languages, you know that much of your work centers on implementing objects—also known as data structures—to represent the components in your application's domain. You may need to lay out memory structures, manage memory allocation, implement search and access routines, and so on. These chores are about as tedious (and error-prone) as they sound, and they usually distract from your program's real goals. In typical Python programs, most of this grunt work goes away. Because Python provides powerful object types as an intrinsic part of the language, there's usually no need to code object implementations before you start solving problems. In fact, unless you have a need for special processing that built-in objects don't provide, you're almost always better off using a built-in object instead of implementing your own. Here are some reasons why: Built-in objects make programs easy to write. For simpler tasks, built-in objects are often all you need to represent the structure of problem domains. Because you get powerful tools such as collections (lists) and search tables (dictionaries) for free, you can use them immediately. You can get a lot of work done with

h Python's built-in object types alone. Built-in objects are components of extensions. For more complex tasks, you may need to provide your own objects using Python classes or C-language interfaces. But as you'll see in later parts of this book, objects implemented manually are often built on top of built-in objects such as lists and dictionaries. For instance, a stack data structure may be implemented as a class that manages or customizes a built-in list. Built-in objects are often more efficient than custom data structures. Python's built-in objects employ algorithms that have already been optimized and are often implemented in a lower-level language like C for speed. Although you can write similar object types on your own, you'll usually be hard-pressed to match the level of performance that built-in object types provide. Built-in objects are a standard part of the language. In some ways, Python borrows both from languages that rely on built-in tools (e.g., Lisp) and languages that rely on the programmer to provide tool implementations of their own (e.g., C++). Although you can implement unique object types in Python, you don't need to do so just to get started. Moreover, because Python's built-ins are standard, they're always the same; proprietary toolkits, on the other hand, tend to differ from site to site. In other words, not only do built-in object types make programming easier, they're also more powerful and accessible than most of what can be created from scratch.

tch. Regardless of whether you implement new object types, built-in objects form the core of every Python program.

中文翻译

如果你用过更底层的编程语言，你就会知道，你的大量工作都集中在实现对象（object）——也就是数据结构（data structure）——来表示应用领域中的各个组成部分。你可能需要设计内存布局、管理内存分配、实现查找和访问例程，等等。这些杂活正如听起来那样乏味（而且容易出错），并且通常会分散你对程序真正目标的注意力。在典型的 Python 程序中，这些苦力活大多消失了。因为 Python 把强大的对象类型作为语言本身的一部分提供，通常你不需要在开始解决问题之前先实现对象。事实上，除非你确实需要内置对象提供不了的特别处理，否则使用内置对象几乎总是比自己实现更好。原因如下：内置对象让程序容易编写。对于较简单的任务，内置对象往往就是你表示问题域结构所需的全部。因为你免费得到了集合（列表）和查找表（字典）这样强大的工具，可以立即使用它们。仅靠 Python 的内置对象类型，你就能完成大量工作。内置对象是扩展的组件。对于更复杂的任务，你可能需要用 Python 类或 C 语言接口来提供自己的对象。但正如你在本书后面部分会看到的，手动实现的对象往往是构建在列表和字典等内置对象之上的。例如，一个栈（stack）数据结构可以实现为一个管理或定制内置列表的类。内置对象通常比自定义数据结构更高效。Python 的内置对象采用了已经优化过的算法，而且为了提高速度，通常是用 C 这样的低级语言实现的。虽然你可以自己写出类似的对象类型，但通常很难匹敌内置对象类型提供

的性能水平。内置对象是语言的标准部分。在某些方面，Python 既借鉴了依赖内置工具的语言（如 Lisp），也借鉴了依赖程序员自己实现工具的语言（如 C++）。虽然你可以在 Python 中实现独特的对象类型，但你不需要为了入门而这么做。而且，因为 Python 的内置对象是标准的，它们永远是一样的；相比之下，专有工具包往往因站点而异。换句话说，内置对象类型不仅让编程更容易，而且比大多数从零创建的东西更强大、更易用。无论你是否实现新的对象类型，内置对象都是每个 Python 程序的核心。

深度理解

理由	一句话解释	生活类比
让程序容易编写	列表=现成集合、字典=现成查找表，开箱即用	住酒店——床、桌子、热水器都配好了，不用自己从家搬家具
是扩展的组件	自定义对象（如栈）也建立在 list 之上	搭积木——特殊形状的积木也要卡在标准积木上
比自定义结构更高效	内置算法由 C 语言优化实现	坐高铁 vs 自己开车——多数情况下高铁更快更省心
是语言的标准部分	内置对象到处一致，不因环境而异	标准插座 vs 各家专用充电口

·核心概念：内置对象是“开箱即用”（batteries included）哲学的体现。底层语言（C/C++）把“实现数据结构”当作程序员的日常功课，Python 把这门功课从程序员手里拿走了。

·底层实现：CPython 的 list 是动态数组（C 数组 + 容量预留，扩容约 1.125 倍），dict 是哈希表（开放寻址），它们都在 C 层用精心调优的算法实现。自己用 Python 写同样的结构，每一个操作都经过字节码解释，速度自然无

法相比。

·设计原因：作者明确警告"重新发明轮子"不划算——除非内置对象确实无法满足需求，否则自定义对象几乎没有胜算。这同时是 Lisp（内置工具）与 C++（自己实现）两种路线的折中。

·实际问题：实际项目中 90% 的数据结构需求（去重、查找、排序、排队）都可以用内置对象组合解决；只有当性能瓶颈被证实、或领域模型复杂到需要自定义行为时，才考虑自定义类。

·初学者误区：初学 C/Java 的人容易觉得"自己写链表/哈希表很酷"。在 Python 中这是反模式——你写的 Python 链表几乎不可能比内置 list 快，还会引入 bug。先学会用好内置对象，再谈自定义。

4.4 Python 的核心对象类型 (Python's Core Object Types)

英文原文

Table 4-1 previews Python's built-in objects, and some of the syntax used to code their literals—that is, the expressions that generate these objects. Some of these objects will probably seem familiar if you've used other languages; for instance, numbers and strings represent numeric and textual values, respectively, and file objects provide an interface for processing real files stored on your computer. To some readers, though

h, the object types in Table 4-1 may be more general and flexible than what you are accustomed to. For instance, you'll find that lists and dictionaries alone are powerful data representation tools that obviate most of the work you do to support collections and searching in lower-level languages. In short, lists provide ordered collections of other objects, while dictionaries store objects by key, and both come with automatic memory management, support arbitrarily nesting, can grow and shrink on demand, and may contain objects of any kind. Also shown in Table 4-1, program units such as functions, modules, and classes—which you'll meet in later parts of this book—are objects in Python too; they are created with statements and expressions such as `def`, `class`, `import`, and `lambda` and may be passed around scripts freely, stored within other objects, and so on. Python also provides a set of implementation-related objects such as compiled-code objects, which are generally of interest to tool builders more than application developers; we'll explore these later, though in less depth due to their specialized roles. Despite its title, Table 4-1 isn't really complete because everything we process in Python programs is a kind of object. For instance, when we perform text pattern matching in Python, we create pattern objects, and when we do network scripting, we use socket objects. These other kinds of objects are generally created by importing and using functions in standard or a

dd-on library modules, and have behavior all their own. Patterns and sockets, for example, are made by calling tools in the standard library's `re` and `socket` modules, respectively. We usually call the objects in Table 4-1 core object types, though, because they are effectively built into the Python language itself—that is, there is specific expression syntax for generating most of them. For instance, when you run the following code with characters surrounded by quotes in a REPL or program file: `python >>>'Python'` you are, technically speaking, running a literal expression that generates and returns a new string object. There is Python language syntax to make this object. Similarly, an expression wrapped in square brackets makes a list, one in curly braces makes a dictionary or set, and so on. Even though, as you'll see, Python does not require or use type declarations, the syntax of the expressions you run determines the types of objects you create and use. In fact, object-generation expressions like those in Table 4-1 are generally where types originate in the Python language. Just as importantly, once you create an object, you bind its operation set for all time—you can perform only string operations on a string and list operations on a list. In formal terms, this means that Python is dynamically typed, a model that keeps track of object types for you automatically instead of requiring declaration code, but it is also strongly typed, a constraint that means you can perform on an

object only operations that are valid for its type. We'll study each of the object types in Table 4-1 completely in upcoming chapters. Before digging into the full details, though, let's begin by taking a quicker look at Python's core objects in action. The rest of this chapter provides a preview of the operations we'll explore in more depth in the chapters that follow. Don't expect to find the full story here—the goal of this chapter is just to whet your appetite and introduce some key ideas. Still, the best way to get started is to get started, so let's jump right into some real object-wrangling code.

中文翻译

表 4-1 预览了 Python 的内置对象，以及编写它们的字面量（literal）所用的部分语法——字面量就是生成这些对象的表达式。如果你用过其他语言，其中一些对象可能看起来眼熟：例如数字和字符串分别表示数值和文本值，文件（file）对象则提供了处理存储在计算机上的真实文件的接口。不过，对某些读者来说，表 4-1 中的对象类型可能比你习惯的更加通用和灵活。例如，你会发现，仅仅列表和字典就是强大的数据表示工具，能省去你在低级语言中支持集合和查找的大部分工作。简而言之：列表提供其他对象的有序集合，字典按键（key）存储对象；两者都带有自动内存管理、支持任意嵌套、可以按需增长和收缩，并且可以容纳任何种类的对象。表 4-1 还显示，程序单元（program unit）——如函数（function）、模块（module）和类（class），你会在本书后面部分遇到它们——在 Python 中同样

也是对象；它们由 `def`、`class`、`import`、`lambda` 等语句和表达式创建，可以自由地在脚本中传递、存储在其它对象中，等等。Python 还提供一组与实现相关的对象，例如已编译代码对象（compiled-code object），它们通常对工具构建者比对应用开发者更有意义；我们后面会探索这些，但由于它们角色的特殊性，会讲得浅一些。尽管表 4-1 名为“核心对象”，它其实并不完整，因为我们在 Python 程序中处理的一切都是某种对象。例如，在 Python 中进行文本模式匹配时，我们创建的是模式（pattern）对象；做网络脚本编程时，我们使用的是套接字（socket）对象。这些其他种类的对象通常是通过导入并调用标准库或附加库模块中的函数创建的，并拥有自己的行为。例如，模式和套接字分别是调用标准库 `re` 模块和 `socket` 模块中的工具创建的。不过，我们通常把表 4-1 中的对象称为核心对象类型（core object type），因为它们实际上是内置在 Python 语言本身之中的——也就是说，大多数核心对象都有专门的表达式语法来生成。例如，当你在 REPL 或程序文件中运行下面这行用引号包围字符的代码：`python >>> 'Python'` 严格来说，你运行的是一个字面量表达式（literal expression），它生成并返回一个新的字符串对象。Python 语言中有专门的语法来制造这个对象。同理，用方括号包围的表达式生成列表，用花括号包围的生成字典或集合，依此类推。正如你将会看到的，即使 Python 不需要也不使用类型声明，你运行的表达式的语法也决定了你创建和使用的对象的类型。事实上，表 4-1 中这类生成对象的表达式，就是类型在 Python 语言中的“出生地”。同样重要的是：一旦创建了对象，它的操作集就被永久绑定——你只能对字符串执行字符串操作，对列表执行列表操作。用正式术语来说，这意味

着 Python 是动态类型（dynamically typed）的——它自动跟踪对象类型，而不是要求你写声明代码；同时它也是强类型（strongly typed）的——这是一个约束，意味着你只能对对象执行对其类型有效的操作。我们将在接下来的章节中完整地研究表 4-1 中的每一种对象类型。但在深入全部细节之前，我们先快速看一下 Python 核心对象在实际中的样子。本章其余部分是对后面章节将深入探索的操作的一次预览。别指望在这里看到完整的故事——本章的目标只是勾起你的兴趣，介绍一些关键思想。不过，最好的开始方式就是开始，所以让我们直接进入真正的对象操作代码吧。

表 4-1. Python 内置（核心）对象

对象类型	示例字面量/创建方式
Numbers（数字）	1234, 3.1415, 0b111, 1234, 3+4j, Decimal, Fraction
Strings（字符串）	'code', "app's", b'a\x01c', 'h\u00c4ck', 'hÄck'
Lists（列表）	[1, [2,'three'], 4.5], list(range(10))
Dictionaries（字典）	{'job':'dev','years': 40}, dict(hours=10)
Tuples（元组）	(1,'app', 4,'U'), tuple('hack'), namedtuple
Files（文件）	open('docs.txt'), open(r'C:\data.bin','wb')
Sets（集合）	set('abc'), {'a','b','c'}
Other core objects（其他核心对象）	Booleans（布尔值）、types（类型）、None
Program-unit objects（程序单元对象）	函数、模块、类（第四、五、六部分）
Implementation objects（实现对象）	已编译代码、栈回溯（第四、七部分）

深度理解

·核心概念：什么是"核心"对象类型？判据是：语言本身为它们提供了专门的生成语法（字面量）。'Python' 是一种语法、[1,2] 是一种语法、{...} 是一种语法；而文件对象没有字面量，必须调用 open() 函数，所以文件是核心类型中的"异类"。类型不是来自声明，而是来自"用哪种表达式去制造"——这是 Python 与 C/Java 最根本的差异之一。

·底层实现：类型的"身份"存放在对象的 obtype 指针（指向类型对象）里。当执行 $1 + 2$ 时，解释器通过对象的类型查找"加法"槽位（C 层的 PyNumberAdd），再分发到具体实现。因为类型信息在运行时才被读取，所以类型检查发生在操作执行时，而不是编译时——这就是"动态类型"的机械原理。

·设计原因：动态类型让代码不必为每种类型重复编写（多态的前提），代码更短、更灵活；强类型则保证"只有类型合法的操作才能执行"（比如'abc' + 123 立即报 TypeError），避免 C 语言里"把整数当指针"这类静默灾难。两者组合 = 灵活但不失控。

·实际问题：这个模型解释了三个常见现象：①同一个变量可以先绑整数再绑字符串（类型属于对象，不属于变量）；② $1 + 'a'$ 报错（强类型）；③type() 能随时查对象的类型（类型对象本身也是对象）。

·初学者误区：①认为"Python 变量有类型"——错，类型属于对象，变量只是名字；②把"动态类型"误当"弱类型"——动态管"类型何时确定"，强/弱管"操作是否被强制合

法", Python 两者兼备; ③以为"Python 没有类型"——恰恰相反, Python 的类型意识非常强, 只是不需要你写声明。

代码分析

```
>>> 'Python'      # ""  
'Python'
```

·解释器如何处理: 这条 REPL 输入经历了完整流水线: ①词法分析 (lexing) ——把'Python' 识别为一个 STRING 字面量 token; ②语法分析 (parsing) ——生成抽象语法树 (AST); ③编译——将 AST 编译成字节码, 核心指令是 LOADCONST 0 (把常量表中的字符串对象压入求值栈); ④执行——虚拟机执行 LOADCONST, 栈顶出现一个 str 对象; ⑤REPL 显示——打印该对象的 repr ('Python')。注意: 字面量在编译时就被创建为常量对象, 每次执行 LOADCONST 只是把同一个对象的引用压栈。

·设计思想: 一行字面量就是一个"对象工厂"。方括号→列表、花括号→字典/集合、引号→字符串, 类型由此诞生, 不需要任何 String s = ... 式的声明。这就是"表达式的语法决定对象的类型"。

4.5 数字 (Numbers)

英文原文

If you've done any programming or scripting in the past, some of the object types in Table 4-1 will probably seem familiar. Even if you haven't, numbers are fairly straightforward. Python's core objects set includes the usual suspects: integers that have no fractional part, floating-point numbers that do, and more exotic types—complex numbers with imaginary parts, decimals with flexible precision, rationals with numerator and denominator, and full-featured sets. Built-in numbers are enough to represent most numeric quantities—from your age to your bank balance—but specialized numeric objects like vectors and matrixes are also available as third-party add-ons. Although they offer fancier options, Python's basic number objects are, well, basic. Numbers in Python support the normal mathematical operations. For instance, the plus sign (+) performs addition, a star (*) is used for multiplication, and two stars (**) are used for exponentiation. Here is a demo in a Python REPL of the sort we learned about in Chapter 3:

```
python $ python3 # 启动一个 REPL >>> 123 + 222 #  
整数加法345
```

```
>>> 1.5 * 4 # 浮点乘法6.0
```

```
>>> 1234567, 0x15, bin(21) # 数字分隔符、十六进制、  
二进制(1234567, 21, '0b10101')
```

```
>>> 2 ** 100 # 2 的 100 次方1267650600228229401496  
703205376
```

Notice the last result here: Python's integer object automatically provides extra precision for large numbers like this when needed. You can, for instance, compute 2 to the power 12,345 as an integer in Python, but you probably shouldn't try to grok the result—with nearly 4K digits, it's a lot to take in:

```
python >>> len(str(2**12345)) # 一个超大的数字有多少位
3717
```

This nested-call form works from inside out—first computing the result, then converting it to a string of digits with the built-in `str` function, and finally getting the length of the resulting string with `len`. The end result is the number of digits in the number. `str` and `len` both work on many object types, and we'll use them again as we move along. Besides expressions, there are a handful of useful numeric modules that ship with Python in modules—which are just packages of additional tools that we import to use, using statements like `import` introduced in Chapter 3:

```
python >>> import math
>>> math.pi
3.141592653589793

>>> math.sqrt(85)
9.219544457292887
```

The `math` module contains more advanced numeric tools as functions, while the `random` module performs random-number generation and random selections (here, from a Python list coded in square brackets—an ordered collection of other objects to be introduced

d later in this chapter):

```
python >>> import random >>> random.random() 0.7082048489415967
```

```
>>> random.choice([1, 2, 3, 4]) 2
```

Python also includes more exotic numeric objects—such as complex, fixed-precision, and rational numbers, as well as sets and Booleans—and the third-party open source extension domain has even more (e.g., matrices and vectors, and extended precision numbers). We'll defer discussion of these objects until later in this chapter and book. So far, we've been using Python much like a simple calculator; to do better justice to its built-in objects catalog, let's move on to explore strings.

NOTE (大数字与拒绝服务攻击 Big numbers versus DOS attacks) : Python integers can be arbitrarily large and may even be used to represent floating-point values with extended precision. While integer size is limited only by your computer's memory, though, Python 3.11 and later require extra steps to convert pathologically large numbers to decimal strings (e.g., for str or display). This avoids rare denial-of-service attacks with a 4,300-digit limit that can be lifted with a sys module tool, `setintmaxstrdigits`; call this to count digits in larger numbers, and see Python's docs for all the sortid details.

中文翻译

如果你以前做过任何编程或脚本工作，表 4-1 中的某些对象类型可能看起来眼熟。即使你没做过，数字也是相当直白的。Python 的核心对象集包含通常的那几类：没有小数部分的整数 (integer)、带小数部分的浮点数 (floating-point)，以及更奇特的类型——带虚部的复数 (complex)、精度可调的十进制数 (decimal)、带分子和分母的有理数 (rational)，还有功能完备的集合 (set)。内置数字足以表示大多数数值量——从你的年龄到你的银行存款——但像向量 (vector) 和矩阵 (matrix) 这样的专门数值对象也可以作为第三方附加组件获得。虽然它们提供更花哨的选项，但 Python 的基本数字对象是——嗯——很基础的。Python 中的数字支持常规的数学运算。例如，加号 (+) 执行加法，星号 (*) 用于乘法，两个星号 (**) 用于乘方。下面是在 Python REPL 中的演示（我们在第 3 章学过这种用法）：

```
python $ python3 # 启动一个 REPL
>>> 123 + 222 # 整数加法
345
>>> 1.5 * 4 # 浮点乘法
6.0
>>> 1234567, 0x15, bin(21) # 数字分隔符、十六进制、二进制
(1234567, 21, '0b10101')
>>> 2 ** 100 # 2 的 100 次方
1267650600228229401496703205376
注意这里的最后一个结果：Python 的整数对象在需要时自动为大数提供额外的精度。例如，你可以在 Python 中把 2 的 12,345 次方作为整数计算出来，但你最好不要试图“领会”那个结果——它有将近 4K 位，太多了一时看不完：
python >>> len(str(2 ** 12345)) # 一个超大的数字有多少位
3717
这种嵌套调用的形式由内向外工作——先计算 ** 的结果，然后用内置函数 str 把它转换成数字字符串，最后用 len 得到结果字符
```

串的长度。最终结果就是这个数字的位数。str 和 len 都能作用于许多对象类型，我们后面还会反复用到它们。除了表达式之外，Python 还随附了一些有用的数字模块（module）——模块只是我们导入使用的一包包附加工具，使用的语句就是在第 3 章介绍过的 import：python >>> import math >>> math.pi 3.141592653589793 >>> math.sqrt(85) 9.219544457292887 math 模块以函数的形式提供更高级的数值工具，而 random 模块负责随机数生成和随机选取（这里是从一个用方括号写的 Python 列表中选取——列表是其他对象的有序集合，本章后面会介绍）：python >>> import random >>> random.random() 0.7082048489415967 >>> random.choice([1, 2, 3, 4]) 2 Python 还包含更奇特的数值对象——如复数、定点精度数和有理数，以及集合和布尔值——而第三方开源扩展领域还有更多（例如矩阵和向量、扩展精度数字）。我们会把对这些对象的讨论推迟到本章和本书的后面。到目前为止，我们使用 Python 的方式和简单计算器差不多；为了更充分地展示它的内置对象目录，让我们接着去探索字符串。

NOTE（大数字与拒绝服务攻击）：Python 整数可以任意大，甚至可用来表示扩展精度的浮点值。但整数大小只受计算机内存限制的同时，Python 3.11 及更高版本要求额外的步骤才能把病态巨大的数字转换成十进制字符串（例如用于 str 或显示）。这是为了避免罕见的拒绝服务（denial-of-service）攻击——有一个 4300 位的限制，可以用 sys 模块的工具 setintmaxstrdigits 解除；调用它就能统计更大数字的位数，详见 Python 官方文档。

深度理解

·核心概念：数字家族图谱——int（整数，任意精度）、float（浮点数，IEEE 754 双精度）、complex（复数）、Decimal（十进制定点/可调精度）、Fraction（有理数）、set（集合）。前三种有字面量语法，后几种需要调用类型或 decimal/fractions 模块。

·底层实现：CPython 中 int 是"变长"对象（PyLongObject，内部按 30 位一组存储数字），所以它没有 C 语言的 32/64 位上限——2 的 12345 次方照算不误，代价只是内存。另外，小整数（-5 到 256）在解释器启动时被缓存在单例， $a = 100$ ； $b = 100$ 时 a 和 b 实际指向同一个整数对象——这是"共享引用"的第一个实例。

·设计原因：任意精度整数把"溢出"这个 C 语言头号 bug 从 Python 中彻底移除；str()/len() 对所有类型通用，体现多态性。数字分隔符 1234567（PEP 515，Python 3.6+）则改善了大数字的可读性。

·实际问题：日常计算用 int/float 即可；涉及钱的精确计算用 Decimal；科学计算、矩阵运算用第三方 NumPy（本书第 1 章提及）；蒙特卡洛模拟用 random。random.choice([1,2,3,4]) 是"从序列随机选一个"的常用套路。

·初学者误区：①以为 int 有位数上限（它没有）；②以为 $0.1 + 0.2 == 0.3$ 成立——浮点数是二进制近似，这个等式是 False，这正是第 5 章将深入讨论的主题（本章先不展开）；③看到 bin(21) 返回 '0b10101'（字符串）时以为数字"变成了字符串"——它只是转换出了一个新的字符

串对象，21 这个整数对象安然无恙。

代码分析

```
>>> 123 + 222                                     #
345
>>> 1.5 * 4                                         #
6.0
>>> 1_234_567, 0x15, bin(21)                       #
(1234567, 21, '0b10101')
>>> 2 ** 100                                         # 2   100
1267650600228229401496703205376
>>> len(str(2 ** 12345))                             #
3717
```

解释器如何处理：123 + 222 编译为两条 LOADCONST + 一条 BINARYADD；执行 BINARYADD 时，虚拟机检查栈顶两个对象是否支持加法（都有 obtype 指向 int 类型，加法槽位 PyNumberAdd 存在），随后用 C 层的大整数加法算出 345，构造新对象压栈。len(str(2 12345)) 是"由内向外"的三层嵌套调用：①2 12345 用快速幂（C 层）算出 3717 位的巨整数对象；②str() 把它转成十进制字符串对象（注意：超过 4300 位的转换在 Python 3.11 + 默认被禁止，见上面的 NOTE）；③len() 取字符串长度返回 3717。中间产生的巨整数、字符串对象随后都会被垃圾回收。

```
>>> import math
>>> math.pi
3.141592653589793
>>> math.sqrt(85)
9.219544457292887
>>> import random
>>> random.random()
0.7082048489415967
>>> random.choice([1, 2, 3, 4])
2
```

·解释器如何处理：import math 触发模块查找（sys.path 搜索）、编译并执行 math 模块的顶层代码，最后把模块对象绑定到名字 math（字节码 IMPORTNAME + STOREGLOBAL math + LOADATTR pi，从模块对象的命名空间字典中取出浮点数。random.choice([1, 2, 3, 4]) 的实参 [1, 2, 3, 4] 是字面量——编译时创建为常量列表，传入函数后由 C 层用随机数索引取值。

·设计思想：模块 = 带命名空间的工具包；"函数式 API"（math.sqrt(x)）与"方法式 API"（S.upper()）并存，是 Python 工具箱的两大形态，本章后面会反复见到。

4.6 字符串 (Strings)

英文原文

Strings are used to record both textual information (your name, for instance) as well as arbitrary collections of bytes (such as an image file's contents). They are

our first example of what in Python we call a sequence—a positionally ordered collection of other objects. Sequences maintain a left-to-right order among the items they contain: their items are stored and fetched by their relative positions. Strictly speaking, strings are sequences of one-character strings; other, more general sequence objects include lists and tuples, covered later.

中文翻译

字符串既用来记录文本信息（比如你的名字），也用来记录任意的字节集合（比如图片文件的内容）。它们是在 Python 中所称的序列（sequence）的第一个例子——序列是按位置有序排列的其它对象的集合。序列在其包含的条目之间维持从左到右的顺序：条目按相对位置存储和获取。严格来说，字符串是单字符字符串的序列；其它更通用的序列对象包括列表（list）和元组（tuple），后面会讲到。

深度理解

·核心概念：字符串 = 序列（有序），且是“字符的序列”。序列的三要素：顺序（从左到右）、位置（偏移量从 0 开始）、通用操作（len、索引、切片、拼接、重复）。字符串是第一个序列，也是理解列表、元组（它们共享同一套序列协议）的跳板。

·底层实现：CPython 3.12 中 str 对象内部按 Unicode 码点存储，根据字符串内容选择紧凑的 1/2/4 字节存储布局（compact ASCII / compact 1-byte / 2-byte / 4-by

te)，同内容短字符串（如'Code'）可能被驻留（interned）共享。len() 走类型槽位 PyObjectSize，O(1) 取得长度。

·设计原因：把"文本"和"字节"都放进字符串这个统一概念里，一个类型两用；把字符串归入序列，则让"按位置取字符"等操作对列表、元组自动复用（多态的又一次体现）。

·实际问题：序列模型是后续一切文本处理（索引、切片、遍历、正则）的基础。你在身份证号、URL、文件路径上的几乎所有操作都是序列操作。

·初学者误区：以为'hÄck' 的长度 = 字符个数在内存里占的字节数——错！长度是码点（code point）个数（5），编码成 UTF-8 后才是 6 字节（Ä 占 2 字节）。"字符数"和"字节数"是两个世界，Unicode 小节会展开。

4.6.1 序列操作（Sequence Operations）

英文原文

As sequences, strings support operations that assume a positional ordering among items. For example, if we have a four-character string coded inside quotes (of the single or double straight kind—they mean the same thing, but single is more common and less busy), we can verify its length with the built-in len function and fetch its components with indexing expressions:

```
python >> S = 'Code' # 生成一个 4 字符字符串，并赋给名字 S
>> len(S) # 长度：字符个数 4
>> S[0] # S 的第一
```

个字符，从 0 开始的按位索引'C'>> S[1] # 从左数第二个字符'o'

In Python, indexes are coded in square brackets as offsets from the front, so start from 0: the first item is at index 0, the second is at index 1, and so on.

Notice how we assign the string to a variable named S here. We'll go into detail on how this works later (especially in Chapter 6), but Python variables never need to be declared ahead of time. A variable is created when you assign it a value, may be assigned any type of object, and is replaced with its value when it shows up in an expression.

It must also have been previously assigned by the time you use its value. For the purposes of this chapter, it's enough to know that we need to assign an object to a variable in order to save it for later use.

In Python, we can also index backward, from the end—positive indexes count forward from the left, and negative indexes count backward from the right. Continuing our REPL session:

```
python >> S[-1] # S 中从末尾数最后一个字符'e'>> S[-2]  
] # 从末尾数倒数第二个字符'd'
```

Formally, a negative index is simply added to the string's length to yield a positive offset, so the following two operations are equivalent (though the first is easier to code and less easy to get wrong):

```
python >> S[-1] # S 中最后一个字符'e'>> S[len(S)-1] #  
负索引的"绕远"写法'e'
```

Notice that we can use an arbitrary expression in the square brackets, not just a hardcoded number literal—anywhere that Python expects a value, we can use a literal, a variable, or any expression we wish. Python's syntax is completely general this way.

In addition to simple positional indexing, sequences also support a more general form of indexing known as slicing, which is a way to extract an entire section (a.k.a. slice) in a single step. For example:

```
python >> S # 一个 4 字符字符串'Code'>> S[1:3] # S  
从偏移 1 到 2 的切片 (不含 3) 'od'
```

Probably the easiest way to think of slices is that they are a way to extract an entire column from a string in a single step. Their general form, $X[l:j]$, means "give me everything in X from offset l up to but not including offset j ." The result is returned in a new object.

The second of the preceding operations, for instance, gives us all the characters in string S from offsets 1 through 2 (that is, 1 through $3 - 1$) as a new string. The effect is to slice or "parse out" the two characters in the middle at offsets 1 and 2.

In a slice, the left bound defaults to zero, and the right bound defaults to the length of the sequence being sliced. This leads to some common usage variation

S:

```
python >> S[1:] # 第一个字符之后的所有内容 (1:len(S))  
) 'ode'>> S # S 本身没有变'Code'>> S[0:3] # 除了最后一个以外的全部'Cod'>> S[:3] # 与 S[0:3] 相同'Cod'>> S[:-1] # 再去掉最后一个, 写法更简 (0:-1) 'Cod'>> S[:] # S 的整体顶层副本 (0:len(S)) 'Code'
```

Note in the second-to-last command how negative offsets can be used to give bounds for slices, too, and how the last operation effectively copies the entire string. As you'll learn later, there is no reason to copy a string, but this form can be useful for other sequences like lists.

Finally, as sequences, strings also support concatenation with the plus sign (joining two strings into a new string) and repetition (making a new string by repeating another):

```
python >> S'Code'>> S + 'xyz' # 拼接'Codexyz'>> S # S 不变'Code'>> S * 8 # 重复'CodeCodeCodeCodeCodeCodeCodeCode'
```

Notice that the plus sign (+) means different things for different objects: addition for numbers, and concatenation for strings. This is a general property of Python that we'll regularly call polymorphism in this book—in short, this means that the meaning of an operation depends on the objects being operated on.

As you'll see when we study dynamic typing, this pol

ymorphism property accounts for much of the conciseness and flexibility of Python code. Because object types aren't constrained, a Python-coded operation can normally work on many different types of objects automatically, as long as they support a compatible interface (like the + operation here). This turns out to be a huge idea in Python; you'll learn more about it later on this tour.

中文翻译

作为序列，字符串支持假设条目间存在位置顺序的操作。例如，如果我们有一个写在引号里的四字符字符串（单引号或双引号都行——它们含义相同，但单引号更常见、更不“忙乱”），可以用内置函数 `len` 验证它的长度，用索引（indexing）表达式获取它的各个组成部分：`python >>> S = 'Code'` # 生成一个 4 字符字符串，并赋给名字 `S` `>>> len(S)` # 长度：字符个数 4 `>>> S[0]` # `S` 的第一个字符，从 0 开始的按位索引 `'C'` `>>> S[1]` # 从左数第二个字符 `'o'` 在 Python 中，索引写在方括号里，表示从前端数起的偏移量，所以从 0 开始：第一个条目在索引 0，第二个在索引 1，依此类推。注意这里我们把字符串赋给了一个叫 `S` 的变量（variable）。我们后面会详细讲它的工作原理（尤其是第 6 章），但 Python 变量永远不需要提前声明。变量在给它赋值时被创建，可以被赋给任何类型的对象，并且在表达式中出现时会被替换为它的值。在你使用它的值之前，它必须已经被赋过值。就本章而言，只需要知道：要把对象保存下来供以后使用，就必须把它赋给一个变量。在 Python 中，我们还可以从末尾向后索引——正索引从左向前数，负索引从

右向后数。继续我们的 REPL 会话：python >>> S[-1] # S 中从末尾数最后一个字符'e'>>> S[-2] # 从末尾数倒数第二个字符'd'正式地说，负索引就是简单地加上字符串的长度，得到一个正偏移，所以下面两个操作是等价的（不过第一个更好写、更不容易写错）：python >>> S[-1] # S 中最后一个字符'e'>>> S[len(S)-1] # 负索引的"绕远"写法'e'注意，方括号里可以使用任意表达式，而不只是写死的数字字面量——凡是 Python 期望一个值的地方，我们都可以使用字面量、变量或任何我们想要的表达式。Python 的语法在这一点上是完全通用的。除了简单的按位索引之外，序列还支持一种更通用的索引形式，叫做切片（slicing）——用一步操作提取整整一段（又称 slice）。例如：python >>> S # 一个 4 字符字符串'Code'>>> S[1:3] # S 从偏移 1 到 2 的切片（不含 3）'od'理解切片最简单的方式大概是：它是一步从字符串中提取整整一"列"的方法。它的一般形式 X[l:j] 的意思是"把 X 中从偏移 l 起、一直到但不包括偏移 j 的内容给我"。结果放在一个新对象里返回。上面第二个操作，比如，把字符串 S 中从偏移 1 到 2（也就是 1 到 3-1）的所有字符作为新字符串给我们。效果就是切下或者说"解析出"偏移 1 和 2 这两个中间字符。在切片中，左边界默认为 0，右边界默认为被切序列的长度。这带来了一些常见用法变体：python >>> S[1:] # 第一个字符之后的所有内容 (1:len(S)) 'ode'>>> S # S 本身没有变'Code'>>> S[0:3] # 除了最后一个以外的全部'Cod'>>> S[:3] # 与 S[0:3] 相同'Cod'>>> S[:-1] # 再去掉最后一个，写法更简 (0:-1) 'Cod'>>> S[:] # S 的整体顶层副本 (0:len(S)) 'Code'注意倒数第二条命令：负偏移也可以用作切片的边界；再看最后一条操作，它实际上是复制了整个字符串。你以

后会学到，复制字符串没有意义，但这种写法对列表等其他序列很有用。最后，作为序列，字符串还支持用加号拼接（concatenation，把两个字符串连成一个新字符串）和重复（repetition，通过重复另一个字符串造出新字符串）：

```
python >>> S'Code'>>> S +'xyz' # 拼接'Codexyz'>>> S  
# S 不变'Code'>>> S 8 # 重复'CodeCodeCodeCodeCodeCodeCodeCodeCodeCode'注意，加号（+）对不同对象意义不同：对数字是加法，对字符串是拼接。这是 Python 的一个通用属性，我们在本书中会经常称之为多态（polymorphism）——简单说，就是操作的含义取决于被操作的对象。当我们学习动态类型时你会看到，这种多态属性在很大程度上造就了 Python 代码的简洁和灵活。因为对象类型不受约束，一段 Python 操作通常能自动作用于许多不同类型的对象——只要它们支持兼容的接口（比如这里的 + 操作）。这是 Python 中一个巨大的思想；你在这次巡游中后面还会了解更多。
```

深度理解

·核心概念：序列操作全家桶——len、正向索引 S[0]、负索引 S[-1]、切片 S[1:3]（左闭右开）、拼接 +、重复。其中左闭右开（含头不含尾）的切片区间是 Python 最独特的约定，也是初学者最早接触的"Python 式"记号。

·底层实现：S[-1] 在运行时被解释为 S[len(S) - 1]——这是解释器在索引处理中自动做的"加长度"运算，所以负数索引 O(1)。切片 S[1:3] 编译为 BUILDSLICE 指令，C 层创建一个新的 str 对象并拷贝对应的码点区间（字符串切片是拷贝不是视图）。S 8 走 BINARYOP，C 层按 8 倍预

分配内存再填充。变量 `S = 'Code'` 编译为 `STORENAME`：在名字表中建立 `S → str` 对象的引用。

·设计原因：左闭右开让 `[ :3 ] + [ 3: ]` 恰好拼回整串、`len(a[:b]) == b`、且负边界 `[:-1]` 表示"去掉最后一项"——数学上自洽，杜绝了 C 语言里"off-by-one"（差一错误）的经典 bug。负索引则让"取最后一个"变成 `S[-1]` 而非 `S[len(S)-1]`，符合直觉。

·实际问题：`S[:]` 复制整串、`S[1:]` 去掉首字符、`S[:-1]` 去掉尾字符、`S[::-1]` 反转（本书后面会讲步长）——一切片是文本处理与数据清洗中最常用的四个姿势。

·初学者误区：①索引越界（`S[4]`）会抛 `IndexError`——Python 不做静默的越界读；②`S[1:3]` 取不到第 3 个字符，很多人刚学时常错把上限当"包含"；③以为切片返回的是原串的"部分视图"——字符串切片返回新对象，改它不会影响原串（况且字符串根本不能改，见下一节）。

代码分析

```
>>> S = 'Code'                                # 4 S
>>> len(S)                                    #
4
>>> S[0]                                       # S 0
'C'
>>> S[1]                                       #
'o'
>>> S[-1]                                      # S
'e'
>>> S[len(S)-1]                               # ""
'e'
>>> S[1:3]                                    # S 1 2 3
'od'
```

```

>>> S[1:]                                #
'ode'
>>> S[:]                                  # S
'Code'
>>> S + 'xyz'                             #
'Codexyz'
>>> S * 8                                 #
'CodeCodeCodeCodeCodeCodeCodeCode'

```

·解释器如何处理：S='Code'：字面量在编译期进入常量表（coconsts），字节码为 LOADCONST'Code' + STORENAME S，此后 S 这个名字在局部名字空间中指向该 str 对象。S[0] 编译为 LOADNAME S + BINARYSUBSCR：虚拟机会先查看索引是否为负数——是则加上序列长度（-1 → 4-1=3），再按正偏移做一次内存定位取字符。S[1:3] 编译为 BUILDSLICE 2（生成 slice 对象）再 BINARYSUBSCR，C 层的 str 切片函数按"1 起、3 止、不含 3"拷贝出'od'——边界越界会被自动"裁剪"到合法范围（S[0:99] 也只给整串）。S + 'xyz' 与 S * 8 都先为结果预分配新内存，再逐段复制——这两个操作不改动 S 指向的原对象，而是生成全新对象。最后，len(S) 直接读对象的内部长度字段，常数时间。

·设计思想："任意表达式都可以出现在方括号里"（S[len(S)-1] 合法）体现了 Python 语法的一致性——任何期待值的位置都接受任何表达式，这条规则贯穿整个语言（参数、索引、切片、推导式四处皆然）。

4.6.2 不可变性 (Immutability)

英文原文

Also notice in the prior examples that we were not changing the original string with any of the operations we ran on it. As it turns out, every string operation is defined to produce a new string as its result, because strings are immutable in Python—they cannot be changed in place after they are created.

More generally, you can never overwrite the values of immutable objects. For example, you can't change a string by assigning to one of its positions, but you can always build a new one and assign it to the same name. Because Python automatically cleans up old objects as you go (as you'll learn later), this isn't as inefficient as it may sound:

```
python >> S='Code'>> S[0] = 'z' # 不可变对象不能被修改
TypeError: 'str' object does not support item assignment
>> S = 'Z' + S[1:] # 但我们可以用表达式构造新对象
>> S'Zode'
```

Every object in Python is classified as either immutable (unchangeable) or not. In terms of the core objects, numbers, strings, and tuples are immutable; but lists, dictionaries, and sets are not—they can be changed in place freely, as can most new objects you'll code with classes. This distinction turns out to be crucial in Python work, in ways that we can't yet fully explore.

Among other things, immutability can be used to gua

rantee that an object remains constant throughout your program; mutable objects' values, by contrast, can be changed at any time and place (and whether your code expects it or not!).

Strictly speaking, you can change text-based data in place if you either expand it into a list of individual characters and join it back together with nothing between, or use the special-purpose bytearray object:

```
python >> S = 'Python'>> L = list(S) # 展开成列表: [...]>> L ['P','y','t','h','o','n']>> L[0] = 'C' # 原地修改>> ''.join(L) # 用空分隔符拼接'Cython'>> B = bytearray(b'application') # bytes 与 list 的混合体 (后文)>> B.extend(b'lication') # 前缀 b 表示字节字符串>> B # 这里 B[i] = ord(x) 也行 bytearray(b'application')>> B.decode() # 翻译回普通字符串'application'
```

The bytearray supports in-place changes for text, but only for text whose characters are all at most 8-bits wide (e.g., ASCII). All other strings are still immutable—bytearray is a distinct hybrid of immutable bytes strings (whose `b'...'` syntax distinguishes them from normal text strings) and mutable lists (coded and displayed in `[]`), but we have to wait until we learn more about both these and Unicode text ahead to fully grasp this code.

中文翻译

还要注意，在前面的例子中，我们运行的任何操作都没有改

变原始字符串。事实证明，每一个字符串操作都被定义为生成一个新的字符串作为结果，因为字符串在 Python 中是不可变 (immutable) 的——创建之后不能就地修改。更一般地说，你永远无法覆盖不可变对象的值。例如，你无法通过给字符串的某个位置赋值来改变它，但你可以构造一个新字符串并赋给同一个名字。因为 Python 会随着你的操作自动清理旧对象（你后面会学到），这并不像听起来那么低效：

```
python >>> S='Code'>>> S[0]='z' # 不可变对象不能被修改
TypeError:'str' object does not support item assignment
>>> S='Z' + S[1:] # 但我们可以用表达式构造新对象
>>> S='Zode'
Python 中的每个对象都被归类为不可变的（不能改变）或不是。就核心对象而言，数字、字符串和元组是不可变的；但列表、字典和集合不是——它们可以随意就地修改，就像你用类编写的大多数新对象一样。这种区分在 Python 工作中至关重要，其影响之大我们现在还无法完全探索。其中之一是：不可变性可以用来保证对象在程序的整个生命周期内保持不变；相比之下，可变对象的值可以在任何时间、任何地点被改变（不管你的代码有没有料到！）。严格来说，如果你把文本数据展开成单个字符的列表，再用空分隔符把它们拼回去，或者使用专用对象 bytearray，你确实可以就地修改文本数据：
```

```
python >>> S='Python'>>> L=list(S) # 展开成列表: [...]
>>> L['P','y','t','h','o','n']
>>> L[0]='C' # 原地修改
>>> ''.join(L) # 用空分隔符拼接
'Cython'
>>> B=bytearray(b'app') # bytes 与 list 的混合体 (后文)
>>> B.extend(b'lication') # 前缀 b 表示字节字符串
>>> B # 这里 B[i] = ord(x) 也行
bytearray(b'application')
>>> B.decode() # 翻译回普通字符串
'application'
bytearray 支持就地修改文本，但
```


只限于所有字符至多 8 位宽（例如 ASCII）的文本。其他所有字符串仍然不可变——`bytearray` 是不可变 `bytes` 字符串（其 `b'...'` 语法把字节串和普通文本字符串区分开）与可变列表（用 `[]` 编写和显示）的独特混合体，但我们必须等到后面更深入地学习了这两者以及 Unicode 文本之后，才能完全理解这段代码。

深度理解

·核心概念：不可变性 = 对象创建后其值永远不能改变。

核心对象中的"铁三角"：数字、字符串、元组不可变；列表、字典、集合可变。注意"不可变"指的是对象本身不能变，而不是名字不能变——`S = 'Zode'` 只是让 `S` 指向了新对象，旧的 `'Code'` 对象并未被"改写"。

·底层实现：CPython 里不可变对象通常没有"修改"的字节码操作：`S[0] = 'z'` 编译为 `BINARYSUBSCR + STORESUBSCR`，执行时 VM 检查 `str` 类型没有 `item` 赋值槽位（`sqassitem` 为 `NULL`），直接抛 `TypeError`。相反，`list` 实现了该槽位，所以 `L[0] = 'C'` 合法且直接改写同一块内存。`'Z' + S[1:]` 生成新对象后，旧对象引用计数减一（原 `S` 的引用被替换），计数归零即被回收——这正是"改字符串不低效"的原因：垃圾回收（reference counting）自动处理了旧对象。

·设计原因：不可变对象可以安全地被到处共享：作字典的键（后面学）、作集合元素（元素必须可哈希）、被多个变量同时引用而无需担心"这个改了影响那个"。可变对象（列表、字典）则牺牲了这种安全，换来的是原地增删改

的性能和便利。"能不能改"的区分，本质是"安全共享"与"原地操作"之间的权衡。

·实际问题：list(S) + ''.join(...) 是"把字符串当字符列表改"的经典技巧；bytearray 是处理二进制协议时的高效工具。实际编码中，你不需要频繁"改字符串"——因为 Python 自动回收旧对象，S = S[1:] 之类的重建是惯用法，不必优化。

·初学者误区：①见到 S[0] = 'z' 报 TypeError 就以为"字符串不能赋值"——准确说是对象不允许被就地修改；②以为 S = 'Z' + S[1:] 是在"修改字符串"——不，这是让名字指向新对象，旧对象被回收；③以为"不可变 = 不能变"，于是惊讶于 x = 5; x += 1 合法——变量可以重新绑定，数字对象本身才是不可变的。

代码分析

```
>>> S[0] = 'z'                                     #
TypeError: 'str' object does not support item assignment
>>> S = 'Z' + S[1:]                                #
>>> S
'Zode'

>>> S = 'Python'
>>> L = list(S)                                     #
>>> L
['P', 'y', 't', 'h', 'o', 'n']
>>> L[0] = 'C'                                     #
>>> ''.join(L)                                     #
'Cython'

>>> B = bytearray(b'app')                          # bytearraybytes list
>>> B.extend(b'lication')                          # b =
```

```
>>> B
bytearray(b'application')
>>> B.decode()                                #
'application'
```

·解释器如何处理：S[0] = 'z' 执行时，虚拟机的 STORESU BSCR 例程查询 str 类型对象的"条目赋值"槽位——不存在，于是抛出 TypeError。这是强类型在运行时的一次具体体现：类型不支持的操作，执行期直接报错，绝不静默。L[0] = 'C' 则是真正的原地写内存：list 的 C 数组第 0 个槽位被替换为指向'C' 的引用，L 对象自始至终是同一个。list(S) 把字符串逐字符拆成 6 个新字符串对象装进新列表；''.join(L) 在 C 层一次性分配 6 字节空间把字符拷出，再包成新 str。bytearray 则更接近"可变字节数组"：extend 直接扩容底层字节缓冲区；decode() 按默认编码（UTF-8）把字节解码成 str。

·设计思想：可变与不可变的对比在同屏上演——S[0]='z' 失败、L[0]='C' 成功。这正是 Python"不可变对象靠重建、可变对象靠原地改"的双轨设计。bytearray 的存在说明 Python 不搞教条：真需要可变字节时，给你一个专门的工具。

4.6.3 类型特有方法 (Type-Specific Methods)

英文原文

Every string operation we've studied so far is really a sequence operation—that is, these operations will work on other sequences in Python as well, including lists and tuples. In addition to generic sequence opera

tions, though, strings also have operations all their own, available as methods—functions that are attached to and act upon a specific object, which are triggered with a call expression using parentheses.

For example, the string `find` method is the basic substring search operation (it returns the offset of the passed-in substring, or `-1` if it is not present), and the string `replace` method performs global searches and replacements; both act on the subject that they are attached to and called from:

```
python >> S = 'Code'>> S.find('od') # 查找子串在 S 中的偏移 1
>> S'Code'>> S.replace('od','abl') # 把 S 中所有子串'od' 替换为'abl'
'Cable'>> S'Code'
```

Again, despite the names of these string methods, we are not changing the original strings here but creating new strings as the results—because strings are immutable, this is the only way this can work. String methods are the first line of text-processing tools in Python.

Other methods split a string into substrings on a delimiter (handy as a simple form of parsing), perform case conversions, test the content of the string (digits, letters, and so on), and strip whitespace characters off the ends of the string:

```
python >> line = 'aaa,bbb,cccc,dd'>> line.split(',') # 按分隔符拆成子串列表 ['aaa','bbb','cccc','dd']
>> S = 'code'>> S.upper() # 大小写转换'CODE'>> S.isalpha() #
```

内容测试: isalpha、isdigit 等 True >> line = 'aaa,bbb,cccc,dd\n'>> line.rstrip() # 去掉右侧空白字符'aaa,bbb,cccc,dd'>> line.rstrip().split(',') # 组合两个操作, 从左到右执行 ['aaa','bbb','cccc','dd']

Notice the last command here—it strips before it splits because Python runs it from left to right, making a temporary result along the way. You can chain method calls this way, as long as the prior call returns an object with methods.

Strings methods are also one way to run an advanced substitution operation known as formatting, available as an expression (the original), a string method call (newer), and a literal form called f-strings (newest). The first two replace keys with separate values, the third replaces embedded Python expressions with their results, and all three resolve substitution values and build new strings when they are run:

```
python >> tool = 'Python'>> major = 3 >> minor = 3
>> 'Using %s version %s.%s' % (tool, major, minor +
9) # 格式化表达式'Using Python version 3.12'>> 'Using
{} version {}.{}'.format(tool, major, minor + 9) # 格式化
方法'Using Python version 3.12'>> f'Using {tool} versi
on {major}.{minor + 9}' # 格式化字面量 (f-string) 'Usi
ng Python version 3.12'
```

And yes, this comes in three flavors today. While you may want to pick one for your own code, all three are fair game in code you may read (and inclusive books)

. Each form is rich with features, which we'll postpone discussing until later in this book, and which tend to matter most when you must generate readable output and numeric reports:

```
python >> '%.2f | % +05d' % (3.14159, -62) # 小数位数、正负号、补零'3.14 | -0062'>> '{1:,.2f} | {0}'.format('sapp'[1:], 296999.256) # 千位分隔、小数位数'296,999.26 | sapp'>> f'{296999.256:,.2f} | {'sapp'[1:]}' # 同上，用嵌套引号'296,999.26 | sapp'
```

Although sequence operations are generic, methods are not—while some objects share some method names, string method operations generally work only on strings, and nothing else.

As a rule of thumb, Python's toolset is layered: generic operations that span multiple object types show up as built-in functions or expressions (e.g., `len(X)`, `X[0]`), but type-specific operations are method calls (e.g., `a String.upper()`). Finding the tools you need among all these categories will become more natural as you use Python, but the next section gives a few tips you can use right now.

中文翻译

到目前为止我们学过的每一个字符串操作其实都是序列操作——也就是说，这些操作在 Python 的其他序列上也能用，包括列表和元组。不过，除了通用的序列操作，字符串还有自己独有的操作，以方法（method）的形式提供——方

法是附着在特定对象上并作用于该对象的函数，用带括号的调用表达式触发。例如，字符串的 `find` 方法是基本的子串搜索操作（它返回传入子串的偏移，若不存在则返回 `-1`），字符串的 `replace` 方法执行全局搜索与替换；两者都作用于它们所附着、所调用的那个主语对象上：

```
python >>> S='Code'>>> S.find('od') # 查找子串在 S 中的偏移1
>>> S'Code'>>> S.replace('od','abl') # 把 S 中所有子串'od' 替换为'abl'
Cable'>>> S'Code'
```

再说一次：尽管这些字符串方法的名字听起来像在“改”，我们并没有改变原始字符串，而是创建了新字符串作为结果——因为字符串不可变，这是唯一可行的方式。字符串方法是 Python 文本处理工具的第一梯队。其他方法还有：按分隔符把字符串拆成子串（作为简单的解析手段非常方便）、大小写转换、测试字符串内容（数字、字母等）、去掉字符串两端的空白字符：

```
python >>> line='aaa,bbb,cccc,dd'>>> line.split(',')
# 按分隔符拆成子串列表['aaa','bbb','cccc','dd']
>>> S='code'>>> S.upper() # 大小写转换'CODE'
>>> S.isalpha() # 内容测试：isalpha、isdigit 等True
>>> line='aa
a,bbb,cccc,dd\n'>>> line.rstrip() # 去掉右侧空白字符'
aaa,bbb,cccc,dd'>>> line.rstrip().split(',') # 组合两个
操作，从左到右执行['aaa','bbb','cccc','dd']
```

注意这里的最后一条命令——它先 `strip` 再 `split`，因为 Python 从左到右执行，沿途产生临时结果。只要前面的调用返回的是带方法的对象，你就能这样链式调用。字符串方法也是执行一种高级替换操作——格式化（formatting）——的途径之一，它有三种形态：表达式（最古老）、字符串方法调用（较新）、以及叫 `f-string` 的字面量形式（最新）。前两种用独立的值替换键，第三种用内嵌 Python 表达式的结果替

换，三种形式都在运行时解析替换值并构建新字符串：python >>> tool = 'Python'>>> major = 3>>> minor = 3>>>'Using %s version %s.%s' % (tool, major, minor + 9) # 格式化表达式'Using Python version 3.12'>>>'Using {} version {}.{}'.format(tool, major, minor + 9) # 格式化方法'Using Python version 3.12'>>> f'Using {tool} version {major}.{minor + 9}' # 格式化字面量 (f-string) 'Using Python version 3.12'是的，如今它有三种口味。你可以在自己的代码里选一种，但三种形式在你读到的代码里（以及兼容并蓄的书籍里）都是正当合法的。每种形式都功能丰富，我们会推迟到本书后面再讨论，当需要生成可读的输出和数值报表时它们才最要紧：python >>>'%.2f | %+05d' % (3.14159, -62) # 小数位数、正负号、补零'3.14 | -0062'>>>'{:1:,.2f} | {}'.format('sapp'[1:], 296999.256) # 千位分隔、小数位数'296,999.26 | app'>>> f'{296999.256:,.2f} | {'sapp'[1:]}' # 同上，用嵌套引号'296,999.26 | app'虽然序列操作是通用的，方法却不是——虽然有些对象会共享一些方法名，但字符串方法一般只作用于字符串，别的什么也不行。按经验法则，Python 的工具箱是分层的：跨多个对象类型的通用操作表现为内置函数或表达式（例如 len(X)、X[0]），而类型特有的操作是方法调用（例如 aString.upper()）。随着你使用 Python，在这些类别中找到你需要的工具会越来越自然，不过下一节先给你几个立即可用的技巧。

深度理解

·核心概念：方法的本质是“挂在对象上的函数”。S.find('od') 就是“对对象 S 调用它的 find 工具”。通用操作 (len

、索引、切片)与类型特有操作 (upper、split) 分开摆放: 通用的是内置函数/表达式, 特有的是方法——这是 Python 工具箱的分层设计。

·底层实现: 方法调用 `S.find('od')` 的字节码是 `LOADMET HOD find + CALLMETHOD 1`。执行时先在 `str` 对象 (严格说是其类型对象) 的属性字典里查到名为 `find` 的函数 (C 函数), 然后"绑定"到 `S` 上调用——C 层实现中, `S` 会作为第一个参数 `self` 传入。所以 `S.find('od')` 与 `str.find(S,'od')` 完全等价 (这是很多老手会用的一种"去糖"写法)。链式调用 `line.rstrip().split(',')` 按从左到右求值: 先执行 `line.rstrip()` 得到临时字符串对象, 再对临时对象调用 `.split(',')`。

·设计原因: 把共性操作做成内置函数、特性操作做成方法, 符合"对象只暴露它该有的接口": `len(X)` 对所有序列通用, `S.upper()` 只属于字符串——你在一个数字对象上不可能看到 `.upper()`, 也就不会误用。方法名空间 (attributes) 也顺带解决了"内置函数名太多"的命名冲突问题。

·实际问题: `line.rstrip().split(',')` 是解析文本行的标准姿势 (去掉行尾换行再按逗号拆列); `.isalpha()/isdigit()` 是输入校验 (判断"是不是字母/数字") 的常用工具; `find` 返回 `-1` 的约定需要记住——它不抛异常, 要靠返回值判断"没找到"。

·初学者误区: ①以为 `find` 找不到会抛异常——不, 它返回 `-1`, 很多 bug 来自忘了检查 `-1`; ②以为 `S.upper()` "把 `S` 变大了"——不, 它返回新串, `S` 不变; ③三种格式化都能用, 初学者常纠结选哪个——新代码用 f-string 即

可，但读老代码时三种都要认识。

代码分析

```
>>> S = 'Code'
>>> S.find('od')                                # 'od' S
1
>>> S.replace('od', 'abl')                      # S 'od' 'abl'
'Cable'
>>> line = 'aaa,bbb,ccccc,dd'
>>> line.split(',')                            # ','
['aaa', 'bbb', 'ccccc', 'dd']
>>> S = 'code'
>>> S.upper()                                  #
'CODE'
>>> S.isalpha()                                #
True
>>> line = 'aaa,bbb,ccccc,dd\n'
>>> line.rstrip()                              #
'aaa,bbb,ccccc,dd'
>>> line.rstrip().split(',')                   # strip split
['aaa', 'bbb', 'ccccc', 'dd']
```

·解释器如何处理：S.find('od')：LOADNAME S 取到 str 对象 → LOADMETHOD find 找到方法对象（并隐式记住主语 S）→ LOADCONST'od' 压入实参 → CALLMETHOD 1 真正调用。C 层实现是一个字符串搜索算法（CPython 的 find 使用快速子串搜索），返回命中位置的偏移 1；未命中则返回 -1（S.find('x') → -1）。replace('od','abl') 则是先扫描所有命中位置，再一次性分配新内存、逐段拷贝替换后的内容，返回新串'Cable'——两个操作都不触碰原对象。line.rstrip().split(',') 分两段求值：第一段 rstrip() 裁掉尾部 \n 得到临时串，第二段 split(',') 在该临

时串上按逗号切分，返回列表。链式调用能成功，是因为 `rstrip` 返回的是 `str`，而 `str` 恰好还有 `split` 方法。

·设计思想：方法调用 = “名词 + 动词”（主语 S + 行为 `find/replace`），代码读起来像英语句子；链式调用则像流水线作业，每个环节的输出恰好是下一环节的输入。这比 C 里的 `strstr(S, "od")` 式的函数套函数可读得多。

```
>>> tool = 'Python'
>>> major = 3
>>> minor = 3
>>> 'Using %s version %s.%s' % (tool, major, minor + 9)      # ...
'Using Python version 3.12'
>>> 'Using {} version {}.{}'.format(tool, major, minor + 9)  # ...
'Using Python version 3.12'
>>> f'Using {tool} version {major}.{minor + 9}'              # ...
'Using Python version 3.12'
>>> '%.2f | %+05d' % (3.14159, -62)                          # ...
'3.14 | -0062'
>>> '{1:,.2f} | {}'.format('sapp'[1:], 296999.256)          # ...
'296,999.26 | app'
>>> f'{296999.256:,.2f} | {'sapp'[1:]}'                      # ...
'296,999.26 | app'
```

·解释器如何处理：三种格式化最终殊途同归：`%` 格式化在 C 层按 `%s/%.2f` 等说明符逐个把值转成字符串并插入；`str.format` 则解析花括号字段 `{1:,.2f}`（1 = 位置 1 的参数，`,.2f` = 千位分隔 + 两位小数），再把参数格式化后填入；f-string 在编译期被解析——`{major}` 之类会编译成独立的取值/表达式指令嵌进常量字符串的格式化流水线，因此 f-string 中的表达式是真正的 Python 表达式（这里 `minor + 9` 先算出 12，`'sapp'[1:]` 先切出 `'app'`）。Python 3.12+ 允许 f-string 里嵌套同种引号（如 `f'...{'sapp'['`

1:}]...}')) , 本书第六版正是基于 3.12 写的。

·设计思想：从 % 到 .format() 到 f-string 是"越来越内嵌"的演进：值的位置从括号参数、到花括号字段、再到直接在字面量里写表达式。新代码用 f-string，老代码三种并存。

4.6.4 获取帮助 (Getting Help)

英文原文

The methods introduced in the prior section are a representative, but small, sample of what is available for string objects. In general, this book is not exhaustive in its coverage of object methods, but for more details, you can always call the built-in `dir` function. This function lists variables assigned in the caller's scope when called with no argument; more usefully, it returns a list of all the attributes available for any object passed to it. Because methods are callable attributes, they will show up in this list. Assuming `S` is still the string `'Code'`, here are its attributes:

```
python >> dir(S) ['add','class','contains','delattr','dir','doc','eq','format','ge','getattr','getitem', ...中间部分省略...'rindex','rjust','rpartition','rsplit','rstrip','split','splitlines','startswith','strip','swapcase','title','translate','upper','zfill']
```

Run this live for the full list; its middle was cut at ...etc... for space here (strings have 81 attributes today!).

The names without underscores in the second half of this list are all the callable methods on string objects. The names with double underscores won't be important until later in the book, when we study operator overloading in classes. In short, they represent the implementation of the string object and support customization. The `add` method of strings, for example, is how concatenation ultimately works; Python maps the first of the following to the second internally, though you shouldn't usually use the second form yourself (it's less intuitive, and might even run slower):

```
python >> S + 'head!' 'Codehead!'
```

```
>> S.add('head!') 'Codehead!'
```

The `dir` function simply gives methods' names. To ask what they do, you can pass them to the `help` function:

```
python >> help(S.replace) # 或者 help(str.replace)
Help on built-in function replace:
replace(old, new, count=-1, /) method of builtins.str instance
Return a copy with all occurrences of substring old replaced by new
...

```

Press the `Q` key to exit a help display in most console windows. `help` is one of a handful of interfaces to a system of code that ships with Python known as `Pydoc`—a tool for extracting documentation from objects. Later in the book, you'll see that `Pydoc` can also render its reports in HTML format for display in a web browser.

rowser.

You can also ask for help on an entire string with `help(S)`, but it may not help: the string's value is used instead of the string itself, unless it's empty. To do better, you need to know either the name of the string type, or that the type built-in returns any object's type as another object: `help(str)` and `help(type(S))` both give help for strings but may be more than you want—because they describe every method, help may be best used on specific methods.

For more info, you can also consult Python's standard-library reference manual, but `dir` and `help` are the first level of documentation in Python, especially when working in an interactive REPL. Try them soon on an object near you.

中文翻译

上一节介绍的方法只是字符串对象可用方法的代表性小样本。总的来说，本书对对象方法的覆盖并不详尽，但想要更多细节时，你随时可以调用内置函数 `dir`。不带参数调用时，它列出调用者作用域中已赋值的变量；更有用的是，它会返回传入的任何对象的所有可用属性列表。因为方法是可调用的属性，它们会出现在这个列表里。假设 `S` 还是字符串 `'Code'`，它的属性如下：`python >>> dir(S) ['add','class','contains','delattr','dir','doc','eq','format','ge','getattribute','getitem',...中间部分省略...'rindex','rjust','rpartition','rsplit','rstrip','split','splitlines','startswith','strip','swapcase','title','translate','upper','zfill']` 自己运行一下就能看

到完整列表；中间部分为了省篇幅被...等...截掉了（如今的字符串有 81 个属性！）。列表后半部分不带下划线的名字都是字符串对象上可调用的方法。带双下划线的名字要等到本书后面学习类中的操作符重载（operator overloading）时才重要。简而言之，它们代表字符串对象的内部实现，并支持定制化。例如，字符串的 `add` 方法就是拼接最终的工作方式；Python 内部会把下面的第一种写法映射为第二种，不过你通常不应该自己用第二种（它不够直观，甚至可能更慢）：`python >>> S + 'head!'``Codehead!>>> S.add('head!')``Codehead!` `dir` 函数只给出方法的名字。想知道它们是干什么的，可以把它们传给 `help` 函数：`python >>> help(S.replace)` # 或者 `help(str.replace)` Help on built-in function replace: `replace(old, new, count=-1, /)` method of builtins.str instance Return a copy with all occurrences of substring old replaced by new....在大多数控制台窗口里按 Q 键退出 `help` 显示。`help` 是与随 Python 发行的文档提取系统 `Pydoc` 交互的几个接口之一——`Pydoc` 是一个从对象中提取文档的工具。在本书后面你会看到，`Pydoc` 还能把它的报告渲染成 HTML 格式，显示在网页浏览器中。你也可以对整个字符串调用 `help(S)` 来求助，但可能没啥用：除非字符串为空，否则显示的是字符串的值而不是字符串本身。想做得更好，你需要知道字符串类型的名字，或者知道内置 `type` 能把任何对象的类型作为另一个对象返回：`help(str)` 和 `help(type(S))` 都能给出字符串的帮助，但可能内容过多——因为它们描述了每个方法，`help` 最好用在具体方法上。想了解更多，你还可以查阅 Python 的标准库参考手册，但 `dir` 和 `help` 是 Python 中第一层文档，尤其在交互式 REPL 中工作时。尽快在你身

边的某个对象上试试它们。

深度理解

·核心概念：dir 看"有什么"，help 问"怎么用"——这是 REPL 时代的两大自文档化 (self-documenting) 工具。Python 语言自带"查看对象属性"和"查询文档"的能力，不用翻书就能探索任何对象。

·底层实现：dir(S) 的底层是遍历对象的属性字典 + 类型对象的属性字典（对内置类型还包括 C 层 slot 表），汇总成排序列表。双下划线名字是特殊方法 (special method, 俗称 dunder)：它们不是给人直接调的 API，而是 Python 操作符/内置函数在 C 层的"挂钩"。S + 'head!' 的字节码是 BINARYOP +，虚拟机实际执行的就是 S.add('head!')（查 str 类型对象里的 add 槽位）。这就是作者说的"第一种被映射到第二种"。

·设计原因：特殊方法体系让操作符语法 (+、[], len()) 和类自定义能力统一起来——内置类型用它们实现操作符，以后你写的类也能靠实现 add 获得 +。先认识这套命名规则 (xxx)，学 OOP 时不至于一头雾水。

·实际问题：在 REPL 里 dir(str) 一眼扫尽字符串全家福；help(str.split) 快速查签名；不知道一个对象的类型就 type(x)。这三个命令是排查问题的第一梯队。

·初学者误区：①把 help(S) 当"字符串的帮助"——实际上传的是值，应该用 help(str) 或 help(type(S))；②以为可以随便调用 S.add——能跑但没必要，直接 + 更快更清晰；③以为 dir(S) 里的 ... 都是"高级功能"——它们多数

是解释器内部挂钩，普通编程很少直接碰。

代码分析

```
>>> dir(S)                                     # S
['__add__', '__class__', '__contains__', ..., 'split', 'startswith',
'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']

>>> S + 'head!'                                #
'Codehead!'

>>> S.__add__('head!')                         #
'Codehead!'

>>> help(S.replace)                            #
Help on built-in function replace:
replace(old, new, count=-1, /) method of builtins.str instance
    Return a copy with all occurrences of substring old replaced by n...
```

·解释器如何处理：dir(S) 由 C 层实现：合并对象的命名空间与类型命名空间，输出排序后的名字列表——不带下划线的是普通方法，带双下划线的是特殊方法。S + 'head!' 走 BINARYOP 指令：虚拟机按 C 层操作符分发表（type slots 表）找到 str 的 concat 实现（即 add 对应槽位）并执行；S.add('head!') 则是普通属性查找 + 调用，路径更长、还多一层 Python 调用开销，所以作者警告“可能更慢”。help(S.replace) 调用 Pydoc：读取方法对象的 doc 字符串并格式化为分页文档显示（按 Q 退出）。

·设计思想：+ 与 add 的等价性揭示了一个深刻事实：Python 的语法糖背后是普通对象方法。内置对象如此，未来你自己的类只要实现 add，+ 也能对你自定义的对象工作——这正是操作符重载。

4.6.5 字符串的其他写法 (Other Ways to Code Strings)

英文原文

So far, we've looked at the string object's sequence operations and type-specific methods. Python also provides a variety of ways for us to code strings, which we'll explore in greater depth later. For instance, special characters can be represented as backslash escape sequences, which Python displays in `\xNN` hexadecimal escape notation, unless they stand for printable characters:

```
python >> S = 'A\nB\tC' # 转义: \n 是换行, \t 是制表符
>> len(S) # 每个转义只算一个字符 5
>> S = 'A\0B\0C' # \0 是二进制零字节, 不会终止字符串
>> len(S) 5
>> S # 不可打印字符显示为 \xNN 十六进制转义 'A\x00B\x00C'
```

As hinted earlier, Python allows strings to be enclosed in single or double quote characters—they mean the same thing but allow the other type of quote to be embedded without an escape (most programmers prefer single quotes for less clutter, unless they're pinning for other-language pasts).

You can also code multiline string literals enclosed in triple quotes (single or double)—when used, all the lines are concatenated together, and newline characters (`\n`) are added where line breaks appear. This is us

eful for embedding things like multiline HTML, YAML, or JSON code in a Python script, as well as stubbing out lines of code temporarily—just add three quotes above and below:

```
python >> msg = """aaaaaaaaaaaaa bbb'''bbb'''bbb c
cccccc

""" >> msg'\naaaaaaaaaaaaaa\nbbb\'\'\'bbb'''bbb\nccc
cccc\n'
```

Python also supports a raw string literal that turns off the backslash escape mechanism. Such literals start with the letter `r` and are useful for strings like regular-expression patterns, and directory paths on Windows sans doubled-up backslashes (e.g., `r'C:\Users\you\code'`).

中文翻译

到目前为止，我们已经看了字符串对象的序列操作和类型特有方法。Python 还提供了多种编写字符串的方式，我们后面会深入探索。例如，特殊字符可以用反斜杠转义 (backslash escape) 序列来表示，Python 会把它们显示为 `\xNN` 十六进制转义记号（除非它们代表可打印字符）：`python >>> S = 'A\nB\tC' # 转义：\n 是换行，\t 是制表符 >>> len(S) # 每个转义只算一个字符 5 >>> S = 'A\0B\0C' # \0 是二进制零字节，不会终止字符串 >>> len(S) 5 >>> S # 不可打印字符显示为 \xNN 十六进制转义 'A\x00B\x00C'`正如前面暗示过的，Python 允许字符串用单引号或双引号包围——它们含义相同，但可以免转义地嵌入另一种引号（

多数程序员偏爱单引号以减少凌乱，除非他们怀念其他语言的习惯）。你还可以用三引号（单或双）编写多行字符串字面量——使用时，所有行被连接在一起，并在出现换行的地方添加换行符（\n）。这在把多行 HTML、YAML 或 JSON 代码嵌入 Python 脚本时很有用，也适合临时注释掉几行代码——上下各加三个引号即可：

```
python >>> msg =  
""" aaaaaaaaaaaa bbb'bbb"bbb cccccccc """ >>> m  
sg\'naaaaaaaaaaaa\nbbb\'\'\'bbb"bbb\ncccccccc\n'
```


Python 还支持关闭反斜杠转义机制的原始字符串字面量（raw string literal）。这种字面量以字母 r 开头，对正则表达式模式这样的字符串，以及 Windows 上不必写双反斜杠的目录路径（例如 r'C:\Users\you\code'）非常有用。

深度理解

·核心概念：字符串字面量的四种写法——单/双引号（等效）、三引号（多行）、r 前缀（原始字符串，关闭转义）、以及后面小节将讲的 b 前缀（字节串）和 \u 转义（Unicode）。转义（escape）是“用几个字符代表一个特殊字符”的记号法。

·底层实现：'A\nB\tC' 在编译期由词法分析器处理：\n、\t 是已定义好的转义序列，直接替换为对应的码点（换行 U+000A、制表 U+0009）。所以它只有 5 个字符（A、换行、B、制表、C）而不是 7 个。'\0' 是真正的零字节（U+0000）——与 C 语言不同，Python 字符串不以 \0 结尾，长度照实计算（5）。三引号字面量在词法阶段把物理换行转成 \n 字符。r'...' 前缀让词法分析器跳过转义处理，\n 原样保留为两个字符。

·设计原因：转义是为了"在源码里写特殊字符"；原始字符串是为了"写正则/路径时不想被转义烦扰"；三引号为了多行文本与临时注释。每种植被对应一种实际需求，按需选用。

·实际问题：Windows 路径用 `r'C:\Users\you\code'`（否则 `\U` 会被当成 Unicode 转义报错！）；正则表达式用 `r'\d+'`；多行 JSON/HTML/SQL 用三引号。`'\0'` 不终止字符串意味着 Python 能安全处理含零字节的二进制文本数据。

·初学者误区：①Windows 路径里写 `'C:\Users\QK'` 会因 `\U` 不是合法转义而报错——要么用 `r` 前缀要么用双反斜杠；②以为 `len('A\nB\tC')` 是 7——转义序列占 2 个源码字符但只算 1 个字符；③以为三引号里可以随便写任何引号——同种三引号对（`"""..."""` 里不能再直接写 `"""`）要小心。

代码分析

```
>>> S = 'A\nB\tC'                                # \n \t
>>> len(S)                                         #
5
>>> S = 'A\0B\0C'                                # \0
>>> len(S)
5
>>> S                                              # \xNN
'A\x00B\x00C'

>>> msg = """                                     #
aaaaaaaaaaaaa
bbb' ' 'bbb""bbb
cccccccc
```

```
"""
```

```
>>> msg
```

```
# \n
```

```
'\naaaaaaaaaaaa\nbbb\'\'\'bbb"'bbb\ncccccccc\n'
```

·解释器如何处理：编译器的词法阶段是这里的“翻译官”：'\n' 在 token 化时就被转换成码点 10，'\t' 转成 9，'\0' 转成 0——字符串对象里存的是转换后的码点序列，len() 数的是码点（5）。repr 显示时反向操作：可打印字符原样显示（A、B、C），不可打印的（换行、制表、零字节）用 \xNN 十六进制转义显示（\x00、\x0a、\x09 等）。三引号字面量则在词法阶段把“物理行尾”统一转成 \n 字符（跨平台：Windows 的 \r\n 在源文件里也会被规范为 \n），所以 repr 里出现'\n...\n'。原始字符串 r'\n' 则会跳过这一切，得到反斜杠 + n 两个字符。

·设计思想：显示与存储分离——对象内部是码点数组，repr 只是“人读的投影”。这解释了为什么 print(S) 与直接敲 S 显示不同（print 走 str()，直接敲走 repr()）：前者解释控制字符，后者转义显示。

4.6.6 Unicode 字符串 (Unicode Strings)

英文原文

Python's strings also come with full Unicode support required for processing non-ASCII text. Such text includes characters in non-English languages, as well as symbols and emojis, and is common today in web pages, emails, GUIs, documents, and data.

Python's string objects let you process such text sea

mlessly: its normal str string handles Unicode text (including ASCII, which is just a simple kind of Unicode); and its bytes string, together with its bytearray mutable cousin used earlier, handles raw byte values (including media and encoded text):

```
python >> 'hÄck' # 普通 str 字符串就是 Unicode 文本'hÄck'>> b'a\x01c' # bytes 字符串是基于字节的数据 b'a\x01c'
```

Formally, Python's byte strings are sequences of 8-bit bytes that print with ASCII characters when possible, and its text strings are sequences of Unicode code points—identifying numbers for characters, which print as the usual glyphs we've come to know and do not necessarily map to single bytes when stored in memory or encoded in files.

In fact, the notion of bytes doesn't quite apply to Unicode: it includes a host of character code points too large to fit in a byte, and defines encodings for storage and transmission in which characters may be any size at all:

```
python >> 'Code' # 字符在内存中可以是任意大小'Code'>> 'Code'.encode('utf-8') # 在文件中按 UTF-8 编码为 4 字节 b'Code'>> 'Code'.encode('utf-16') # 但按 UTF-16 编码则是 10 字节 b'\xff\xfeC\x00o\x00d\x00e\x00'
```

This is especially true for richer text (ord gives a character's code point, and hex gives its hexadecimal string):

```
python >> hex(ord('')) # 大到一个字节装不下的码点'0
x1f40d'>> len('hÄck') # 每个字符（码点）各算一个 6
>> len('hÄck'.encode('utf-8')) # 但编码后的字节数各
不相同 13 >> len('hÄck'.encode('utf-16')) 18
```

To code non-ASCII characters in text strings, use `\x` hexadecimal escapes; short `\u` or long `\U` Unicode escapes; or raw text interpreted per source encodings optionally declared in program files when code is read (the default for code is the universal UTF-8). To illustrate, here's our non-ASCII A-with-diaeresis character `Ä` coded four ways in Python:

```
python >> 'h\xc4\u00c4\U000000c4Äck' # 非 ASCII 字
符的四种写法: hex、短、长、原生'hÄÄÄÄck'
```

In text strings, all these forms specify Unicode code points that stand for characters. By contrast, byte strings use only `\x` hexadecimal escapes to embed the values of raw bytes; this isn't always text, but when it is, it's the encoded form of text, not its decoded code points, and encoded bytes are the same as code points only for simple text and encodings:

```
python >> '\u00A3', '\u00A3'.encode('latin1'), b'\xa3'.
decode('latin1') ('£', b'\xa3', '£')
```

Apart from these string types, Unicode processing of text reduces to transferring text data to and from files—which automatically encode text to bytes when stored in a file and decode it to characters (a.k.a. code points) when read back into memory. Once loaded, we

usually process text as strings in decoded form only. To make this work, text files implement encodings and accept and return text strings, but binary files instead deal in bytes strings for raw data.

You'll meet Unicode again in the files coverage later in this chapter, but we will save the rest of the Unicode story for later in this book. It crops up briefly in Chapters 7, 9, and 15, but for the most part is postponed until this book's advanced topics part, in Chapter 37. Unicode is crucial in many (or most) domains today, but many Python newcomers can get by with just a passing acquaintance until they've mastered string basics.

In addition to its built-in string objects, Python's standard toolset includes support for text pattern matching with its `re` module, as well as parsing textual data like JSON, CSV, XML, and HTML. You'll meet additional examples of some of these tools later in this book, but this tutorial intro has already said enough about strings and must move on.

中文翻译

Python 的字符串还带有处理非 ASCII 文本所需的完整 Unicode 支持。这类文本包括非英语语言的字符，以及符号和 emoji，如今在网页、电子邮件、GUI、文档和数据中都很常见。Python 的字符串对象让你无缝处理这类文本：普通的 `str` 字符串处理 Unicode 文本（包括 ASCII——ASCII 只是 Unicode 的一种简单情形）；而 `bytes` 字符串，连同

前面用过的可变的 bytearray 表亲，处理原始字节值（包括媒体和已编码文本）：`python >>> 'hÄck' # 普通 str 字符串就是 Unicode 文本'hÄck'>>> b'a\x01c' # bytes 字符串是基于字节的数据b'a\x01c'`正式地说，Python 的字节字符串是 8 位字节的序列，能显示成 ASCII 字符时就用 ASCII 显示；它的文本字符串是 Unicode 码点（code point）的序列——码点是标识字符的数字，显示为我们熟悉的字形，在内存中存储或编码进文件时不一定映射为单个字节。事实上，字节这个概念对 Unicode 不太适用：Unicode 包含大量大到一个字节装不下的字符码点，并且为存储和传输定义了编码方式，在这些编码里字符可以是任意大小：`python >>> 'Code' # 字符在内存中可以是任意大小'Code'>>> 'Code'.encode('utf-8') # 在文件中按 UTF-8 编码为 4 字节b'Code'>>> 'Code'.encode('utf-16') # 但按 UTF-16 编码则是 10 字节b'\xff\xfeC\x00o\x00d\x00e\x00'`这对更丰富的文本尤其如此（`ord` 给出字符的码点，`hex` 给出它的十六进制字符串）：`python >>> hex(ord('')) # 大到一个字节装不下的码点'0x1f40d'>>> len('hÄck') # 每个字符（码点）各算一个6>>> len('hÄck'.encode('utf-8')) # 但编码后的字节数各不相同13>>> len('hÄck'.encode('utf-16')) 18`要在文本字符串中编写非 ASCII 字符，可以用 `\x` 十六进制转义；短 `\u` 或长 `\U` Unicode 转义；或者使用按源编码（source encoding）解释的原始文本——源编码可以在读取程序文件时在文件里声明（代码文件的默认编码是通用的 UTF-8）。为了说明，这里是我们的非 ASCII 字符带分音符的 Ä 在 Python 中的四种写法：`python >>> 'h\xc4\u00c4\u000000c4Äck' # 非 ASCII 字符的四种写法：hex、短、长、原生'hÄÄÄÄck'`在文本字

字符串中，所有这些形式都指定了代表字符的 Unicode 码点。相比之下，字节字符串只用 `\x` 十六进制转义来嵌入原始字节的值；这不一定总是文本，但当它是文本时，它是文本的编码后形式，而不是解码后的码点，而且只有当文本和编码都很简单时，编码字节才与码点相同：`python >>> '\u00A3'.encode('latin1'), b'\xA3'.decode('latin1') ('£', b'\xa3', '£')` 除了这些字符串类型之外，Unicode 处理往往归结为与文件之间的文本数据转移——文件在存储时自动把文本编码（encode）成字节，读回内存时再把字节解码（decode）成字符（也就是码点）。一旦载入，我们通常只以解码后的字符串形式处理文本。为了做到这一点，文本文件实现编码、接受并返回文本字符串，而二进制文件则用 bytes 字符串处理原始数据。本章后面的文件部分还会再次遇到 Unicode，但其余 Unicode 的故事要留到本书后面了。它会在第 7、9、15 章短暂出现，但大部分内容推迟到本书的高级主题部分，即第 37 章。Unicode 如今在众多（或大多数）领域都至关重要，但许多 Python 新手在掌握字符串基础之前，只需要浅尝辄止即可。除了内置字符串对象，Python 的标准工具集还包含用 re 模块进行文本模式匹配的支持，以及解析 JSON、CSV、XML 和 HTML 等文本数据的能力。本书后面你会看到其中一些工具的更多示例，但这个教程式的介绍对字符串已经说得够多了，必须继续往前走了。

深度理解

·核心概念：Python 的两种字符串世界观：`str` = Unicode 码点序列（文本的世界）；`bytes` = 8 位字节序列（数据的世界）。`encode` 从文本到字节，`decode` 从字节回

文本。码点数 $\neq$ 字节数——`len()` 数的是码点，字节数取决于编码（UTF-8 变长、UTF-16 双字节/四字节、latin1 单字节）。

·底层实现：CPython 的 `str` 内部按码点存储，并自动选择紧凑布局（ASCII 用 1 字节/码点，其他字符可能用 2 或 4 字节）。`encode('utf-8')` 调用 C 层的 UTF-8 编解码器：U+0041 ('A') $\rightarrow$ 1 字节，U+00C4 ('Ä') $\rightarrow$ 2 字节，U+1F40D (🐼) $\rightarrow$ 4 字节。`'Code'.encode('utf-16')` 得到 10 字节 = 2 字节 BOM (`\xff\xfe`) + 4 字符 $\times$ 2 字节。`'hÄck'.encode('utf-16')` 是 18 字节 = 2 BOM + 4×2 (h、Ä、c、k) + 2×4 (两个 emoji 各占 2 个 UTF-16 码元 = 4 字节)。

·设计原因：把"文本"和"字节"严格分离（`str` vs `bytes`），是为了杜绝"乱码"的根源：文本必须带着编码信息才有意义，字节只是裸数据。文件接口顺势分为文本文件（自动编解码）和二进制文件（原样存取）——分工明确，责任清晰。

·实际问题：爬虫抓网页（`resp.content.decode('utf-8')`）、读写中文文件（`open(f, encoding='utf-8')`）、网络协议（`bytes` 字节流）——今天几乎每个真实项目都在 `str` 与 `bytes` 之间切换。掌握"码点-编码-字节"三段模型，乱码问题迎刃而解。

·初学者误区：①对 emoji 文本调 `len()` 得到"6"却以为存了 6 字节（实际 UTF-8 下 13 字节）；②'`中文.encode('utf-8')` 后忘了 `decode` 直接打印成 `b'\xe4\xb8\xad...`' 就以为"乱码了"——不是，这是 `bytes` 的正常 repr；③

用 latin1 存中文——latin1 只有 256 个码位，根本存不下，报错而不是乱码就是 Python 强类型的恩惠。

代码分析

```
>>> 'hÄck'                                     # str Unicode
'hÄck'

>>> b'a\x01c'                                   # bytes
b'a\x01c'

>>> 'Code'.encode('utf-8')                       # ->
b'Code'

>>> 'Code'.encode('utf-16')                       # 2  BOM + 4×2
b'\xff\xfeC\x00o\x00d\x00e\x00'

>>> hex(ord(''))                                 # ord hex
'0x1f40d'

>>> len('hÄck')                                  # 6
6

>>> len('hÄck'.encode('utf-8'))                  # UTF-8  13
13

>>> 'h\xc4\u00c4\u000000c4Äck'                  # hex
'hÄÄÄÄck'

>>> '\u00A3', '\u00A3'.encode('latin1'), b'\xA3'.decode('latin1')
('£', b'\xa3', '£')
```

·解释器如何处理：'hÄck' 中的 Ä 由词法分析器根据源码编码（默认 UTF-8）读成码点 U+00C4，存入 str 对象；repr 显示时，这个码点可打印，所以原样显示。b'a\x01c' 的 \x01 是原始字节值 1，不是字符——bytes 的 repr 只保证可打印 ASCII 原样显示。'Code'.encode('utf-16') 调用 C 层编解码器：先写 BOM（\xff\xfe，小端标记），再逐字符写 2 字节（C\x00 表示 C 的低字节在前）。'h\xc4\u00c4\u000000c4Äck' 中的四种写法在词法阶段

全部折算成同一个码点 U+00C4，所以运行时对象里是 5 个相同的 Ä。最后一行是"编码一致性"的演示：£ (U+00A3) 在 latin1 里恰好就是单字节 \xA3——只有这种简单编码下，字节才等于码点。

·设计思想：encode/decode 成对出现，构成了 Python 处理文本的标准管道：文本 → 编码 → 字节 → (文件/网络) → 解码 → 文本。理解了这条管道，Unicode 就不再神秘。

4.7 列表 (Lists)

英文原文

The Python list object is the most general sequence provided by the language. Lists are positionally ordered collections of arbitrarily typed objects, and they have no fixed size. They are also mutable—unlike strings, lists can be modified in place by assignment to offsets as well as a variety of list method calls. Accordingly, they provide a very flexible tool for representing arbitrary collections—files in a folder, employees in a company, emails in your inbox, and so on.

中文翻译

Python 的列表 (list) 对象是这门语言提供的最通用的序列。列表是任意类型对象按位置有序排列的集合，而且没有固定大小。它们还是可变 (mutable) 的——与字符串不同，列表可以通过向偏移赋值以及各种列表方法调用来就地

修改。因此，它们是表示任意集合的非常灵活的工具——文件夹里的文件、公司里的员工、收件箱里的电子邮件，等等。

深度理解

·核心概念：列表 = 任意类型的序列（可异构：一个列表里可以混装整数、字符串、浮点数）+ 可变（原地增删改）+ 无固定大小（按需伸缩）。它是 Python 中最常用的“通用容器”。

·底层实现：CPython 的 list 是动态数组：底层是 PyObject 指针数组，对象头里记录已用长度（obsize）和已分配容量（allocated）。append 只在容量不足时才扩容（约 1.125 倍），均摊 $O(1)$ ；pop(2) 则把后面的元素整体前移（ $O(n)$ ）。因为存的是指针数组，列表元素可以是任何类型——数组里的每个槽位都只是指向对象的引用。

·设计原因：为什么是动态数组而不是链表？因为 Python 的索引 $L[0]$ 、 $L[-1]$ 使用太频繁，数组的 $O(1)$ 随机访问远胜链表的 $O(n)$ 遍历；代价是中间插入/删除较慢，但 Python 更偏爱“按位置取”的用法。

·实际问题：文件夹列表、待办清单、CSV 行、参数集合——凡是有顺序、要频繁按位置访问的集合，优先用列表。append/pop 组合还能当栈用。

·初学者误区：把列表当 C 数组——C 数组固定长度且类型一致，Python 列表两者都不受限制；以为 $L + [4,5]$ 会改变 L——拼接产生新列表，L 原封不动。

4.7.1 序列操作 (Sequence Operations)

英文原文

Because they are sequences, lists support all the sequence operations we discussed for strings; the only difference is that most of them return lists instead of strings. For instance, given a three-item list:

```
python >> L = [123,'text', 1.23] # 一个包含三种不同类型对象的列表
>> len(L) # 列表中条目数 3
```

we can index, slice, and so on, just as for strings:

```
python >> L[0] # 按位置 (偏移) 索引 123
>> L[:1] # 切片返回新列表 [123,'text']
>> L + [4, 5, 6] # 拼接/重复也生成新列表 [123,'text', 1.23, 4, 5, 6]
>> L * 2 # [123,'text', 1.23, 123,'text', 1.23]
>> L # 我们没有改变原列表 [123,'text', 1.23]
```

中文翻译

因为列表是序列，它们支持我们为字符串讨论过的所有序列操作；唯一的区别是，其中大多数操作返回的是列表而不是字符串。例如，给定一个三条目列表：python >>> L = [123,'text', 1.23] # 一个包含三种不同类型对象的列表>>> len(L) # 列表中条目数3 我们可以像对字符串那样做索引、切片等等：python >>> L[0] # 按位置 (偏移) 索引123>>> L[:1] # 切片返回新列表[123,'text']>>> L + [4, 5, 6] # 拼接/重复也生成新列表[123,'text', 1.23, 4, 5, 6]>>

> L 2 [123,'text', 1.23, 123,'text', 1.23] >>> L # 我们没有改变原列表[123,'text', 1.23]

深度理解

·核心概念：序列操作是"一套动作、处处通用"——len、索引、切片、拼接、重复对字符串和列表几乎一样，只是返回类型跟着对象走（切片字符串得字符串、切片列表得列表）。

·底层实现：L + [4, 5, 6] 在 C 层为新列表预分配 6 个槽位，把两个列表的指针拷贝过去；L 2 同理重复拷贝。切片 L[: -1] 创建新列表对象，只拷贝指针（1 层浅拷贝）——列表切片是"新列表、旧元素"。注意 L[: -1] 对列表是意义且常用的（去掉最后一项），而对字符串同样适用——这就是序列操作跨类型的价值。

·设计原因：通用操作复用，让语言工具集"小而一致"；列表与字符串的行为差异只出现在"可改变"这一点上（列表可变、字符串不可变），其余全部共享。

·实际问题：L[: -1] 取除尾项、L[:] 复制列表（之后改副本不影响原件——这是"拷贝列表"的标准姿势）、L + [x] 造新列表——这些是在函数式风格代码里避免原地修改的常用手法。

·初学者误区：L[:] 复制后，若列表里有嵌套列表，改嵌套列表仍会"传染"到原列表（浅拷贝）；以为 L 2 是"复制对象"，其实 [[]]3 得到的三个空列表是同一个列表对象——这是经典陷阱，等学到第 6 章"共享引用"时会有完整的解释（本书第 6 章"Shared References"一节会专门讲）。

代码分析

```
>>> L = [123, 'text', 1.23]          #
>>> len(L)                          # 3
3
>>> L[0]                            # → 123
123
>>> L[:-1]                          #
[123, 'text']
>>> L + [4, 5, 6]                   #
[123, 'text', 1.23, 4, 5, 6]
>>> L * 2                           #
[123, 'text', 1.23, 123, 'text', 1.23]
>>> L                               #
[123, 'text', 1.23]
```

·解释器如何处理：L = [123,'text', 1.23] 编译为三条 LO
ADCONST + BUILDLIST 3——虚拟机在堆上分配一个 3
槽位的指针数组，把三个常量对象的引用依次填入。L[0]
的 BINARYSUBSCR 对列表走"下标直访"：直接按偏移算
地址取指针，O(1)。L[:-1] 的切片由 C 层 listslice 实现：
新列表的槽位复制指针（元素本身不复制）。L + [4,5,6]
的 BINARYOP 走 listconcat：分配 6 槽，前 3 槽指向 L
的元素、后 3 槽指向新字面量列表的元素，然后旧字面量
列表被回收。L * 2 同理。整个过程中 L 指向的列表对象从
未改变——每次都是"新对象 + 新内容"。

·设计思想：拼接/重复/切片返回新列表、append/pop
原地改——两种风格泾渭分明，取决于是否制造新对象。
这是 Python 的一贯纪律：要么纯函数式地返回新值，要
么有副作用的原地操作，从不混着来。

4.7.2 类型特有操作 (Type-Specific Operations)

英文原文

Python's lists may be reminiscent of arrays in other languages, but they tend to be more powerful. For one thing, they have no fixed type constraint—the list we just looked at, for example, contains three objects of completely different types (an integer, a string, and a floating-point number). Further, lists have no fixed size. That is, they can grow and shrink on demand, in response to list-specific operations:

```
python >> L.append('Py') # 增长: 在列表末尾添加对象
>> L [123,'text', 1.23,'Py'] >> L.pop(2) # 收缩: 删除中间的一个条目 1.23
>> L [123,'text','Py'] # "del L[2]" 也能从列表中删除
```

Here, the list append method expands the list's size and inserts an item at the end; the pop method (or an equivalent del statement) then removes an item at a given offset, causing the list to shrink. Other list methods insert an item at an arbitrary position (insert), remove a given item by value (remove), add multiple items at the end (extend), and more.

Because lists are mutable, most list methods also change the list object in place, instead of making a new one:

```
python >> M = ['bb','aa','cc'] >> M.sort() # 原地排序
```

```
>> M ['aa','bb','cc'] >> M.reverse() # 原地反转 >> M ['cc','bb','aa']
```

The list sort method here, for example, orders the list in ascending fashion by default, and reverse reverses it; in both cases, the methods modify the list directly (though similarly named functions we'll explore later do not).

中文翻译

Python 的列表可能让人联想到其他语言中的数组 (array)，但它们往往更强大。首先，列表没有固定的类型约束——我们刚才看的列表就包含三个完全不同类型的对象（一个整数、一个字符串、一个浮点数）。其次，列表没有固定的大小。也就是说，它们可以响应列表特有的操作按需增长和收缩：python >>> L.append('Py') # 增长：在列表末尾添加对象 >>> L [123,'text', 1.23,'Py'] >>> L.pop(2) # 收缩：删除中间的一个条目 1.23 >>> L [123,'text','Py'] # "del L[2]" 也能从列表中删除这里，列表的 append 方法扩大列表的尺寸并在末尾插入一个条目；pop 方法（或等效的 del 语句）则删除给定偏移处的条目，使列表收缩。其他列表方法还有：在任意位置插入条目 (insert)、按值删除给定条目 (remove)、在末尾添加多个条目 (extend) 等等。因为列表是可变的，大多数列表方法也是就地修改列表对象，而不是制造新的：python >>> M = ['bb','aa','cc'] >>> M.sort() # 原地排序 >>> M ['aa','bb','cc'] >>> M.reverse() # 原地反转 >>> M ['cc','bb','aa'] 例如这里的列表 sort 方法默认按升序给列表排序，reverse 则反转它；这两种情况里，方法都直接修改了列表（不过我们后面会探

索的同名函数就不会)。

深度理解

·核心概念：列表的"独门绝技"全是原地操作：append/pop/insert/remove/extend/sort/reverse——它们直接改写列表对象本身，返回 None（而不是返回新列表）。这与字符串方法（返回新串）形成鲜明对照，根源就是可变 vs 不可变。

·底层实现：append 在 C 层检查容量，不足则 realloc 扩容（新容量 $\approx 1.125 \times$ 旧容量，预留下一次空间，均摊 $O(1)$ ）；pop(2) 取出索引 2 的指针（返回它，故显示 1.23），然后把后面的元素整体左移一位；sort 用 C 层的 TimSort 算法（Python 发明的混合排序，对真实数据极快）；reverse 原地交换首尾指针。del L[2] 编译为 DELETESUBSCR，与 pop 的删除部分等价，但不返回值。

·设计原因：可变对象把"改"作为第一公民操作，换来 $O(1)$ 的末尾增删和 $O(n \log n)$ 的快排；不可变对象把"安全共享"作为第一目标。同一套序列接口，两种对象哲学——这是理解 Python 数据类型的一把总钥匙。

·实际问题：L.append(x) 是循环里收集结果的标配（比 $L = L + [x]$ 快得多且不产生垃圾）；sort() 注意它不返回值—— $M = M.sort()$ 会得到 None，这是新手最常踩的坑之一；sorted(M)（同名内置函数）则返回新列表、不动原列表，两者互补。

·初学者误区：① $M = M.sort()$ 把排序结果赋给 M 得到 None（方法原地改并返回 None）；②以为 pop 是"弹栈"

专用——它还能指定任意位置 pop(2); ③认为 append 有返回值——它返回 None。

代码分析

```
>>> L.append('Py') #
>>> L
[123, 'text', 1.23, 'Py']
>>> L.pop(2) # 2
1.23
>>> L # 3
[123, 'text', 'Py']

>>> M = ['bb', 'aa', 'cc']
>>> M.sort() #
>>> M
['aa', 'bb', 'cc']
>>> M.reverse() #
>>> M
['cc', 'bb', 'aa']
```

·解释器如何处理：L.append('Py') 编译为 LOADMETHOD append + CALLMETHOD 1, C 层 listappend 先检查 `obsize < allocated`——不满足就 `realloc` 扩容，然后把 'Py' 的引用写入新槽位、`obsize+1`。L.pop(2) 的 C 实现：取索引 2 的指针（这就是它“返回 1.23”的原因），把后续指针逐个前移，`obsize-1`，返回弹出的对象。M.sort() 在 C 层对指针数组跑 TimSort——注意排序只重排指针，不移动元素对象本身；比较用的是元素间的 `<`（默认升序）。M.reverse() 双指针从两端向中间交换。这些方法全部原地修改 M 这一个对象——执行前后 M 的身份（id）不变。

·设计思想: pop 同时满足"删"与"取"两个需求(删除并返回); sort/reverse 原地改、sorted/reversed 内置函数造新——方法 vs 内置函数的对偶设计贯穿 Python 工具箱, 书里特意提示"同名函数以后会出现", 就是为第 5、9 章的对照埋的伏笔。

4.7.3 边界检查 (Bounds Checking)

英文原文

Although lists have no fixed size, Python still doesn't allow us to reference items that are not present. Indexing off the end of a list is always a mistake, but so is assigning off the end (error messages are condensed here and elsewhere):

```
python >>> L [123,'text','Py'] >>> L[99] # 越界读IndexError: list index out of range >>> L[99] = 1 # 越界写IndexError: list assignment index out of range
```

This is intentional, as it's usually an error to try to assign off the end of a list (and a particularly nasty one in the C language, which doesn't do as much error checking as Python). Rather than silently growing the list in response, Python reports an error. To grow a list, we call list methods such as `append` instead (or make a new list). Unlike indexing, slicing scales offsets to be in bounds, but this will have to await coverage in the in-depth chapters ahead.

中文翻译

虽然列表没有固定大小，Python 仍然不允许我们引用不存在的条目。索引越过列表末尾永远是错误，向末尾之外赋值同样如此（这里的错误信息和其他地方一样有删节）：`python >>> L [123,'text','Py'] >>> L[99] # 越界读IndexError: list index out of range >>> L[99] = 1 # 越界写IndexError: list assignment index out of range` 这是故意的：试图给列表末尾之外的位置赋值通常是错误（在 C 语言里这尤其恶劣，因为 C 不像 Python 做那么多错误检查）。Python 不会静默地增长列表来响应，而是报告错误。要增长列表，我们改用 `append` 之类的列表方法（或者造一个新列表）。与索引不同，切片会缩放（scale）偏移使其落在边界内，不过这要留到后面深入章节再讲了。

深度理解

·核心概念：读越界报错、写越界报错、但“该增长就调用方法”——Python 把“越界”视为程序员的逻辑错误，绝不静默补救。对比 C 语言：数组越界读写是未定义行为（可能静默破坏内存、留下安全漏洞）。

·底层实现：BINARYSUBSCR 执行时，虚拟机检查索引范围 $0 \leq \text{idx} < \text{obsize}$ （负数先加 `obsize`），不满足立即构造 `IndexError` 异常并抛到调用栈；STORESUBSCR 同样检查后才会写内存。先检查、后访问，所以越界永远到不了“真的读写内存”那一步。

·设计原因：宁可程序崩溃也要暴露错误——“报错早于出错”。这是 Python 强类型哲学在容器上的延伸：检测到错误立即停止，而不是带着错误继续运行。

·实际问题：实际代码中，取列表末元素 `L[-1]`、循环 `for x in L` 都不会越界；越界多发生在“手算索引”时（例如 `L[len(L)]`）。处理“可能不存在”的需求，应该用 `L[index:index+1]` 切片（自动缩放，返回空列表或单元素列表）等手法，而不是指望越界合法。

·初学者误区：以为“列表没固定大小，所以 `L[99] = 1` 会自动把列表撑到 100 项”——不会，它报 `IndexError`；以为越界索引返回 `None`——不会，它抛异常。

代码分析

```
>>> L[99]                                     #
IndexError: list index out of range
>>> L[99] = 1                                 #
IndexError: list assignment index out of range
```

·解释器如何处理：执行 `L[99]` 时，虚拟机的 subscript 例程先做边界检查： $99 \geq 3$ （列表长度），检查失败 → 创建 `IndexError` 实例，按异常处理机制沿调用栈逐层查找 `except` 处理者，REPL 中没有 → 打印 `traceback` 并终止该条语句。`L[99] = 1` 的 `STORESUBSCR` 同理。注意两次报错的文案不同（`index / assignment index`），说明读写走的是不同的 C 入口（`sqitem / sqassitem`），但都做同样的守卫。

·设计思想：索引要精确、增长要走方法——这是“越界宁可报错、也绝不擅自扩表”的纪律。C 语言做不到这点（数组越界是未定义行为，可能写坏相邻内存），Python 把它变成了可预期的异常，这正是“强类型语言替你兜底”的实例。

4.7.4 嵌套 (Nesting)

英文原文

One nice feature of Python's core object types is that they support arbitrary nesting—we can nest them in any combination, and as deeply as we like. For example, we can have a list that contains a dictionary, which contains another list, and so on—as deeply and mixed as needed to describe things in our real world. An immediate application of this feature is to represent matrixes, or "multidimensional arrays" in Python. A list with nested lists like the following will do the job for basic applications (coding fine print: indentation doesn't matter in this, Python expressions with unclosed brackets can span multiple lines this way, and some REPLs show "..." continuation-line prompts, which are often omitted in this book for easier copy and paste from media):

```
python >>> M = [[1, 2, 3], # 一个 3×3 矩阵，用嵌套列表表示... [4, 5, 6], # 括号未闭合时代码可以跨行... [7, 8, 9]] >>> M
```

`[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` Here, we've coded a list that contains three other lists. The effect is to represent a 3×3 matrix of numbers. Such a structure can be accessed in a variety of ways:

```
python >>> M[1] # 取第 2 行[4, 5, 6] >>> M[1][2] # 取第 2 行，再取该行内第 3 个条目6
```

The first operation here fetches the entire second row, and the second grabs the third item within that row (it runs left to right, like the earlier string strip and split combo). Stringi

ng together index operations takes us deeper and deeper into our nested-object structure. Tip: this matrix structure works for small-scale tasks, but for more serious number crunching you will probably want to use the open source NumPy extension for Python, which can store and process large matrixes much more efficiently than our nested list structure (and is well out of scope here; see Chapter 1).

中文翻译

Python 核心对象类型的一个优点，是它们支持任意嵌套（nesting）——我们可以按任意组合、任意深度嵌套它们。例如，我们可以有一个包含字典的列表，字典里又包含另一个列表，如此往复——按描述现实世界所需的多深、多混合都行。这个特性的直接应用是用 Python 表示矩阵，即“多维数组”。下面这种带嵌套列表的列表对基本应用足够了（编码细节：这里的缩进无关紧要——括号未闭合的 Python 表达式可以这样跨行；有些 REPL 会显示“...”续行提示，本书为了便于从电子媒体复制粘贴常常省略）：

```
python >>> M = [[1, 2, 3], # 一个 3×3 矩阵，用嵌套列表表示...
[4, 5, 6], # 括号未闭合时代码可以跨行... [7, 8, 9]] >>> M
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

这里我们编写了一个包含三个其他列表的列表。效果是表示一个 3×3 的数字矩阵。这样的结构可以用多种方式访问：

```
python >>> M[1] # 取第 2 行[4, 5, 6] >>> M[1][2] # 取第 2 行，再取该行内第 3 个条目6
```

第一个操作取回整个第二行，第二个操作抓取那一行里的第三个条目（它从左到右运行，和前面字符串 `strip + split` 的组合一样）。把索引操作串起来，就让我们在嵌

套对象结构中越走越深。提示：这种矩阵结构适合小规模任务，但更认真的数值计算时，你大概会想用开源的 NumPy 扩展——它存储和处理大型矩阵的效率比我们的嵌套列表结构高得多（这远远超出本书范围；参见第 1 章）。

深度理解

·核心概念：嵌套 = 对象里装对象，任意深度、任意混合。这是 Python 表达复杂数据的核心武器：矩阵 = 列表的列表，记录 = 字典里嵌字典/列表。

·底层实现：M[1] 先对外层列表取偏移 1，得到内层列表对象的引用（此时没有拷贝，只是指针）；M[1][2] 是两次 BINARYSUBSCR 的串接——每次返回的都是"引用"而不是副本，所以链条多长都只花 O(1) 取指针的时间。M = [[1,2,3],...] 在字节码上是：内层三条 BUILDLIST 先构建 3 个小列表，外层 BUILDLIST 再装 3 个指针。

·设计原因：底层语言（C）表示多维数组要声明固定维度和类型、手写内存布局；Python 用"容器套容器"的递归结构，天然支持"行长不一、类型各异、按需扩展"的异构数据——这正是脚本语言处理真实世界的优势（作者原话：在 C 里做同样的事"tedious"）。

·实际问题：嵌套结构在真实代码中无处不在：数据库记录（字典列表）、游戏棋盘（二维列表）、JSON 数据（字典/列表混合嵌套）。M[1][2] 这种"逐层深入"的索引风格，后面讲推导式时会被一行替代（[row[2] for row in M]）。

·初学者误区：以为 M[1] 返回了内层列表的"副本"——它

返回的是引用，对 `M[1][0] = 99` 的修改会体现在 `M` 里（这其实正是我们想要的）；以及把嵌套列表和 NumPy 的 `ndarray` 混为一谈——后者是连续内存的数值数组，前者是引用数组，性能差一两个数量级。

代码分析

```
>>> M = [[1, 2, 3],           # 3×3
...      [4, 5, 6],           #
...      [7, 8, 9]]
>>> M[1]                       # 2
[4, 5, 6]
>>> M[1][2]                   # 3
6
```

·解释器如何处理：编译时，语法分析器看到左方括号后括号数未配对，就继续吞并后续行，直到 `]]` 闭合——所以三行文本被编译成一个 `BUILDLIST` 指令序列：先建三个内层列表，再建外层列表收容它们。运行 `M[1][2]` 时：第一个 `BINARYSUBSCR` 用索引 1 从外层数组取指针 → 得到内层列表对象（长度为 3 的数组）；第二个 `BINARYSUBSCR` 用索引 2 从该数组取指针 → 得到整数对象 6。每一步都是 $O(1)$ 指针搬运。整个结构在内存里像一棵树：根列表 → 3 个分支列表 → 各含 3 个整数对象（小整数 1–9 其实还是解释器缓存的单例）。

·设计思想：`M[1][2]` 从左到右逐步“下钻”，与 `line.rstrip().split(',')` 的链式调用同构——Python 的表达总是“一步一个结果、结果接着下一步”。嵌套结构让“任意复杂的数据”有了统一的表达语法，这就是为什么 JSON 与 Python 对象如此亲缘（本章后面会提到）。

4.7.5 列表推导式 (Comprehensions)

英文原文

In addition to sequence operations and list methods, Python includes a more advanced operation known as a list comprehension expression, which turns out to be a powerful way to process structures like our matrix. Suppose, for instance, that we need to extract the second column of the prior section's matrix. It's easy to grab rows by simple indexing because the matrix is stored by rows, but it's almost as easy to get a column with a list comprehension:

```
python >> col2 = [row[1] for row in M] # 收集第 2 列的所有条目
>> col2
[2, 5, 8]
>> M
# 矩阵本身没有变化
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

List comprehensions are a way to build a new list by running an expression on each item in a sequence, one at a time, from left to right. They are coded in square brackets (to tip you off to the fact that they make a list) and are composed of an expression and a looping construct that share a variable name (row, here).

The preceding list comprehension means basically what it says: "Give me row[1] for each row in matrix M, in a new list." The result is a new list containing column 2 of the matrix.

List comprehensions can be more complex in practice

e:

```
python >> [row[1] + 1 for row in M] # 第 2 列的每个条  
目加 1 [3, 6, 9] >> [row[1] for row in M if row[1] % 2  
== 0] # 过滤掉奇数 (留下偶数) [2, 8]
```

The first operation here, for instance, adds 1 to each item as it is collected, and the second uses an if clause to filter odd numbers out of the result using the % modulus expression (remainder of division).

List comprehensions make new lists of results, but they can be used to iterate over any iterable object—a term we'll flesh out later in this book, but which simply means either a physical sequence or a virtual one that produces its items on request. Here, for instance, we use list comprehensions to step over a hardcoded list of coordinates and a string:

```
python >> diag = [M[i][i] for i in [0, 1, 2]] # 收集矩阵  
对角线 >> diag [1, 5, 9] >> doubles = [c * 2 for c in 'hack']  
# 重复字符串中的字符 >> doubles ['hh','aa','cc','kk']
```

These expressions can also be used to collect multiple values, as long as we wrap those values in a nested collection. The following illustrates using range—a built-in that generates successive integers and requires a surrounding list to force it to yield all its values for display in the REPL (there's more on this mystery ahead):

```
python >> list(range(4)) # 整数 0..(N-1) [0, 1, 2, 3] >>
```

```
list(range(-6, 7, 2)) # -6 到 +6, 步长 2 [-6, -4, -2, 0, 2, 4, 6] >> [[x 2, x 3] for x in range(4)] # 多值收集, 配合 "if" 过滤 [[0, 0], [1, 1], [4, 8], [9, 27]] >> [[x, x // 2, x 2] for x in range(-6, 7, 2) if x > 0] [[2, 1, 4], [4, 2, 8], [6, 3, 12]]
```

As you can probably tell, list comprehensions are too involved to cover more formally in this preview. The main point of this brief introduction is to illustrate that Python includes both simple and advanced tools in its arsenal. List comprehensions are optional, but they can be very useful in practice and often provide a processing speed advantage.

In fact, comprehension syntax is not just for making lists: enclosing it in parentheses can also be used to create an iterable object known as a generator, which produces results on demand per Python's iteration protocol—in the following, summing items in a matrix row with the built-in sum, on each call to next:

```
python >> M [[1, 2, 3], [4, 5, 6], [7, 8, 9]] >> G = (sum(row) for row in M) # 生成器: 行和的"按需生产" >> next(G) # 运行迭代协议 (后文详述) 6 >> next(G) # 每次调用产生一个新的行和 15 >> next(G) # 第 3 行: 7 + 8 + 9 24
```

And comprehension syntax can also be used to create sets and dictionaries when enclosed in curly braces:

```
python >> {sum(row) for row in M} # 生成一个无序的行和集合 {24, 6, 15} >> {i: sum(M[i]) for i in range(3)}
```


生成 key:value 行和表 {0: 6, 1: 15, 2: 24}

But to grasp concepts like the iteration protocol and objects like sets and dictionaries, we must move ahead.

中文翻译

除了序列操作和列表方法，Python 还包含一种更高级的操作，叫做列表推导式（list comprehension）表达式，它是处理矩阵这类结构的有力工具。假设，例如，我们需要提取上一节矩阵的第二列。因为矩阵按行存储，用简单索引抓行很容易，但用列表推导式取列也几乎一样容易：python

```
>>> col2 = [row[1] for row in M] # 收集第 2 列的所有条目
>>> col2
[2, 5, 8]
>>> M
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

列表推导式是这样一种方法：对序列中的每个条目逐个（从左到右）运行一个表达式，从而构建一个新列表。它们写在方括号里（提示你它们制造的是一个列表），由共享一个变量名的表达式和循环结构组成（这里变量名是 row）。上面的列表推导式说的基本就是它的字面意思："给我矩阵 M 中每个 row 的 row[1]，放到一个新列表里。"结果是一个包含矩阵第 2 列的新列表。列表推导式在实践中可以更复杂：python

```
>>> [row[1] + 1 for row in M] # 第 2 列的每个条目加 1
[3, 6, 9]
>>> [row[1] for row in M if row[1] % 2 == 0] # 过滤掉奇数（留下偶数）
[2, 8]
```

例如，第一个操作在收集每个条目时给它加 1，第二个用 if 子句配合 % 取模表达式（除法的余数）把奇数从结果中过滤掉。列表推导式制造结果的新列表，但可以在任何 iterable（可迭代）对象上——这个词我们后面会展开，但简单说就是：物理序列，或者按需产出条目的虚拟

序列。例如，这里我们用列表推导式遍历一个硬编码的坐标列表和一个字符串：

```
python >>> diag = [M[i][i] for i in
[0, 1, 2]] # 收集矩阵对角线>>> diag [1, 5, 9] >>> dou
bles = [c * 2 for c in 'hack'] # 重复字符串中的字符>>> do
ubles ['hh','aa','cc','kk']
```

只要把多个值包进一个嵌套集合，这些表达式还可以用来收集多个值。下面演示了 range 的用法——range 是一个生成连续整数的内置函数，需要外面套个 list 强制它在 REPL 里产出全部值以供显示（这个谜团后面还有更多）：

```
python >>> list(range(4)) # 整数
0..(N-1) [0, 1, 2, 3] >>> list(range(-6, 7, 2)) # -6 到 +
6, 步长 2 [-6, -4, -2, 0, 2, 4, 6] >>> [[x * 2, x * 3] for x i
n range(4)] # 多值收集，配合 "if" 过滤[[0, 0], [1, 1], [4,
8], [9, 27]] >>> [[x, x // 2, x * 2] for x in range(-6, 7, 2) i
f x > 0] [[2, 1, 4], [4, 2, 8], [6, 3, 12]]
```

你大概能看出来，列表推导式内容太多，在这个预览里没法更正式地展开。这个简短介绍的要点是说明 Python 的武器库里既有简单的也有高级的工具。列表推导式是可选的，但实践上非常有用，而且常常带来处理速度上的优势。事实上，推导式语法不只是用来制造列表：把它包在圆括号里，还可以创建一种叫做生成器（generator）的可迭代对象，它按照 Python 的迭代协议（iteration protocol）按需产出结果——下面这个例子，在每次调用 next 时用内置函数 sum 求和矩阵的一行：

```
python >>> M [[1, 2, 3], [4, 5, 6], [7, 8, 9]] >>> G
= (sum(row) for row in M) # 生成器：行和的"按需生产"
>>> next(G) # 运行迭代协议（后文详述）6 >>> next(G)
# 每次调用产生一个新的行和15 >>> next(G) # 第 3 行
: 7 + 8 + 9 24
```

推导式语法还可以在花括号里用来创建集合和字典：

```
python >>> {sum(row) for row in M} # 生
```

成一个无序的行和集合{24, 6, 15} >>> {i: sum(M[i]) for i in range(3)} # 生成 key:value 行和表{0: 6, 1: 15, 2: 24}
但要真正理解迭代协议以及集合、字典这些对象，我们得继续往前走。

深度理解

·核心概念：列表推导式 = "对每个元素做一件事，收集结果成列表"的浓缩写法。语法：[表达式 for 变量 in 可迭代对象 (if 条件)]。它是"循环 + 收集"的声明式替代品，且允许加 if 过滤、包元组收集多值。

·底层实现：[row[1] for row in M] 在字节码层面实际上是隐式循环：LISTAPPEND 指令把每次计算结果追加进新列表——它和 col2=[]; for row in M: col2.append(row[1]) 几乎等价，但更快：因为推导式在专用字节码 (GETITER + FORITER + LISTAPPEND) 下运行，没有 Python 函数调用的 col2.append(...) 查属性开销。range(4) 是一个"惰性"对象——它不生产 4 个整数，而是按需产出 (0, 1, 2, 3)，所以必须用 list() 包一层强制全部产出，才能显示。

·设计原因：推导式把"造新列表"的意图显式化，读起来像数学集合记号 ($\{x^2 \mid x \in S\}$)；速度优势则来自编译器的专门优化。生成器（圆括号版本）更进一步——不建完整列表，边用边算，海量数据时省内存。

·实际问题：[row[1] for row in M] 取列、[c2 for c in 'hack'] 逐字符变换、[x2 for x in range(n)] 批量计算——推导式是 Python 日常数据处理的第一高频句式。数据量大

时用生成器表达式 (sum(xx for x in data) 不建中间列表)。

·初学者误区：①把推导式当"高级语法"避之不及——它就是带结果的 for 循环，先写 for 循环再浓缩即可；②以为 range(4) 是列表——它是 range 对象，list() 包了才是列表；③在推导式里写副作用（如 print）——推导式是表达式，应保持"纯计算"。

代码分析

```
>>> col2 = [row[1] for row in M]      # 2 row row[1]
[2, 5, 8]
>>> [row[1] + 1 for row in M]        # 1
[3, 6, 9]
>>> [row[1] for row in M if row[1] % 2 == 0]  # if
[2, 8]
>>> diag = [M[i][i] for i in [0, 1, 2]]      # M[0][0]M[1]...
[1, 5, 9]
>>> doubles = [c * 2 for c in 'hack']      #
['hh', 'aa', 'cc', 'kk']
>>> list(range(-6, 7, 2))                  # range-6 +6 7 2
[-6, -4, -2, 0, 2, 4, 6]
>>> [[x ** 2, x ** 3] for x in range(4)]    #
[[0, 0], [1, 1], [4, 8], [9, 27]]

>>> G = (sum(row) for row in M)            # =
>>> next(G)                                # 1+2+3
6
>>> next(G)                                #
15
>>> {sum(row) for row in M}                # =
{24, 6, 15}
>>> {i: sum(M[i]) for i in range(3)}      # =
{0: 6, 1: 15, 2: 24}
```

·解释器如何处理：列表推导式的字节码序列是 GETITER (从 M 取得迭代器) → FORITER 循环头 (每次取一个 row) → 对 row 执行 row[1] → LISTAPPEND (追加进正在构建的列表) → 循环结束返回列表。带 if 时多了 POPJUMPIFFALSE 指令：row[1] % 2 == 0 为假就跳过追加。range(-6, 7, 2) 是 range 对象的构造调用——它内部只存 start/stop/step 三个整数，迭代时才逐项算出 0、2、4... (负数和步长语义：-6 到 +6 步长 2)。生成器表达式 (sum(row) for row in M) 不立即求值：它创建一个生成器对象，next(G) 时解释器挂起/恢复这个生成器的帧——这是惰性求值：求和真正发生的时间被推迟到 next 被调用的那一刻。

·设计思想：同一套 for ... in ... 语法 + 不同外壳 = 四种工厂：[] 列表、() 生成器、{} 集合、{k:v} 字典。"外壳决定产物"与字面量语法一脉相承 ([] 是列表、{} 是字典/集合)，体现了 Python 语法的高度一致性。

4.8 字典 (Dictionaries)

英文原文

Unlike strings and lists, Python dictionaries are not sequences at all but are instead the only core member of a category known as mappings. Mappings are also collections of other objects, but they store objects by key instead of by relative position. While dictionaries retain the insertion order of their keys today, this ma

y not apply to your goals, and key-to-value mapping remains their main role. Dictionaries are also mutable: like lists, they may be changed in place and can grow and shrink on demand. Also like lists, they are a flexible tool for coding collections, but their more mnemonic keys are better suited when a collection's items are named or labeled—as in fields of a database record.

中文翻译

与字符串和列表不同，Python 字典根本就不是序列，而是被称为映射（mapping）的类别中唯一的成员。映射也是其它对象的集合，但它们按键（key）而不是按相对位置存储对象。虽然如今的字典会保留键的插入顺序，但这未必适用于你的目标，键到值的映射仍然是它们的主要角色。字典也是可变的：和列表一样，它们可以就地修改、按需增长和收缩。也和列表一样，它们是编写集合的灵活工具，但当集合的条目有名字或标签时——比如数据库记录的字段——它们更助记（mnemonic）的键就更合适。

深度理解

·核心概念：字典 = 映射（键 → 值），与序列（位置 → 元素）是两种并列的集合哲学。查找的“地址”从数字偏移变成了任意（可哈希的）键。字典是 Python 中唯一的映射类型。

·底层实现：dict 在 CPython 中是哈希表（hash table）：存储键的哈希值，通过哈希定位桶（bucket），再经探

测 (probing) 解决冲突。平均 $O(1)$ 的查找/插入/删除，远快于列表的 $O(n)$ 线性查找——这就是作者说"字典索引是 Python 中编码搜索的最快方式"的原因。Python 3.7+ (以及 CPython 3.6 的实现细节) 采用"紧凑 + 插入序"布局：条目数组只记录插入顺序的索引，所以键保持插入顺序 (本书将这一顺序当作常态)。

·设计原因：用"键"代替"偏移"，是因为现实数据大多是"按名字找值" (工资单按员工找、设置按选项找)。可哈希键 (数字、字符串、元组) 是"内容相等即同一把钥匙"的保证——这也是为什么列表不能当键 (可变内容会破坏哈希)。

·实际问题：数据库记录、配置表、计数器 (`wordcount[w] = wordcount.get(w, 0) + 1`)、JSON 对象 (`{}` 语法与 JSON 几乎一一对应) ——字典是 Python 数据处理的"心脏"。

·初学者误区：①把字典当"有序列表"用 (插入顺序保留是特性不是契约，删除插入后键的顺序仍保持插入序但不要依赖遍历顺序做业务)；②以为键必须从 0 开始连续——键可以是任何可哈希对象；③ `D[key]` 与 `D.get(key)` 混淆——前者键不存在抛 `KeyError`，后者返回默认值。

4.8.1 映射操作 (Mapping Operations)

英文原文

When written as literals, dictionaries are coded in curly braces and consist of a series of "key: value" pairs. Dictionaries are useful anytime we need to associate

a set of values with keys—to describe the properties of something. As an example, consider the following three-item dictionary with keys "name," "job," and "age," that record the attributes of a fictitious (and wholly generic and neutral) worker:

```
python >>> D = {'name':'Pat','job':'dev','age': 40} We
can index this dictionary by key to fetch and change i
ts keys' associated values. The dictionary index opera
tion uses the same syntax as that used for sequences,
but the item in the square brackets is a key, not a rel
ative position:
```

```
python >>> D['name'] # 取键'name' 对应的值'Pat'
>>> D['job'] = 'mgr' # 修改 Pat 的职位>>> D {'name':
Pat','job':'mgr','age': 40}
```

Although the curly-braces literal form does see use, it is perhaps more common to see dictionaries built up in different ways (after all, it's rare to know all your program's data before your program runs). The following code, for example, starts with an empty dictionary and fills it out one key at a time. Unlike out-of-bounds assignments in lists, which are forbidden, assignments to new dictionary keys create those keys:

```
python >>> D = {} >>> D['name'] = 'Pat' # 通过赋值创
建键>>> D['job'] = 'dev'>>> D['age'] = 40 >>> D {'na
me':'Pat','job':'dev','age': 40}
```

Here, we're effectively using dictionary keys as field

names in a record that describes an imaginary person. In other roles, dictionaries can also be used to replace searching operations—indexing a dictionary by key is often the fastest way to code a search in Python. As you'll learn later, we can also make dictionaries by passing to the dict type name either keyword arguments (a special name=value syntax in function calls), or the result of zipping together sequences of keys and values obtained at runtime (e.g., from files). Both of the following make the same dictionary as the prior example and its equivalent {} literal form; the first requires string keys but tends to make for less typing (and subjective noise), and the second uses the zip built-in we'll study later in this part of the book:

```
python >>> pat1 = dict(name='Pat', job='dev', age=40) # 关键字参数
>>> pat1 {'name': 'Pat', 'job': 'dev', 'age': 40}
```

```
>>> pat2 = dict(zip(['name', 'job', 'age'], ['Pat', 'dev', 40])) # zip 配对
>>> pat2 {'name': 'Pat', 'job': 'dev', 'age': 40}
```

Notice how the left-to-right order of dictionary keys is always the same. Though not sequences, dictionary keys retain their insertion order, even in the presence of changes: the order of keys is the order in which keys were added. This wasn't the norm until Python 3.7, and prior to this, key order was scrambled and sometimes required separate ordering. The new ordering m

ay be more intuitive and will be assumed in this book but adds a sequence flavor to dictionaries not shared by other nonsequences. Like most mods, it also rewrites history and invalidates Python learning resources that cannot be updated as frequently as Python. Change is almost always a double-edged sword.

中文翻译

作为字面量书写时，字典用花括号编写，由一串“键: 值”对组成。每当我们需把一组值与键关联起来——描述某物的属性——字典就有用武之地。例如，考虑下面这个三条目字典，键为“name”、“job”和“age”，记录了一位虚构（而且完全普通、中立的）工人的属性：python >>> D = {'name': 'Pat', 'job': 'dev', 'age': 40} 我们可以按键索引这个字典，来取回和修改键关联的值。字典的索引操作使用与序列相同的语法，但方括号里的条目是一个键，而不是相对位置：python >>> D['name'] # 取键'name' 对应的值'Pat'>>> D['job'] = 'mgr' # 修改 Pat 的职位>>> D {'name': 'Pat', 'job': 'mgr', 'age': 40} 虽然花括号字面量形式确实在用，但更常见的做法可能是用别的方式把字典搭建起来（毕竟，程序运行前就把所有数据都搞清楚的情况很少见）。例如，下面的代码从空字典开始，一次一个键地填充它。与列表里被禁止的越界赋值不同，给新字典键赋值会创建这些键：python >>> D = {}>>> D['name'] = 'Pat' # 通过赋值创建键>>> D['job'] = 'dev'>>> D['age'] = 40>>> D {'name': 'Pat', 'job': 'dev', 'age': 40} 在这里，我们实际上是把字典的键当作记录中的字段名，用来描述一个虚构的人。在其他角色里，字典还可以用来替代搜索操作——按键索引字

典往往是 Python 中编码搜索的最快方式。你以后会学到，我们还可以把关键字参数 (keyword argument, 函数调用里一种特殊的 name=value 语法) 传给 dict 类型名来构造字典，或者传入把运行时得到的键、值序列压缩 (zip) 在一起的结果 (例如从文件读取)。下面两种方式都构造出与前面例子及其 {} 字面量形式相同的字典；第一种要求字符串键，但往往打字更少 (主观噪音也更少)，第二种使用我们将在本部分后面学习的 zip 内置函数：

```
python >>> pat1 = dict(name='Pat', job='dev', age=40) # 关键字参数
>>> pat1 {'name': 'Pat', 'job': 'dev', 'age': 40}
>>> pat2 = dict(zip(['name', 'job', 'age'], ['Pat', 'dev', 40])) # zip 配对
>>> pat2 {'name': 'Pat', 'job': 'dev', 'age': 40}
```

注意字典键的从左到右顺序总是一样的。虽然字典不是序列，但字典键保留它们的插入顺序 (insertion order)，即使有修改也是如此：键的顺序就是键被添加的顺序。这在 Python 3.7 之前并不是常态，此前键的顺序是乱序的，有时需要单独排序。新顺序可能更直观，本书将把它视为常态，但这给字典增添了一点其他非序列类型没有的“序列风味”。和大多数修改一样，它也重写了历史，让那些无法像 Python 一样频繁更新的学习资源失效。变化几乎总是一把双刃剑。

深度理解

·核心概念：字典的三大语法事实：①字面量 {'k': v, ...}；②D[k] 按键索引 (读和写)；③写新键即创建新键 (与列表越界赋值被禁止形成鲜明对比)。三种构造法：字面量、dict(key=value) 关键字参数、dict(zip(keys, values))

。

·底层实现：D['job'] = 'mgr' 是哈希表写入：对键'job' 计算哈希 → 定位桶 → 若键已存在则替换值；D['name'] 是哈希查找：算哈希 → 定位 → 返回值，平均 $O(1)$ 。{} 是空哈希表字面量。dict(zip(...)) 中 zip 把两个序列"拉链"成 (k, v) 元组序列，dict 构造器再逐对写入。插入顺序来自 CPython 3.6+ 的"紧凑字典"布局（indices 表 + entries 表，entries 按插入序追加）。

·设计原因：键必须是可哈希的（数字、字符串、元组）——哈希让查找从 $O(n)$ 变成 $O(1)$ ；"赋值即创建"让"动态记录"（运行时才知道有哪些字段）成为可能，这是脚本语言的看家本领。作者特意提到 Python 3.7 前的乱序历史：这是"版本升级会改变行为"的活教材。

·实际问题：D['key'] 不存在会抛 KeyError——真实代码里要么先'key' in D 判断、要么用 D.get('key', default)（下一小节详述）。dict(zip(列名, 一行数据)) 是处理表格数据的经典模式（CSV 表头与行配对）。

·初学者误区：①用 D[key] 读一个可能不存在的键——应改用 get；②以为字典键有序、可以像列表一样切片——D[1:2] 是错误，字典不是序列；③以为"键必须保持插入顺序"是语言契约——它是 3.7+ 的实现保证，跨语言 JSON 解析也不要依赖它（不过 Python 内基本可以放心）。

代码分析

```

>>> D = {'name': 'Pat', 'job': 'dev', 'age': 40}    # :
>>> D['name']                                     #
'Pat'
>>> D['job'] = 'mgr'                             # →
>>> D
{'name': 'Pat', 'job': 'mgr', 'age': 40}

>>> D = {}                                       #
>>> D['name'] = 'Pat'                           # =
>>> D['age'] = 40
>>> D
{'name': 'Pat', 'age': 40}

>>> pat1 = dict(name='Pat', job='dev', age=40)    #
>>> pat2 = dict(zip(['name', 'job', 'age'], ['Pat', 'dev', 40])) ...
>>> pat1 == pat2
True

```

·解释器如何处理：{'name': 'Pat', ...} 编译为 BUILDMAP + 若干 MAPADD 指令——虚拟机制造空哈希表，然后逐对执行 MAPADD：对键算哈希（str 的哈希按内容缓存），找桶位，插入条目。D['name'] 编译为 BINARYSUBSCR（与列表索引同一条指令！），执行时运行时类型分派——发现是 dict，就走哈希查找而非下标访问——同一个 [] 语法，两种对象两种语义，这是操作符重载/多态的又一次体现。D['job'] = 'mgr' 是 STORESUBSCR：哈希命中已存在的键 → 原地替换值对象引用（键对象不变）。dict(name='Pat', ...) 是调用 dict 类型对象：关键字参数被打包成字典传给构造器；dict(zip(...)) 则先由 zip 惰性产出 ('name', 'Pat') 等元组，构造器逐一写入。

·设计思想：同一语法 D[k] 同时承担“序列的偏移”与“映射的键”两种语义——Python 的 [] 是一个通用的“索引协议

”。而字典作为“最快的搜索表”，让 C 程序员要手写哈希表的活变成了一个字面量。

4.8.2 嵌套再探 (Nesting Revisited)

英文原文

In the prior example, we used a dictionary to describe a hypothetical person, with three keys. Suppose, though, that the information is more complex. Perhaps we need to record a first name and a last name, along with multiple job titles. This leads to another application of Python's object nesting in action. The following dictionary, coded all at once as a literal, captures more-structured information (again, indentation here and “...” in some REPLs are moot):

```
python >>> rec = {'name': {'first': 'Pat', 'last': 'Smith'}, ..  
.. 'jobs': ['dev', 'mgr'], ... 'age': 40.5}
```

Here, we again have a three-key dictionary at the top (keys “name,” “jobs,” and “age”), but the values have become more complex: a nested dictionary for the name to support multiple parts, and a nested list for the jobs to support multiple roles and future expansion. We can access the components of this structure much as we did for our list-based matrix earlier, but this time most indexes are dictionary keys, not list offsets:

```
python >>> rec['name'] # 'name' 是一个嵌套字典{'first': 'Pat', 'last': 'Smith'}
```

```
>>> rec['name']['last'] # 索引嵌套字典'Smith'
>>> rec['jobs'] # 'jobs' 是一个嵌套列表['dev','mgr']
>>> rec['jobs'][-1] # 索引嵌套列表'mgr'
>>> rec['jobs'].append('janitor') # 原地扩展 Pat 的职位列表
>>> rec {'name': {'first': 'Pat', 'last': 'Smith'}, 'jobs': ['dev', 'mgr', 'janitor'], 'age': 40.5}
```

Notice how the last operation here expands the nested jobs list—because the jobs list is a separate piece of memory from the dictionary that contains it, it can grow and shrink freely (the pop method we met earlier is one way to shrink objects). This may seem quite a trick to readers with backgrounds in more rigid languages, but is natural in Python. More fundamentally, this example demos the flexibility of Python's core object types. As you can see, nesting allows us to build up complex information structures directly and easily. Building a similar structure in a low-level language like C would be tedious and require much more code: we would have to lay out and declare structures and arrays, fill out values, link everything together, and so on. In Python, this is all automatic—running the expression creates the entire nested object structure for us. In fact, this is one of the main benefits of scripting languages like Python. Just as importantly, in lower-level languages we may have to allocate memory space for objects ahead of time and be careful to release it when we no longer need it. In Python, this is all aut

omatic: object memory is allotted as needed and freed when we lose the last reference to the object—by assigning its variable to something else, for example:

```
python >>> rec = 0 # 现在之前的对象空间被回收
```

Technically speaking, Python uses a scheme called garbage collection that reclaims unused memory as your program runs and frees you from having to manage such details in your code. In standard Python (a.k.a. CPython) this uses object reference counts primarily, along with a supplemental garbage collector for cycles. We'll study how this works later in Chapter 6; for now, it's enough to know that you can use objects freely, while Python handles their memory. Watch for a record structure similar to the one we just coded in Chapters 8, 9, and 27, where we'll use it to compare and contrast lists, dictionaries, tuples, named tuples, and classes—an array of data structure options with trade-offs we'll cover in full later. It's also worth noting that the record structure we used here can be saved in a file with a variety of techniques in Python, including its pickle module and support for language-neutral JSON (a data format that's strikingly similar to Python dictionary objects); more on such tools later in this book.

中文翻译

在前面的例子里，我们用三个键的字典描述了一个虚构的人。但假设信息更复杂呢？也许我们需要记录名和姓，再加上

多个职位头衔。这就引出了 Python 对象嵌套的另一个实际应用。下面这个一次写成的字面量字典，捕获了更结构化的信息（再说一遍，这里的缩进和某些 REPL 里的"..."无关紧要）：`python >>> rec = {'name': {'first': 'Pat', 'last': 'Smith'}, ... 'jobs': ['dev', 'mgr'], ... 'age': 40.5}` 这里，我们仍然有一个三个键的顶层字典（键 "name"、"jobs" 和 "age"），但值变得更复杂了：名字是一个嵌套字典，以支持多个组成部分；职位是一个嵌套列表，以支持多个角色和未来扩展。我们可以像前面访问基于列表的矩阵那样访问这个结构的各个部分，但这次大多数索引是字典键，而不是列表偏移：`python >>> rec['name']` # 'name' 是一个嵌套字典{'first': 'Pat', 'last': 'Smith'} `>>> rec['name']['last']` # 索引嵌套字典'Smith' `>>> rec['jobs']` # 'jobs' 是一个嵌套列表['dev', 'mgr'] `>>> rec['jobs'][-1]` # 索引嵌套列表'mgr' `>>> rec['jobs'].append('janitor')` # 原地扩展 Pat 的职位列表 `>>> rec` {'name': {'first': 'Pat', 'last': 'Smith'}, 'jobs': ['dev', 'mgr', 'janitor'], 'age': 40.5} 注意最后一个操作如何扩展了嵌套的 jobs 列表——因为 jobs 列表是与包含它的字典分离的一块内存，它可以自由地增长和收缩（我们前面遇到的 pop 方法是收缩对象的一种方式）。对来自更僵硬语言背景的读者来说，这可能像是戏法，但在 Python 里是自然的。更根本的是，这个例子演示了 Python 核心对象类型的灵活性。如你所见，嵌套让我们能直接而轻松地搭建复杂的信息结构。在 C 这样的低级语言里构建类似结构会非常繁琐，需要多得多的代码：我们必须设计和声明结构体与数组、填好值、把一切链接起来，等等。在 Python 中这一切都是自动的——运行表达式就为我们创建了完整的嵌套对象结构。事实上，这是 Python 这类脚本语言的主要好处之一。同

样重要的是：在低级语言里，我们可能不得不提前为对象分配内存空间，并在不再需要时小心地释放它。在 Python 中这一切都是自动的：对象内存按需分配，当我们失去对对象的最后一个引用时——例如把它的变量赋成别的值——内存就被释放：python >>> rec = 0 # 现在之前的对象空间被回收严格来说，Python 使用一种叫垃圾回收（garbage collection）的方案，在程序运行时回收未使用的内存，让你不必在代码中管理这些细节。在标准 Python（即 CPython）中，这主要使用对象引用计数（reference count），外加一个用于处理循环引用的补充性垃圾回收器。我们会在第 6 章研究它的工作原理；现在只需要知道：你可以自由使用对象，而 Python 负责它们的内存。注意一个与我们刚写的记录结构相似的结构，会在第 8、9、27 章再次出现，届时我们将用它来比较和对照列表、字典、元组、命名元组（named tuple）和类——一系列各有权衡的数据结构选项，后面会完整覆盖。还值得注意的是，我们这里用的记录结构可以用多种技术保存到文件里，包括 pickle 模块，以及对语言无关的 JSON（一种与 Python 字典对象惊人相似的数据格式）的支持；这类工具的更多内容在后面。

深度理解

·核心概念：嵌套字典+列表 = 一条“人话记录”：rec['name']['last']（键→键）、rec['jobs'][-1]（键→偏移）——不同类型的容器可以任意混搭、层层下钻。嵌套对象之间彼此独立（jobs 列表与字典是两块不同的内存），所以内层列表能独立伸缩。

·底层实现：rec['jobs'].append('janitor') 的求值链：①BINARYSUBSCR 从字典取出内层列表对象的引用（不是副

本)；②LOADMETHOD append 绑定到该列表；③CALL 执行 append 原地扩容。外层字典毫不知情也不需要知情——它只是持有一个指向列表的指针。rec = 0 则让 rec 这个名字不再指向记录字典：旧记录的引用计数减到 0，它的内存（连同嵌套的字典、列表）被立即归还给堆（CPython 的引用计数立即回收；对象图里的循环则由补充 GC 处理）。

·设计原因：为什么这么省事？因为 Python 选择“一切由运行时管理内存”：malloc/free 的苦活被垃圾回收接管，嵌套结构因此可以“一步建好、随意改造”。这在 C 里需要：声明 struct + 嵌套指针、逐个 malloc、用完逐个 free——繁琐且极易内存泄漏。

·实际问题：这份 rec 记录其实就是“字典版对象”——第 27 章会展示用 class 表达同样的东西。pickle 和 json.dumps(rec) 能让这种结构直接落盘/传网络（JSON 是字典的“外甥”，语法几乎一致）——这就是为什么“用字典表达数据”是 Python 世界的通用语。

·初学者误区：①以为 rec['jobs'] 返回的是拷贝——它返回引用，所以 append 是“真正改到了 rec 里”（这正是需求）；②以为 Python 的回收要等“程序结束”——CPython 引用计数归零就立刻回收（rec = 0 当场生效）；③把“对象内存自动管理”误解为“内存无限”——大对象仍要避免无谓持有（比如循环引用的日志缓存）。

代码分析

```

>>> rec = {'name': {'first': 'Pat', 'last': 'Smith'}, #
...        'jobs': ['dev', 'mgr'], #
...        'age': 40.5}
>>> rec['name']['last'] # →
'Smith'
>>> rec['jobs'][-1] # →
'mgr'
>>> rec['jobs'].append('janitor') #
>>> rec['jobs']
['dev', 'mgr', 'janitor']

>>> rec = 0 # →
# →

```

·解释器如何处理：字面量求值时，解释器从内到外构建：先建内层字典 {'first': 'Pat', 'last': 'Smith'} 和内层列表 ['dev', 'mgr']，再建外层字典，把内层的引用作为值填入。rec['name']['last']：第一次 BINARYSUBSCR 命中键'name'返回内层字典引用，第二次 BINARYSUBSCR 命中键'last'返回'Smith'。rec['jobs'].append('janitor')：取到列表引用 → 方法绑定 → 原地扩容追加，引用计数全程只增不减（字典还持有该列表）。rec = 0 执行 STORENAME：rec 的旧引用被替换，外层字典引用计数 0 → 触发 C 层的 t_pdealloc 递归释放：外层字典减内层字典/列表的计数，它们也归零并释放，'Smith'、'janitor' 等字符串对象若无人引用同样被回收。这就是 CPython 的引用计数（reference counting）；若对象图里存在互相引用（A 引用 B、B 引用 A），计数永不归零，就由补充的循环垃圾收集器（cyclic garbage collector，分代式）定期扫描回收。

·设计思想：这个例子浓缩了本章最重要的三个机制：嵌套（结构自由）、引用（下钻不拷贝）、垃圾回收（内存自

动管理)。作者说"rec = 0 就能回收整棵对象树"——在 C 语言里这需要手动释放每一个嵌套节点。Python 的选择是：让程序员只关心结构，不关心内存。

4.8.3 缺失键：if 测试 (Missing Keys: if Tests)

英文原文

As mappings, dictionaries support accessing items by key only, with the sorts of operations we've just seen. In addition, though, they also support type-specific operations with methods that are useful in a variety of common roles. For example, the dictionary copy and update methods copy a dictionary and merge one into another in place, respectively (the `|` operator does the same, but makes a new dictionary for its result: it's just a copy plus an update). Dictionary methods also play parts in common key use cases. For instance, while we can assign to a new key to expand a dictionary, fetching a nonexistent key is still a mistake:

```
python >>> D = {'a': 1, 'b': 2, 'c': 3} >>> D['d'] = 4 #  
给新键赋值会增长字典 >>> D {'a': 1, 'b': 2, 'c': 3, 'd': 4}  
>>> D['e'] # 引用不存在的键是错误 KeyError: 'e'
```

This is what we want—it's usually a programming error to fetch something that isn't really there. But in generalized programs, we can't always know what keys will be present when we write our code. How do we handle such cases and avoid errors? One solution is to

test ahead of time. The dictionary in membership expression allows us to query the existence of a key and branch on the result with a Python if statement. Which brings us to our first Python compound statement—one with nested parts. If you're working along, here are a few practical bits: in the following, press the Enter key twice to run the if interactively after typing its code (an empty line means "go" in most REPLs, as explained in Chapter 3); the prompt changes to a "..." after the first line in some interfaces (as for the earlier multiline dictionaries and lists); and indentation matters this time (for reasons up next):

```
python >>>'e' in D # 布尔结果: True 或 False (后文详述) False
```

```
>>> if not'e' in D: # Python 的主要选择语句... print('missing key!') missing key!
```

This book has more to say about the if statement in later chapters, but its syntax is straightforward. It consists of the word if, followed by an expression whose result is interpreted as true or false, followed by a block of code to run if the expression result is true. In its full regalia, the if statement can also have an else clause for the false case, and one or more elif ("else if") clauses for other tests. Functionally, the if is the main selection tool in Python, and how we code most of the logic of choices and decisions in our scripts. It's joined by its ternary if/else expression cousin you'll meet

t in a moment, the if comprehension filter lookalike we used earlier, and the newer match multiple-selection statement you'll meet later in this book. If you've used some other programming languages in the past, you might now be wondering how Python knows when the if statement ends. We'll explore Python's syntax rules in depth in later chapters, but in short, if you have more than one action to run in a statement block, you simply indent all their statements the same way—which both promotes readable code and reduces the number of characters you have to type. All multiline statements follow this pattern: a header line ending in ":" and a block of (usually) indented code, with no "{}" around blocks, and no ";" required after statements (though fair warning: forgetting the ":" is the most common beginner's mistake in Python!):

```
python >>> if not'e' in D: ... print('missing') # 语句块
需要缩进... print('no, really...') # （除非是简单语句：见后
文） missing no, really...
```

Besides the in test, there are a variety of other ways to avoid accessing nonexistent keys in the dictionaries we create: the dictionary get method, which is a conditional index with a default; the if/else ternary (three-part) expression, which is essentially a limited if statement squeezed onto a single line; and the try statement, which is a tool we'll first use in Chapter 10 that catches and recovers from errors altogether. Here are the first two in action:

```
python >>> D.get('a','missing') # 类似 D['a'], 但带默认值1
```

```
>>> D.get('e','missing') # 键不存在时返回默认值'missing'
```

```
>>> D['e'] if 'e' in D else 0 # if/else 三元表达式形式0
```

We'll save the details on such alternatives until a later chapter. For now, let's turn to other dictionary methods' roles in another common use case.

中文翻译

作为映射，字典只支持按键访问条目，就是我们刚看到的那些操作。但除此之外，它们还支持用方法完成的类型特有操作，在许多常见场景中很有用。例如，字典的 `copy` 和 `update` 方法分别复制字典、把一个字典原地合并进另一个（`|` 运算符也做同样的事，但为结果生成一个新字典：它其实就是一个 `copy` 加一个 `update`）。字典方法在常见的键用法中也扮演重要角色。例如，虽然我们可以给新键赋值来扩展字典，但获取不存在的键仍然是个错误：`python >>> D = {'a': 1, 'b': 2, 'c': 3} >>> D['d'] = 4 # 给新键赋值会增长字典 >>> D {'a': 1, 'b': 2, 'c': 3, 'd': 4} >>> D['e'] # 引用不存在的键是错误 KeyError: 'e'`这正是我们想要的——获取并不存在的东西通常是编程错误。但在通用化的程序里，写代码时我们不可能总知道会有哪些键。怎么处理这些情况、避免错误呢？一个方案是提前测试。字典的 `in` 成员测试表达式让我们能查询键是否存在，并用 Python 的 `if` 语句根据结果分支。这就把我们带到了第一个 Python 复合语句（`compound statement`）——一种带嵌套部分的语句。如果你

跟着操作，这里有几个实用要点：下面输入 if 代码后要按两次回车键才能交互式运行（在大多数 REPL 里，空行意味着“执行”，第 3 章解释过）；第一行之后提示符会变成“...”（和前面多行字典、列表一样）；这次缩进很重要（原因马上讲）：`python >>>'e' in D` # 布尔结果：True 或 False（后文详述）`False >>> if not'e' in D:` # Python 的主要选择语句... `print('missing key!')` missing key! 本书后面章节还会讲更多 if 语句的内容，但它的语法很直白。它由单词 if、一个结果被解释为真或假的表达式、以及一个在表达式结果为真时运行的代码块组成。全套状态下，if 语句还可以有一个处理假情况的 else 子句，以及一个或多个 elif（“else if”，否则如果）子句做其他测试。从功能上说，if 是 Python 的主要选择（selection）工具，是我们脚本里大部分“抉择与决策”逻辑的编写方式。与它同行的还有：一会儿你会遇到的三元 if/else 表达式表亲、我们前面用过的 if 推导式过滤器（外形相近），以及本书后面会遇到的较新的 match 多重选择语句。如果你以前用过其他编程语言，你可能会好奇：Python 怎么知道 if 语句在哪里结束？我们后面会深入探讨 Python 的语法规则，但简而言之：如果语句块里有不止一个动作要执行，你只需把所有语句缩进成同样的方式——这既促进了可读代码，也减少了你要输入的字符数。所有多行语句都遵循这个模式：以“:”结尾的头行，加上一块（通常缩进的）代码，块周围没有“{}”，语句后也不需要“;”（不过要提醒你：忘了写“:”是 Python 初学者最常见的错误！）：`python >>> if not'e' in D: ... print('missing')` # 语句块需要缩进... `print('no, really...')` # （除非是简单语句：见后文）missing no, really... 除了 in 测试，还有很多其他方式可以避免访问我们创建的字典中不存

在的键：字典的 `get` 方法，它是一种带默认值的有条件索引；`if/else` 三元（三部分）表达式，本质上是把受限的 `if` 语句挤进一行；还有 `try` 语句，这是我们在第 10 章才会首次使用的工具，它把错误整体捕获并从中恢复。下面演示前两种：

```
python >>> D.get('a','missing') # 类似 D['a'],
但带默认值1 >>> D.get('e','missing') # 键不存在时返回
默认值'missing'>>> D['e'] if 'e' in D else 0 # if/else 三
元表达式形式0 这些替代方案的细节留到后面的章节。现在
，我们转向另一个常见场景中其他字典方法的作用。
```

深度理解

·核心概念：缺失键的四种处理策略，按“防御程度”递进：

①先 `'e' in D` 再取（显式测试）；②`D.get('e', default)`（带默认值的索引）；③三元表达式 `D['e'] if 'e' in D else 0`（一行版测试）；④`try/except KeyError`（第 10 章，异常处理）。同时本章首次出现复合语句：以 `:` 结尾的头行 + 缩进代码块——这是 Python 全部控制流的语法模板。

·底层实现：`'e' in D` 编译为 `CONTAINSOP`，C 层对 dict 做哈希查找，返回布尔对象。`if` 语句编译为条件跳转：先求值条件表达式，`not` 翻转布尔值，`POPJUMPIFFALSE` 决定是否进入语句块。语句块的范围完全由缩进（`INDENT/DEDENT token`）界定——词法分析器根据行首空格增减发出 `INDENT/DEDENT` 记号，语法分析器据此把同一缩进级的语句归入一个块。`D.get('e','missing')` 是 C 层哈希查找失败后返回默认参数的实现（成功则返回值本身）。

·设计原因：为什么 `D['e']` 报错而不是返回 `None`？因为“取不存在的东西”通常是逻辑 bug，报错（fail-fast）让 bug 在开发期现形；而 `get` 是“我知道可能没有”的显式表达。默认行为保守、逃生舱显式——这是 Python 错误哲学的一贯风格。缩进替代 `{} / end`，则消除了“花括号放哪儿”的争论，还强制了排版（可读性优先）。

·实际问题：解析配置文件/请求参数时，`D.get('name', '')` 几乎必用；统计词频 `counts[w] = counts.get(w, 0) + 1` 是教科书级惯用法；`if 'admin' in roles:` 是权限判断的标准写法。三元表达式适合“一个表达式二选一”（如返回默认值），多个分支还是用 `if/elif/else`。

·初学者误区：①忘记：或缩进不一致——`SyntaxError/IndentationError`，这是 Python 初学者第一大错误源；②用 `D['e']` 读可能不存在的键导致程序崩溃——应改用 `get / 三元/in`；③认为 `get` 与 `D['e']` 完全等价——`get` 只在“可能缺失”时使用，否则反而掩盖 bug；④对“缩进即语法”不理解，从别的语言带来“缩进只是风格”的惯性。

代码分析

```
>>> D = {'a': 1, 'b': 2, 'c': 3}
>>> D['d'] = 4                                # =
>>> D['e']                                    # → KeyError
KeyError: 'e'

>>> 'e' in D                                  #
False

>>> if not 'e' in D:                          # if +
...     print('missing key!')
missing key!
```

```

>>> if not 'e' in D:
...     print('missing')           #
...     print('no, really...')
missing
no, really...

>>> D.get('a', 'missing')          # get
1
>>> D.get('e', 'missing')          # get
'missing'
>>> D['e'] if 'e' in D else 0      #
0

```

·解释器如何处理：D['e'] 的哈希查找在 C 层"未命中"，随即创建一个 KeyError 实例抛出——异常沿调用栈传播，REPL 中没有处理者，于是打印 traceback。'e' in D 走 CONTAINSOP 指令：查哈希表，未命中返回 False（注意：与索引不同，成员测试不抛异常）。if not 'e' in D: 的求值顺序：先 'e' in D → False; not False → True; POPJUMPIFFALSE 不跳转 → 进入缩进块执行 print。第二个 if 演示块内两条语句：它们共享同一缩进级，词法分析器只发一次 INDENT，两条 print 同属一个块。D.get('e', 'missing')：C 层查找键'e'，未命中 → 返回第二个参数'missing'。三元表达式 D['e'] if 'e' in D else 0 编译为：求值条件 → 真则求左支 (D['e'])，假则求右支 (0) ——只有被选中的分支会被求值（短路）。

·设计思想：D['e']（会炸）与 D.get('e', default)（不炸）的分工，是"默认严格、可选宽容"的设计；而缩进式块是 Python 最著名的取舍——牺牲灵活性，换取强制可读性。作者特意预告 try 和 match，说明"处理缺失键"有多

条路线，按场景选择。

4.8.4 条目迭代：for 循环 (Item Iteration: for Loops)

英文原文

Dictionaries collect a lot of useful info, but what do we do if we need to process their items one at a time? As it turns out, dictionaries come with methods designed for the job:

```
python >>> D = dict(a=1, b=2, c=3) >>> D {'a': 1, 'b': 2, 'c': 3}
```

```
>>> list(D.keys()) # 键、值、键值对['a','b','c']
```

```
>>> list(D.values()) # list 强制结果全部生成[1, 2, 3]
```

```
>>> list(D.items()) [('a', 1), ('b', 2), ('c', 3)]
```

As shown, a dictionary's keys, values, and items methods return its keys, values, and key/value pairs (the latter is tuples, up next on this tour). Really, though, these methods all return an object that produces results one at a time, which is why they've been wrapped in list calls, as we did for range earlier. This reflects the iteration protocol in Python—a concept we'll explore in full later, but which boils down to an iterable object, which has an iterator object, which responds to next calls to produce one result at a time:

```
python >>> D.keys() # 得到一个可迭代对象dictkeys(['a
```

```
','b','c'])
```

```
>>> l = iter(D.keys()) # 从可迭代对象得到迭代器>>> n  
ext(l) # 每次从迭代器取一个结果'a'
```

```
>>> next(l) # 迭代协议的大部分就是这样'b'
```

We used the same `next` built-in to force results from a generator comprehension earlier, but without the `iter` step: because generators don't support multiple scans, they are their own iterators, and `iter` is a no-op. Tools that support this protocol can both save memory and minimize delays, because they don't produce all their results at once. The iteration protocol works on all sorts of objects in Python, but you can usually forget its details if you use the Python `for` loop, which runs the iteration protocol automatically to step through items one at a time—both for physical collections like strings and lists, and virtual sequences like generators, `range`, and `keys`:

```
python >>> for key in D.keys(): # 自动运行迭代协议...  
print(key,'=>', D[key]) # 显示多个条目, 以空格分隔  
a => 1 b => 2 c => 3
```

To code a `for`, provide a variable (e.g., `key`) and an iterable object (e.g., `D.keys()`); for each item in the object, the `for` assigns the item to the variable and runs the nested (and usually indented) block of code—which uses the variable to refer to the current item each time through. The `for` is one of the main repetition tools in Python, together with the comprehensions you met earlier, and the more general-

purpose while loop you'll meet later in this book. Because dictionary iteration is so common, the `for`, and similar iteration tools, can also step through keys implicitly, as well as key/value pairs. The following loops, for example, produce the same output as the preceding example; the choice between all these forms is partly a matter of personal preference, though explicit is generally better:

```
python >>> for key in D: # 隐式的 keys() 迭代... print(
key,'=>', D[key]) >>> for (key, value) in D.items(): #
键值对元组的迭代... print(key,'=>', value) The last of th
ese uses something known as tuple assignment, whic
h automatically unpacks items into variables. But to f
ully understand the sorts of stuff we get back from th
e dictionary items method, we have to move ahead.
```

中文翻译

字典收集了大量有用的信息，但如果我们需要逐个处理它的条目，该怎么做呢？事实证明，字典自带为此设计的方法：

```
python >>> D = dict(a=1, b=2, c=3) >>> D {'a': 1, 'b':
2, 'c': 3} >>> list(D.keys()) # 键、值、键值对['a', 'b', 'c'] >
>> list(D.values()) # list 强制结果全部生成[1, 2, 3] >>>
list(D.items()) [('a', 1), ('b', 2), ('c', 3)] 如图所示，字典的 k
eys、values 和 items 方法分别返回它的键、值和键/值对
（键值对是元组，是本次巡游的下一个主题）。不过说实话
，这些方法返回的都是"一次产生一个结果"的对象，所以它
们都被 list 调用包了起来，就像我们前面给 range 做的一
样。这反映了 Python 的迭代协议（iteration protocol）
```

——我们后面会完整探讨这个概念，但它可以归结为：一个可迭代对象（iterable）拥有一个迭代器对象（iterator），迭代器响应 next 调用，一次产生一个结果：python

```
>>> D.keys() # 得到一个可迭代对象dictkeys(['a','b','c'])
>>> I = iter(D.keys()) # 从可迭代对象得到迭代器
>>> next(I) # 每次从迭代器取一个结果'a'
>>> next(I) # 迭代协议的大部分就是这样'b'

我们前面用同一个 next 内置函数从生成器推导式里强制取结果，但省了 iter 那一步：因为生成器不支持多次扫描，它们自身就是自己的迭代器，iter 是个空操作（no-op）。支持这个协议的工具既能节省内存，又能把延迟降到最低，因为它们不会一次性产生全部结果。迭代协议作用于 Python 里各种对象，但如果你用 Python 的 for 循环，通常可以忘掉它的细节——for 会自动运行迭代协议逐个遍历条目——无论是字符串、列表这样的物理集合，还是生成器、range、keys 这样的虚拟序列：python
```

```
>>> for key in D.keys(): # 自动运行迭代协议... print(key,'=>', D[key]) # 显示多个条目，以空格分隔
a => 1
b => 2
c => 3
```

要编写 for，提供一个变量（例如 key）和一个可迭代对象（例如 D.keys()）；对该对象中的每个条目，for 把条目赋给变量并运行嵌套的（通常缩进的）代码块——每次循环代码用这个变量引用当前条目。for 是 Python 的主要重复（repetition）工具之一，与前面遇到的推导式，以及本书后面更通用的 while 循环并列。因为字典迭代太常见，for 及类似的迭代工具也可以隐式地遍历键，以及键/值对。例如，下面的循环产生与前面例子相同的输出；这些形式之间的选择部分是个人的偏好问题，不过一般说来显式更好：python

```
>>> for key in D: # 隐式的 keys() 迭代... print(key,'=>', D[key])
>>> for (key, value) in D
```


.items(): # 键值对元组的迭代... print(key,'=>', value) 最后这种写法用到了元组赋值 (tuple assignment), 它自动把条目拆包 (unpack) 进变量。但要完全理解我们从字典 items 方法拿回来的到底是什么东西, 我们得继续往前走。

深度理解

·核心概念: 迭代协议 = "一次一个、按需供给"的生产线: iter(obj) 取迭代器 → next(iter) 逐个取结果。keys/values/items 返回视图对象 (view), 不是立即生成的列表——这就是为什么必须用 list() 包一层才能看到全部内容。for 循环是协议的自动版。

·底层实现: D.keys() 返回 dictkeys 视图——它不复制任何东西, 只持有字典的引用, 迭代时实时查哈希表的插入序数组。iter() 编译为 GETITER 指令 (C 层调 tpiter 槽位), next() 编译为 GETITER 对应的 NEXT 指令 (FORITER 内部也用)。for key in D 的字节码是: GETITER → FORITER (内部调 next) → 每次把结果 STORENAME 到 key → 执行循环体 → 回到 FORITER 直到 StopIteration 异常 (迭代器耗尽时抛出, 被 FORITER 静默捕获, 循环结束)。生成器是"自带迭代器的可迭代对象": 对它 iter() 返回它自己。

·设计原因: 视图/惰性设计让迭代不复制数据、不预产全部结果——处理 1 亿条记录时内存占用是 $O(1)$ 而不是 $O(n)$; for 循环把协议细节全部封装, 日常代码几乎永远不直接碰 next。

·实际问题：for k, v in D.items(): 是遍历字典的绝对主流写法；for line in open('data.txt')（后面文件节）说明协议统一了文件、字典、range、生成器的遍历体验。sum(x for x in data)、any(...)、sorted(...) 都吃协议——一个协议，到处通吃。

·初学者误区：①以为 D.keys() 是列表——它是视图，要 list() 转换；②在 for 循环遍历字典时修改字典大小（增删键）——会抛 RuntimeError ("dictionary changed size during iteration")；③以为 next() 取完会返回 None——它抛 StopIteration，这正是 for 循环内部依赖的信号。

代码分析

```
>>> D = dict(a=1, b=2, c=3)
>>> list(D.keys())                # keys() list()
['a', 'b', 'c']
>>> list(D.values())
[1, 2, 3]
>>> list(D.items())               # (, )
[('a', 1), ('b', 2), ('c', 3)]

>>> I = iter(D.keys())           # iter()
>>> next(I)                      # next()
'a'
>>> next(I)
'b'

>>> for key in D.keys():         # for
...     print(key, '=>', D[key])
a => 1
b => 2
c => 3
```

```
>>> for key in D:                                     #
...     print(key, '=>', D[key])
>>> for (key, value) in D.items():                     # items() (, )
...     print(key, '=>', value)
```

·解释器如何处理：list(D.keys())：先调用 D.keys() 得到 dictkeys 视图对象（不复制键！），再调用 list()——list 构造器对任何可迭代对象调用迭代协议：内部 iter() 获得视图的迭代器，然后循环 next() 直到 StopIteration，把每个结果追加进新列表。iter(D.keys()) 编译为 GETITER：对 dictkeys 调用其 tpiter 槽位，返回一个 dictkeyiterator 对象；next(l) 编译为 GETITER 外的专用指令，C 层直接查哈希表的条目数组，按插入序返回下一个键。for key in D 里，D 本身也是可迭代对象（迭代它产生键）——这是“默认迭代语义”：对字典 for 循环 = 对 keys() 循环。for (key, value) in D.items()：FORITER 每次取回一个二元素元组，元组赋值（UNPACKSEQUENCE 2 指令）把它拆开绑定到 key 和 value。

·设计思想：iterable → iterator → next 的三级管线，是 Python 所有“逐个处理”场景的统一抽象；for 是它的“自动驾驶”，而生成器、视图、range 都是“虚拟序列”——能遍历的东西不需要存在，按需生产即可。这就是为什么 1 亿行文件也能一行行读。

4.9 元组 (Tuples)

英文原文

The tuple object (pronounced "toople" or "tuhple," depending on whom you ask) is roughly like a list that cannot be changed—tuples are sequences, like lists, but they are immutable, like strings. Functionally, they're used to represent fixed collections of items: the components of a specific calendar date, for instance.

Syntactically, they are normally coded in parentheses instead of square brackets and support arbitrary object types, arbitrary nesting, and the usual sequence operations that we used on strings and lists earlier:

```
python >> T = (1, 2, 3, 4) # 一个 4 条目元组 >> len(T)
# 长度 4 >> T + (5, 6) # 拼接: 生成新元组 (1, 2, 3, 4, 5,
6) >> T[0], T[1:] # 索引、切片等等 (1, (2, 3, 4))
```

As usual, tuples also have type-specific callable methods, but not nearly as many as lists:

```
python >> T.index(4) # 元组方法: 4 出现在偏移 3 处 3
>> T.count(4) # 4 出现了 1 次 1
```

The primary distinction for tuples is that they cannot be changed once created. That is, they are immutable sequences (quirk: one-item tuples like the one here require a trailing comma to distinguish them from simple expressions):

```
python >> T[0] = 2 # 元组不可变 TypeError: 'tuple' object
does not support item assignment >> T = (2,) + T[1:]
# 想要新值就构造新元组 >> T (2, 2, 3, 4)
```

Like lists and dictionaries, tuples support mixed types and nesting, but they don't grow and shrink like lists and dictionaries because they are immutable:

```
python >> T = 'hack', 3.0, [11, 22, 33] # 括号常常可以省略
>> T ('hack', 3.0, [11, 22, 33]) >> T[1] 3.0 >> T[2]
[1] 22 >> T.append(4) AttributeError: 'tuple' object has no attribute 'append'
```

Notice the first line in this: the parentheses enclosing a tuple's items can often be omitted, as done here. In contexts where commas don't otherwise matter, the commas are what actually builds a tuple. This also explains why REPLs show results in parentheses when you enter multiple items separated by commas: the input is really a tuple.

中文翻译

元组 (tuple) 对象 (发音是 "toople" 还是 "tuhple", 取决于你问谁) 大致像一个不能改变的列表——元组和列表一样是序列, 但和字符串一样是不可变的。从功能上说, 它们用来表示固定的条目集合: 例如某个日历日期的各个组成部分。从语法上说, 它们通常用圆括号而不是方括号编写, 支持任意对象类型、任意嵌套, 以及我们之前在字符串和列表上用过的常规序列操作: `python >>> T = (1, 2, 3, 4) # 一个 4 条目元组`
`>>> len(T) # 长度4`
`>>> T + (5, 6) # 拼接: 生成新元组(1, 2, 3, 4, 5, 6)`
`>>> T[0], T[1:] # 索引、切片等等(1, (2, 3, 4))` 照例, 元组也有类型特有的可调用方法, 但远没有列表那么多: `python >>> T.index(4) # 元组方法: 4 出现在偏移 3 处`
`>>> T.count(4) # 4 出现了`

1 次1 元组的主要区别在于它们一旦创建就不能改变。也就是说，它们是不可变序列（小怪癖：像这里这样的一条目元组需要尾随逗号，才能与简单表达式区分开）：`python >>> T[0] = 2 # 元组不可变TypeError:'tuple' object does not support item assignment >>> T = (2,) + T[1:] # 想要新值就构造新元组>>> T (2, 2, 3, 4)` 与列表和字典一样，元组支持混合类型和嵌套，但和列表、字典不同，它们不能增长和收缩，因为它们是不可变的：`python >>> T = 'hack', 3.0, [11, 22, 33] # 括号常常可以省略>>> T ('hack', 3.0, [11, 22, 33]) >>> T[1] 3.0 >>> T[2][1] 22 >>> T.append(4) AttributeError:'tuple' object has no attribute 'append'`注意这里的第一行：包围元组条目的圆括号常常可以省略，就像这里这样。在逗号没有其他作用的语境里，是逗号真正构造了元组。这也解释了为什么你在 REPL 里输入多个逗号分隔的条目时，结果会显示在圆括号里：输入实际上就是一个元组。

深度理解

·核心概念：元组 = 序列 + 不可变。“像列表一样有序，像字符串一样不可改”。语法上有个隐藏的真相：构造元组的是逗号，不是圆括号——`1, 2, 3` 就是元组，`(1,)` 的尾逗号是区分“单元素元组”与“普通括号表达式”的关键。

·底层实现：tuple 的 C 结构与 list 几乎一样（都是指针数组 + 长度），只是没有任何修改槽位（没有 `append/pop/原地赋值`）——`T[0] = 2` 与字符串一样抛 `TypeError`；`T.append` 是属性查找失败抛 `AttributeError`。`T = (2,) + T[1:]` 构造了一个全新的元组对象（先把 2 放进新数组，

再拷贝切片部分)，旧元组被回收。因为不可变，tuple 可以被安全共享、可以哈希（所以能当字典键——后续章节讲）。

·设计原因："只读集合"是给代码的契约：数据结构和函数签名里写元组，等于声明"这份数据不可修改"。作者给的理由是完整性约束（integrity constraint）——程序越大，这种约束越值钱。同时不可变也让元组可哈希、可比较、可作字典键。

·实际问题：函数返回"多个值"时用的是元组（return x, y）；for (k, v) in D.items() 里解包的就是元组；坐标 (x, y)、颜色 (r, g, b)、日期 (2026, 8, 5)——凡是"定长的字段组合"，元组是默认选择。字典.items() 返回的元组集可以直接 dict(items) 再转回字典。

·初学者误区：①写 T = (5) 以为是一条目元组——其实只是数字 5，必须 (5,); ②以为元组"完全不可变"——元组里装的可变对象（如列表）仍可修改（T[2][1] = 99 合法，因为改的是列表不是元组），这叫"浅不可变"；③既然有元组为何还要列表——两者用途不同：定长/契约用元组，动态/可变集合用列表。

代码分析

```

>>> T = (1, 2, 3, 4) # 4
>>> len(T) # 4
4
>>> T + (5, 6) #
(1, 2, 3, 4, 5, 6)
>>> T.index(4) # 4
3
>>> T.count(4) # 4
1

>>> T[0] = 2 #
TypeError: 'tuple' object does not support item assignment
>>> T = (2,) + T[1:] # (2,)
(2, 2, 3, 4)

>>> T = 'hack', 3.0, [11, 22, 33] #
>>> T[2][1] #
22
>>> T.append(4) # append
AttributeError: 'tuple' object has no attribute 'append'

```

·解释器如何处理：T = (1, 2, 3, 4) 编译为 BUILDtuple 4（与 BUILDlist 同族的指令）——堆上分配固定大小的指针数组并填入常量引用。T + (5, 6) 走 tupleconcat：分配 6 槽新数组，拷贝两组指针，生成新元组——原 T 不变。T.index(4) 是 O(n) 线性扫描（元组只有 index/count 两个方法，都没有哈希加速）；T.count(4) 同理。T[0] = 2 的 STORESUBSCR 检查 tuple 的条目赋值槽位——不存在 → TypeError。T = 'hack', 3.0, [11, 22, 33]：词法上这是三个逗号分隔的表达式，语法分析器看到“无括号的逗号序列”直接构建元组——因此逗号是核心，括号是装饰。T[2][1] 两次下标，第二下落在内层列表上——列表可变，取 22 没问题；若改成 T[2][1] = 99 也合法（

元组的"浅不可变")。

·设计思想：元组把"不可变"推到极致——连方法都精简到只剩两个 (index/count)。它存在的意义不在"更多功能"，而在"更少功能"：少掉的 append/pop 恰恰是它在多线程、多引用场景下能被安全共享的原因。

4.9.1 为什么用元组？ (Why Tuples?)

英文原文

So, why have a kind of object that is like a list, but supports fewer operations? Frankly, tuples are not used as often as lists in practice, but their immutability is the whole point. If you pass a collection of objects around your program as a list, it can be changed anywhere; if you use a tuple, it cannot. That is, tuples provide a sort of integrity constraint that is convenient in programs larger than those here. We'll talk more about tuples later in the book, including an extension that builds upon them called named tuples. For now, though, let's move on to this tour's last major object.

中文翻译

那么，为什么要有一个像列表、却支持更少操作的对象类型呢？坦白说，元组在实践中不像列表用得那么多，但它的不可变性就是全部意义所在。如果你把一个对象集合以列表的形式在程序里传递，它可能在任何一个地方被修改；如果用的是元组，它就不能被修改。也就是说，元组提供了一种完整性约束 (integrity constraint)，在比这里更大的程序

中非常方便。本书后面还会更多地讨论元组，包括一个建立在它们之上的扩展——命名元组（named tuple）。不过现在，让我们继续本次巡游的最后一个主要对象。

深度理解

·核心概念：元组的价值 = 不可变性本身。传递数据时的"只读保证"：列表会"到处可改"，元组"谁也不能改"。这是接口设计上的约束武器。

·底层实现：正因为不可变，元组可以被哈希（作字典键、集合元素），可以被缓存复用（CPython 对空元组、常用小元组有缓存），也能在解释器内部安全共享——可变对象享受不到这些待遇。

·设计原因：现实世界的类比——合同。合同一旦签定就不该被随便涂改（元组），草稿纸可以随便改（列表）。函数返回的"多个结果"、字典的 items、坐标点，都是"签好字的合同"。

·实际问题：想让人"只能读、不能改"的参数，传元组；函数返回多值时用元组并立刻解包；dict.items() 返回元组集合保证遍历时不被误改。数据结构越复杂、传递链条越长，完整性约束越有价值（作者："convenient in programs larger than those here"）。

·初学者误区：①以为元组是"劣化版列表"，能用列表就不用元组——恰恰相反，语义上"不应被改"的数据就该用元组；②以为"元组不可变 = 元组里的一切不可变"——浅不可变的细节（列表元素仍可改）常被忽略；③不知 namedtuple 的存在——它把元组升级成"带字段名的元组"，

4.10 文件 (Files)

英文原文

File objects are the main way your Python code will access the content of files on your computer. They can be used to read and write text memos, audio clips, Excel documents, saved emails, and whatever else you have stored on your device. Files are a core object type, but they're something of an oddball—there is no specific literal syntax for creating them.

Rather, you create a file object by calling the built-in `open` function with an external filename, and perhaps more depending on your goals.

For example, to create a text output file, pass in its name and the 'w' processing mode string to write text data; Python automatically makes the newline character `\n` portable across platforms when it's transferred to and from files:

```
python >> f = open('data.txt','w') # 以文本输出模式打开新文件
>> f.write('Hello\n') # 向文件写入字符串
6 >> f.write('world!\n') # 返回写入的条目数
7 >> f.close() # 关闭文件，把缓冲刷新到磁盘
```

This creates a file in the current directory and writes text to it (the filename can be a full directory path if y

ou need to access a file elsewhere on your device). To read back what you just wrote, reopen the file in 'r' processing mode, for reading text input—this is also the default if you omit the mode in the call. Then read the file's content into a string and access it.

A file's content is always a string in your script, regardless of the type of data the file contains:

```
python >> f = open('data.txt') # 以文本输入模式打开已有文件
>> text = f.read() # 把整个文件读入字符串
>> text
'Hello\nworld!\n'
>> print(text) # print 会解释控制字符
Hello world!
```

```
>> text.split() # 文件内容永远是字符串 ['Hello','world!']
```

Other file object methods support additional features we don't have time to cover here. For instance, file objects provide more ways of reading and writing (read accepts an optional maximum byte/character size, readline reads one line at a time, and so on), as well as other tools (seek moves to a new file position).

As you'll see later, though, the best way to read a text file is usually to not read it at all—files support the iteration protocol with an iterator that automatically reads line by line in for loops and other contexts:

```
python >> for line in open('data.txt'): # 显示文件中的每一行
... print(line.rstrip()) # 单倍行距 (去掉 \n)
Hello world!
```

You'll meet the full set of file methods later in this book, but if you want a quick preview now, run a `dir` call on any open file and a `help` on any of the method names that come back, as we learned earlier.

中文翻译

文件 (file) 对象是 Python 代码访问计算机上文件内容的主要方式。它们可以用来读写文本备忘录、音频片段、Excel 文档、保存的电子邮件，以及你设备上存储的任何其他东西。文件是核心对象类型，但它多少有点“异类”——没有专门的字面量语法来创建它。相反，你要调用内置函数 `open` 并传入一个外部文件名来创建文件对象，根据目标不同也许还要传更多参数。例如，要创建一个文本输出文件，传入它的名字和 'w' 处理模式字符串，用于写入文本数据；Python 会自动让换行符 `\n` 在跨平台地进出文件时保持可移植：

```
python >>> f = open('data.txt','w') # 以文本输出模式打开新文件
>>> f.write('Hello\n') # 向文件写入字符串
6 >>> f.write('world!\n') # 返回写入的条目数
7 >>> f.close() # 关闭文件，把缓冲刷新到磁盘这会在当前目录创建一个文件并向它写入文本（如果你需要访问设备上其他位置的文件，文件名可以是完整的目录路径）。要读回刚写的内容，以 'r' 处理模式重新打开文件，用于读入文本输入——如果你在调用中省略模式，这也是默认值。然后把文件内容读入字符串再访问它。无论文件包含什么类型的数据，在你的脚本里文件内容永远是字符串：
```

```
python >>> f = open('data.txt') # 以文本输入模式打开已有文件
>>> text = f.read() # 把整个文件读入字符串
>>> text
'Hello\nworld!\n'
>>> print(text) # print 会解释控制字符
Hello world!
```

```
>>> text.split() # 文件内容永远是字符串['Hello','world!']
```

文件对象的其他方法支持更多这里来不及覆盖的特性。例如，文件对象提供更多读写方式（read 接受可选的最大字节/字符数，readline 一次读一行，等等），以及其他工具（seek 移动到新的文件位置）。不过，正如你以后会看到的，读取文本文件的最佳方式通常是根本不读——文件支持迭代协议，其迭代器在 for 循环和其他语境中自动逐行读取：

```
python >>> for line in open('data.txt'): # 显示文件中的每一行... print(line.rstrip()) # 单倍行距（去掉 \n）
```

Hello world! 本书后面你会见到完整的文件方法集合，但现在想快速预览的话，可以对任何打开的文件运行 dir，再对返回的任意方法名运行 help，正如我们前面学过的。

深度理解

·核心概念：文件 = 核心对象中的“异类”（oddball）：没有字面量语法，必须调用 open() 工厂函数。文件内容的两种形态：读进脚本永远是字符串（文本文件）——无论文件里装的是什么；文本模式 vs 二进制模式决定 str 还是 bytes。close() 把缓冲刷到磁盘。

·底层实现：open() 在 C 层调用操作系统 API（Windows 的 CreateFile / POSIX 的 open），返回一个文件描述符（fd）包装对象；文件对象内部有 I/O 缓冲（buffer）。f.write('Hello\n') 把字符串编码（默认 UTF-8）后写入内存缓冲，返回写入的字符数（6）；f.close() 刷缓冲（flush）并关闭 fd。文本模式下，换行符做平台翻译：Windows 上写 \n 实际存 \r\n，读回来又转成 \n——这就是“换行可移植”的含义。for line in open('data.txt') 利用了迭

代协议：文件的迭代器按行缓冲读取（读一块、切出一行、再读），1 GB 文件也不会整体进内存。

·设计原因：文本/二进制分离 + 自动编解码，让“读文件”这件脏活变得无感；“按行迭代”把文件变成了“行的序列”——与列表、range 统一在同一个 for 之下（迭代协议的一统江湖）。

·实际问题：真实项目几乎从不裸用 open() 而不关——现代标准写法是 with open('data.txt') as f:（上下文管理器，自动关闭，第 9 章讲）；open(..., encoding='utf-8') 显式声明编码是跨平台代码的纪律；for line in f 是处理大文件的唯一正确姿势。

·初学者误区：①打开文件忘了 close——在 Windows 上会锁文件导致后续读写失败（教训：用 with）；②把 f.write 的返回值（字符数）当错误——它正常返回写入数；③以为 f.read() 读到的是 bytes——默认文本模式返回 str；读中文文件报 UnicodeDecodeError，多半是没指定 encoding。

代码分析

```
>>> f = open('data.txt', 'w')           #
>>> f.write('Hello\n')                  # 6
6
>>> f.write('world!\n')                  # 7
7
>>> f.close()                           #

>>> f = open('data.txt')                 # 'r'
>>> text = f.read()                      #
>>> text
```

```

'Hello\nworld!\n'
>>> print(text)                # print \n
Hello
world!
>>> text.split()               #
['Hello', 'world!']

>>> for line in open('data.txt'): #
...     print(line.rstrip())     # rstrip \n
Hello
world!

```

·解释器如何处理：open('data.txt','w')：C 层创建文本 I/O 对象，'w' 模式意味着"创建/截断"。f.write('Hello\n')：字符串按 UTF-8 编码成字节 → 写入内存缓冲 → 返回 6（字符数）。此时文件在磁盘上可能还是空的——数据还在缓冲里，直到 f.close()（或缓冲满、flush 调用）才真正落盘。open('data.txt') 省略模式 = 'r'；f.read() 不带参数读全部，字节解码回字符串。for line in open('data.txt')：open 返回的文本文件对象是迭代器，FORITER 每次 next 返回一行（含 \n），循环体 print(line.rstrip()) 去掉行尾换行输出。

·设计思想：文件被设计成"行的序列"，于是文件处理和列表处理共用同一套 for/迭代心智模型；缓冲与 close 的设计则提醒我们：程序与操作系统之间隔着一层缓冲，资源必须显式归还（with 语句是第 9 章的现代解法）。

4.10.1 Unicode 与字节文件 (Unicode and Byte Files)

英文原文

The prior section's examples illustrate file basics that suffice for many roles. Technically, though, they rely on the platform's Unicode encoding default for the host platform. Python text files always use a Unicode encoding to encode strings on writes and decode them on reads. This is often irrelevant for simple ASCII text data, which usually maps to and from file bytes unchanged. But for richer kinds of data, file interfaces can vary by content type. As hinted when we studied Unicode strings earlier, Python draws a sharp distinction between text and binary data in files: text files represent content as normal `str` strings and perform Unicode encoding and decoding automatically as noted, while binary files represent content as the special `bytes` string and allow you to access file content unaltered. For example, binary files are useful for processing media, accessing data created by C programs, and so on. What you send and receive is the literal content of the file, whether it's encoded text or a JPEG image. Add a `b` to the mode to invoke binary (and expect a `\r\n` instead of `\n` at the end on Windows, because its newlines vary from Unix):

```
python >>> bf = open('data.bin','wb') # 二进制写模式
>>> bf.write(b'h\xFFa\xEEc\xDDk\n') # 以 bytes 写入
二进制数据8
```

```
>>> bf.close() >>> open('data.bin','rb').read() # 以 bytes 读回二进制数据b'h\xffa\xeec\xddk\n'
```

For text files, we can't really talk about content without also asking, "What kind?"—files may use any Unicode encoding, especially if they came from another platform, or the internet at large. This applies to portable programs too: if you want your code to work across platforms, you should generally make encodings explicit to avoid unpleasant surprises. Luckily, this is easier than it may sound—simply pass in an encoding name to open to force an encoding:

```
python >>> tf = open('unidata.txt','w', encoding='utf-8') >>> tf.write('h\u00c4ck') # 编码为 UTF-8
```

```
>>> tf.close()
```

If you read with the same encoding (or one that's compatible), you get back the same text-character code points that you wrote. The encoded bytes on the file are in UTF-8 form, but your code usually doesn't need to care:

```
python >>> open('unidata.txt','r', encoding='utf-8').read() # 从 UTF-8 解码'hÄck'
```

```
>>> open('unidata.txt','rb').read() # 原始编码文本b'\xf0\x9f\x90\x8d\xc3\x84ck\xf0\x9f\x91\x8f'
```

While files automate most encodings, you can also encode and decode manually if your program gets Unicode data from another source—parsed from an email message or fetched over a network connection, for

example:

```
python >>> 'hÄck'.encode('utf-8') # 手动编码b'h\xc3\x84ck'
```

```
>>> b'h\xc3\x84ck'.decode('utf-8') # 手动解码'hÄck'
```

Python also supports non-ASCII file names (not just content), but it's largely automatic. For the whole story on Unicode in Python, stay tuned for Chapter 37.

中文翻译

前一节的例子演示了足以应付许多场景的文件基础。不过严格来说，它们依赖宿主平台的 Unicode 编码（encoding）默认值。Python 文本文件总是用某种 Unicode 编码在写入时编码字符串、在读取时解码。对简单的 ASCII 文本数据，这通常无关紧要——ASCII 进出文件字节时基本原样映射。但对更丰富的数据，文件接口会因内容类型而异。正如我们学习 Unicode 字符串时暗示过的，Python 对文件中的文本数据和二进制数据划了清晰的分界线：文本文件把内容表示为普通 str 字符串，并像前面说的那样自动进行 Unicode 编码和解码；而二进制文件把内容表示为特殊的 bytes 字符串，让你原封不动地访问文件内容。例如，二进制文件对处理媒体、访问 C 程序创建的数据等等很有用。你发送和接收的就是文件的字面内容，不管它是编码文本还是 JPEG 图片。在模式里加一个 b 就进入二进制模式（并且期待在 Windows 上出现 \r\n 而不是 \n 的行尾，因为 Windows 的换行与 Unix 不同）：
python >>> bf = open('data.bin','wb') # 二进制写模式
>>> bf.write(b'h\xFFa\xEEc\xDDk\n') # 以 bytes 写入二进制数据
>>> bf.close() >

```
>> open('data.bin','rb').read() # 以 bytes 读回二进制数据
b'h\xffa\xee\xddk\n' 对文本文件，不谈“哪种编码”就没法谈内容——文件可能用任何 Unicode 编码，尤其是来自其他平台或整个互联网的文件。这对可移植程序同样适用：
想让代码跨平台工作，一般应该显式指定编码以避免不愉快的意外。
幸运的是，这比听起来容易——只要把 encoding 名字传给 open 就能强制编码：
python >>> tf = open('unidata.txt','w', encoding='utf-8')
>>> tf.write('h\u00c4ck') # 编码为 UTF-8
6 >>> tf.close() 如果用相同（或兼容）的编码读回，你得到的就是写入时的相同文本字符码点。
文件上的编码字节是 UTF-8 形式的，但你的代码通常不必关心：
python >>> open('unidata.txt','r', encoding='utf-8').read() # 从 UTF-8 解码
'hÄck' >>> open('unidata.txt','rb').read() # 原始编码文本
b'h\xf0\x9f\x90\x8d\xh\xc3\x84ck\xf0\x9f\x91\x8f' 虽然文件自动处理了大多数编码，
但如果你的程序从其他来源拿到 Unicode 数据——比如从电子邮件消息解析出来、或者通过网络连接抓取——你也可以手动编码和解码：
python >>> 'hÄck'.encode('utf-8') # 手动编码
b'h\xc3\x84ck' >>> b'h\xc3\x84ck'.decode('utf-8') # 手动解码
'hÄck' Python 还支持非 ASCII 的文件名（不只是内容），但基本是全自动的。
想了解 Python 中 Unicode 的完整故事，敬请期待第 37 章。
```

深度理解

·核心概念：文件的双轨制——文本文件（str + 自动编解码）与二进制文件（bytes + 原样存取），模式字符 'w'/'r' 加 b 即二进制（'wb'、'rb'）。文本模式的自动编解码依据：未指定 encoding 时用系统默认（Windows 常见 cp

936/UTF-8, macOS/Linux 为 UTF-8) 。

·底层实现：文本文件对象在 C 层是 TextIOWrapper：写时把 str 编码为字节再交给缓冲 I/O，读时把字节解码回 str。'wb' 模式则直接暴露 BufferedWriter——bytes 原样进出，不做编码、不做换行翻译（Windows 上文本模式的 `\n` → `\r\n` 翻译在二进制模式下被关闭）。`tf.write('h\u00c4ck')` 返回 6（码点数），但落盘是 13 字节——读写之间的“数量差”由编码消化。

·设计原因：为什么二进制文件“原样”、文本文件“自动编解码”？因为文本必须先约定编码才有意义，而二进制（图片、音频、C 结构体）没有“编码”概念。把选择权交给程序员：跨平台代码显式声明 encoding，是消除乱码的工程纪律。

·实际问题：下载文件用'wb'；读 JSON/日志用 `open(..., encoding='utf-8')`；网络抓包到手的是 bytes，`data.decode('utf-8')` 还原文本；rb 读回文件能看到“原始编码字节”——这是排查“文件里到底存了什么”的侦查手段。

·初学者误区：①用'r' 模式读二进制文件或反之——会得到乱码/报错；②写中文文件不指定 encoding，Windows 上默认编码可能与读取方不一致导致乱码；③以为 write 的返回值是字节数——文本模式返回的是字符数（码点数）。

代码分析

```

>>> bf = open('data.bin', 'wb')      # bytes
>>> bf.write(b'h\xFFa\xEEc\xDDk\n') # 8
8
>>> bf.close()
>>> open('data.bin', 'rb').read()    #
b'h\xffa\xeec\xddk\n'

>>> tf = open('unidata.txt', 'w', encoding='utf-8') # UTF-8
>>> tf.write('h\u00c4ck')            # 6
6
>>> tf.close()
>>> open('unidata.txt', 'r', encoding='utf-8').read() #
'hÄck'
>>> open('unidata.txt', 'rb').read() # =4B, Ä=2B, =4B
b'\xf0\x9f\x90\x8d\xc3\x84ck\xf0\x9f\x91\x8f'

>>> 'hÄck'.encode('utf-8')           # →
b'h\xc3\x84ck'
>>> b'h\xc3\x84ck'.decode('utf-8')   # →
'hÄck'

```

·解释器如何处理：'wb' 模式创建不套 TextIOWrapper 的字节流对象——bf.write(b'...') 把 8 个原始字节直接送入缓冲；repr 显示时，可打印 ASCII (h、a、k、\n) 原样，其余显示为 \xNN。读回 b'h\xffa\xeec\xddk\n' 字节完全一致——二进制模式零加工。文本模式下，tf.write('hÄck') 经 UTF-8 编码器：(U+1F40D) → 4 字节、Ä (U+00C4) → 2 字节、其他各 1 字节，共 13 字节落盘；'r' 模式读回时按 UTF-8 解码器还原 6 个码点。open(...,'rb').read() 暴露的 13 字节揭示了编码的"变长"本质 (\xf0\x9f\x90\x8d = , \xc3\x84 = Ä)。手动 encode/decode 与文件自动编解码走的是同一套 C 层编解码器——手动版常用于文件之外的来源（网络、数据库

)。

·设计思想：写码点、存字节、读码点——文件是编解码管道的自动执行者，而'rb' 是给好奇心留的"后门"：想确认磁盘上到底存了什么，就以字节原样打开看。

4.10.2 其他类文件工具 (Other File-Like Tools)

英文原文

The open function is the workhorse for most file processing you will do in Python. For more advanced tasks, though, Python comes with additional file-like tools: pipes, FIFOs, sockets, keyed-access files, persistent object shelves, descriptor-based files, relational and object-oriented database interfaces, and more. We won't cover many of these topics in this language book, but you'll find them useful once you start programming Python in earnest.

中文翻译

open 函数是你在 Python 中大部分文件处理的主力 (workhorse)。不过，对更高级的任务，Python 还带有其他类文件工具：管道 (pipe)、FIFO、套接字 (socket)、按键访问文件、持久化对象架 (persistent object shelf)、基于描述符的文件、关系型和面向对象数据库接口，等等。这本语言书不会覆盖其中许多主题，但一旦你认真开始用 Python 编程，你会发现它们很有用。

深度理解

- 核心概念：文件只是一个起点——Python 里有一整个"类文件对象" (file-like object) 家族，它们共享 read/write 接口：管道、套接字、内存缓冲、压缩文件.....接口统一，代码几乎可以无缝替换。
- 底层实现：这些工具都实现了同样的"鸭子接口" (duck typing 的雏形)：只要对象有 .read() 就有资格被当文件用。例如 io.StringIO 让字符串伪装成文件；socket 对象也有 recv/send；gzip.open 返回的对象用法与 open 相同。
- 设计原因：统一接口让"数据从哪来"（磁盘/网络/内存/压缩包）对业务代码透明——面向接口编程是 Python 的默认姿势。
- 实际问题：内存中模拟文件（StringIO/BytesIO）、网络编程（socket）、进程间通信（subprocess 管道）、数据持久化（shelve/pickle）——这些是文件之外的真实世界。
- 初学者误区：以为"只有 open 才是文件"——只要接口一致，任何对象都能被 for 循环逐行迭代、被 read 读取；这与第 4.11 节讲的"面向接口而非类型"一脉相承。

4.11 其他对象类型 (Other Object Types)

英文原文

Beyond the core object types we've seen so far, there are others that get less publicity than their cohorts but are useful in their intended roles nonetheless. Let's quickly run down some of the stragglers in this category.

中文翻译

除了我们目前看到的核心对象类型，还有一些其他类型，它们不像同侪那样出名，但在各自的角色中同样有用。让我们快速过一遍这个类别里的“掉队者”。

4.11.1 集合 (Sets)

英文原文

Python sets are neither mappings nor sequences; rather, they are unordered collections of immutable (technically, "hashable") objects, which store each object just once. You create sets by calling the built-in set function with a sequence or other iterable, or by using a set literal expression, and sets support the usual mathematical set operations:

```
python >> X = set('hack') # 序列 => 集合 >> Y = {'a', 'p', 'p'} # 集合字面量 >> X, Y ({'c', 'k', 'a', 'h'}, {'p', 'a'}) >> X & Y, X | Y # 交集、并集 ({'a'}, {'p', 'c', 'k', 'h', 'a'}) >> X - Y, X > Y # 差集、超集判断 ({'c', 'k', 'h'}, False)
```

Even less mathematically inclined programmers often find sets useful for common tasks such as filtering out duplicates, isolating differences, and performing order-neutral equality tests without sorting:

```
python >> list(set([3, 1, 2, 1, 3, 1])) # 去重 [1, 2, 3] >>
set('code') - set('hack') # 集合差 {'d','o','e'} >> set('code') == set('deoc') # 与顺序无关的相等比较 True
```

As you'll see later in this part of the book, normal sets themselves are mutable and can be changed (with their remove and add methods, for example), though the immutable items within them by definition cannot.

中文翻译

Python 集合 (set) 既不是映射也不是序列；它们是不可变（严格说是“可哈希”，hashable）对象的无序集合，每个对象只存储一次。你可以调用内置函数 set 并传入一个序列或其他可迭代对象来创建集合，也可以使用集合字面量表达式，集合支持通常的数学集合运算：python >>> X = set('hack') # 序列 => 集合 >>> Y = {'a','p','p'} # 集合字面量 >>> X, Y ({'c','k','a','h'}, {'p','a'}) >>> X & Y, X | Y # 交集、并集 ({'a'}, {'p','c','k','h','a'}) >>> X - Y, X > Y # 差集、超集判断 ({'c','k','h'}, False) 即使数学细胞不太多的程序员也会发现，集合对去重、找差异、以及不做排序的与顺序无关的相等比较等常见任务很有用：python >>> list(set([3, 1, 2, 1, 3, 1])) # 去重 [1, 2, 3] >>> set('code') - set('hack') # 集合差 {'d','o','e'} >>> set('code') == set('deoc') # 与顺序无关的相等比较 True 正如你在本部分后面会

看到的，普通集合本身是可变的，可以被修改（例如用它们的 `remove` 和 `add` 方法），不过它们内部的不可变条目按定义是不能改的。

深度理解

- 核心概念：集合 = 无序 + 唯一（元素去重自动生效）。
三大招牌操作：交 `&`、并 `|`、差 `-`，还有 `>`（超集）等比较运算。注意 `{'a','b'}` 是集合字面量，而 `{}` 是空字典——空集合必须 `set()`。
- 底层实现：set 与 dict 共用哈希表引擎（CPython 里 `set` 就是“没有值的字典”）：元素必须可哈希，插入时按哈希定位，重复元素直接被去重。因此 `set('hack')` 后元素顺序不确定（哈希顺序），显示为 `{'c','k','a','h'}` 而不是原序。
- 设计原因：集合把“集合论”的数学操作直接变成运算符——业务上常见的“哪些是共同好友”（交集）、“哪些是独有”（差集）就一行；去重则是哈希去重 $O(n)$ 秒杀手动循环。
- 实际问题：`list(set(lst))` 去重（注意：会丢失顺序，保序去重可用 `dict.fromkeys(lst)`）；`set(A) - set(B)` 找差异；`set(a) == set(b)` 判断“乱序等价”（如判断题答案比对）；用集合的 `in` 做成员判断 ($O(1)$)。
- 初学者误区：①把 `{}` 当空集合——它是空字典；②集合元素想放列表——列表不可哈希，`TypeError`（要放元组）；③以为集合保持顺序——它是无序的，显示顺序与插入顺序无关。

代码分析

```
>>> X = set('hack') #
>>> Y = {'a', 'p', 'p'} # 'p'
>>> X & Y, X | Y # {a} {a,p,c,k,h}
({'a'}, {'p', 'c', 'k', 'h', 'a'})
>>> X - Y, X > Y #
({'c', 'k', 'h'}, False)

>>> list(set([3, 1, 2, 1, 3, 1])) # set list
[1, 2, 3]
>>> set('code') - set('hack') # 'code' 'hack'
{'d', 'o', 'e'}
>>> set('code') == set('deoc') #
True
```

·解释器如何处理：set('hack')：调用 set 构造器，迭代字符串产出 4 个字符，逐个按哈希插入哈希表。{'a','p','p'}：字面量编译为 BUILDSET + SETADD 指令，第二个'p' 哈希命中已有槽位 → 直接丢弃（去重生效），所以 Y 只有 {'p','a'}。X & Y 编译为 BINARYOP：C 层取较小集合遍历，对每个元素在较大集合里哈希查找，命中即加入结果。X - Y 同理。set('code') == set('deoc') 比较两个集合大小相等且逐元素哈希匹配——与顺序无关。list(set([..])) 则是 set 去重后按哈希表的内部顺序转回列表（所以结果可能是 [1,2,3]，顺序不保证）。

·设计思想：集合把“唯一性”从业务代码提升到数据结构层面——你不需要写循环判断“是否已存在”，数据结构替你保证。这与字典“键唯一”是同一个哈希引擎的两个面孔。

4.11.2 布尔值与 None (Booleans and None)

英文原文

Python also comes with Booleans, with predefined True and False objects that are essentially just the integers 1 and 0 with custom display logic; as well as a special placeholder object called None, commonly used to initialize names and objects and designate an absence of a result in functions:

```
python >>> 1 > 2, 1 < 2 # 布尔值(False, True)
```

```
>>> bool('hack') # 所有对象都有布尔值True # 非空即为 True
```

```
>>> X = None # None 占位符>>> print(X) # 但 None 也是一个"东西" None
```

```
>>> L = [None] * 100 # 初始化一个有 100 个 None 的列表 (The rest of the L display is omitted here; it's a list of 100 Nones.)
```

中文翻译

Python 还带有布尔值 (Boolean) , 提供预定义的 True 和 False 对象——它们本质上就是带自定义显示逻辑的整数 1 和 0; 还有一个特殊的占位对象叫 None, 常用于初始化名字和对象, 并在函数中表示"没有结果":

```
python >>> 1 > 2, 1 < 2 # 布尔值(False, True) >>> bool('hack') # 所有对象都有布尔值True # 非空即为 True >>> X = None # None 占位符>>> print(X) # 但 None 也是一个"东西"
```

" None >>> L = [None] 100 # 初始化一个有 100 个 None 的列表（这里省略了 L 的其余显示；它是一个 100 个 None 的列表。）

深度理解

·核心概念：两个"特殊公民"：True/False（布尔值，本质是 1/0 的定制显示）；None（"无"的占位对象）。所有对象都有布尔语义：空的东西为 False，非空为 True（这是 if 判断、bool() 的通用规则，本书后面会系统讲）。

·底层实现：True/False 是解释器启动时创建的两个单例布尔对象，其整数子类（int 的子类）身份让 True == 1 成立；None 同样是单例对象——所有 x = None 都指向同一个对象（CPython 里的 PyNone）。[None] 100 复用同一个 None 引用填满 100 个槽位——因为 None 不可变，共享无碍。

·设计原因：布尔值作为一等对象，让 if 1 > 2: 的条件本身成为"一个值"（可存储、可返回、可比较）；None 则给了"没有结果/尚未赋值"一个明确的对象，而不是 C 的 NULL 指针（Python 里没有指针，None 是最接近的"空引用"）。

·实际问题：X = None 先占位、稍后填充是常见模式；函数没有 return 时隐式返回 None；if x is None: 判断未赋值（用 is 而不用 ==，因为 None 是单例）；bool('hack') 这类"真假测试"是 if/while 内部做的事。

·初学者误区：①把 None 当 False 用——None == False 是 False，判断"是不是 None"必须用 is None；②以为

True 和 1 是不同东西——它们相等 (True == 1) ; ③print(X) 输出 None 时以为"什么都没打印"——None 是真实的对象, 打印它显示 "None"。

代码分析

```
>>> 1 > 2, 1 < 2                                #
(False, True)
>>> bool('hack')                                # → True
True
>>> bool('')                                     # → False
False
>>> X = None                                     # ""
>>> print(X)                                    # None
None
>>> L = [None] * 100                            # 100 None
```

·解释器如何处理: $1 > 2$ 编译为 COMPAREOP: C 层比较两个整数对象, 结果包装成 True/False 单例对象返回——所以比较表达式"有值", 可以直接放进元组。bool('hack') 调用 bool 类型: 对任意对象执行"真值测试"——str 的规则是"长度非零即真", 'hack' 非空 → True (空字符串 → False)。X = None 是普通绑定: X 指向 PyNone 单例; print(X) 调用 None 的显示逻辑, 打印出 None 这个词。[None] 100 的重复操作把同一个 None 引用复制 100 份填进新数组——零额外对象分配。

·设计思想: 真值测试的"空即假、非空即真"是 Python 最优雅的约定之一: if lst: (非空列表)、if s: (非空字符串)、if x: (非 None) 这些"口语化"判断, 都建立在"对象自带布尔语义"之上——后面学类时, 你也可以用 bool 自定义。

4.11.3 类型对象 (Types)

英文原文

One last core object merits a callout here. The type object, returned by the type built-in function, is an object that gives the type of another object. We used it earlier when exploring the help function, but here's its actual result: `python >>> L = [1, 2, 3] >>> type(L)`
一个列表对象的类型<class'list'> `>>> type(type(L))` # 连类型本身都是对象! <class'type'> Besides allowing you to explore your objects interactively, the type object in its most practical application allows code to check the types of the objects it processes. In fact, there are at least three ways to do so in a Python script: `python >>> type(L) == type([])` # 类型测试 (如果你非要测...) `True` # 用一个真实对象 `>>> type(L) == list` # 用类型名 `True` `>>> isinstance(L, list)` # 面向对象的做法 `True`
But now that this book has shown you all these ways to do type testing, it's required by Python law to tell you that doing so is almost always the wrong thing to do in a Python program (and often a sign of an ex-Java programmer first starting to use Python!). The reason won't become completely clear until later in the book when we start writing larger code units like functions, but it's a—and perhaps the—core Python concept. By checking for specific types in your code, you effectively break its flexibility: you limit it to working on just one type. Without such checks, your code ma

y be able to work on a whole range of types automatically. This is part of the polymorphism mentioned earlier, and it stems from Python's lack of type declarations. As you'll learn more when we step up to coding functions and classes, in Python, we code to object interfaces (operations supported), not to types. That is, we care what an object does, not what it is. Not caring about specific types means that code can be applied to many of them: any object with a compatible interface will work, regardless of its specific type. Although type checking is supported—and even required in some rare cases—you'll see that it's not usually the "Pythonic" way of thinking. In fact, you'll probably find that polymorphism is the key to using Python well.

中文翻译

还有最后一个核心对象值得一提。type 对象由内置函数 type 返回，是一个给出另一个对象类型的对象。我们在探索 help 函数时用过它，但下面是它的实际结果：python >>> L = [1, 2, 3] >>> type(L) # 一个列表对象的类型<class 'list'> >>> type(type(L)) # 连类型本身都是对象！<class 'type'> 除了让你能交互式地探索对象之外，type 对象最实际的应用是让代码检查它处理对象的类型。事实上，在 Python 脚本里有至少三种检查方式：python >>> type(L) == type([]) # 类型测试（如果你非要测...）True # 用一个真实对象>>> type(L) == list # 用类型名True >>> isinstance(L, list) # 面向对象的做法True 但现在本书既然给你看了所有这些类型测试的方法，按 Python 的法律就必须

告诉你：在 Python 程序中这样做几乎总是错误的（而且常常是从 Java 转来、刚开始用 Python 的前 Java 程序员会做的事！）。原因要到本书后面我们开始写函数这样更大的代码单元时才完全清楚，但它是 Python 的一个核心概念——也许是最核心的概念。通过在代码中检查特定类型，你实际上破坏了它的灵活性：你把它限制为只能作用于一种类型。没有这些检查，你的代码也许能自动作用于一整片类型的范围。这就是前面提到的多态（polymorphism）的一部分，它源于 Python 没有类型声明。等我们学习编写函数和类时会更多地理解：在 Python 中，我们面向对象的接口（支持的操作）编程，而不是面向类型。也就是说，我们关心一个对象能做什么，而不是它是什么。不关心特定类型意味着代码可以应用到许多类型上：任何具有兼容接口的对象都能工作，无论它的具体类型是什么。虽然类型检查是被支持的——甚至在少数罕见场合是必需的——你会看到，它通常不是“Python 式”（Pythonic）的思维方式。事实上，你大概会发现，多态是用好 Python 的钥匙。

深度理解

·核心概念：类型也是对象！`type(L)` 返回 `<class'list'>`——这个“类对象”本身又是 `<class'type'>` 类型的对象。三种类型测试写法：`type(L) == type([])`、`type(L) == list`、`isinstance(L, list)`（后者最正）。但作者用近乎押韵的方式警告：类型测试几乎总是错误做法。

·底层实现：`type(L)` 读取对象头的 `obtype` 指针并包装成类型对象返回；`type(type(L))` 读出的是 `type` 类型对象的自身类型（`<class'type'>`，自举）。`isinstance` 比 `==`

更宽容：它检查"是 list 或 list 的子类"，所以自定义列表子类也能通过。type(L) == list 则严格得多（子类实例不通过）——这正是三种写法行为上的细微差别。

·设计原因：Python 把类型检查做成"能用但不该用"，因为检查类型 = 把代码钉死在某一种类型上，摧毁多态。真正的 Python 式做法是鸭子类型：只要对象支持所需接口（有 .upper()、能 +、能迭代）就让它工作——不管它是什么类型。这就是作者说"多态是用好 Python 的钥匙"的深意。

·实际问题：什么时候该检查类型？极少数场合（如处理二进制数据区分 str/bytes、序列化前检查）会用到 isinstance；但绝大多数场景用"接口约定 + 直接调用"即可。遇到 type(x) == list 的旧代码，通常可以重构掉。

·初学者误区：①从 C/Java 带过来的"先查类型再用"的习惯——在 Python 里这是反模式；②以为 type(L) == list 与 isinstance(L, list) 等价——前者不认子类，后者认；③把"动态类型"理解成"类型不重要"——类型很重要，只是由对象决定、按接口使用。

代码分析

```
>>> L = [1, 2, 3]
>>> type(L)                                # L
<class 'list'>
>>> type(type(L))                          # type
<class 'type'>

>>> type(L) == type([])                   # ""
True
```

```
>>> type(L) == list          #
True
>>> isinstance(L, list)     # "..."
True
```

·解释器如何处理：type(L) 编译为 GETAWAITABLE？不——它编译为普通函数调用：type 是内置类型对象，调用它时 C 层返回 L 的对象头里的类型指针（<class'list'>）。type(type(L)) 先取 list 类型对象，再取它的类型——得到 <class'type'>，说明"类型对象是 type 类型的实例"（元类型自举，这背后是 metaclass 体系，OOP 部分才展开）。type(L) == list：比较两个对象是否相同（都是 <class'list'> 单例 → True）。isinstance(L, list) 走 C 层类型检查：L 的 obtype 是 list，且继承链上匹配 → True。三种写法执行结果相同，但 isinstance 走的是"继承链检查"，对子类更宽容。

·设计思想：type(type(L)) 这条"套娃"是 Python"一切皆对象"的终极证明——连类型本身都符合对象规则。正因为类型是对象，你才能把它当参数传、当值存（高级技巧如类型工厂）。但常规代码里，类型检查应当让位给接口检查——多态优于检验。

4.11.4 类型标注 (Type Hinting)

英文原文

That being said, Python has slowly accumulated a type-declaration facility known as type hinting, based originally on its earlier function annotations and inspired by the TypeScript dialect of JavaScript. With these s

yntax and module extensions, it is possible to name expected object types of function arguments and results, attributes in class-based objects, and even simple variables in Python code, and these hints may be used by external type checkers like mypy: `python >>> x: int = 1 # 可选标注: x 也许是个整数>>> x = 'anything'` # 但它不必须是! Importantly, though, Python type hinting is meant only for documentation and use by third-party tools. The Python language does not itself mandate or use type declarations and has no plans to ever do so. Hence, type hinting by most measures is largely academic, no more useful than in-program comments, and oddly contrary to Python's core ideas. The fact that it was nevertheless elevated to language syntax and complex subdomain arguably does a disservice to Python learners and users alike. Especially for beginners, this is an optional and peripheral topic that's best deferred until you master the flexibility of Python's dynamic typing. We'll study it only briefly in this book, in Chapter 6. To be sure, type hinting does not mean that Python is no longer dynamically typed. Indeed, a statically typed Python that requires type declarations would not be a Python at all! Some programmers accustomed to restrictive languages may regrettably code Python type hints anyhow as a hard-to-break habit (or misguided display of prowess), but good programmers focus instead on Python's polymorphism. As you'll find, it's how to code Python in Pyt

中文翻译

话虽如此，Python 还是慢慢积累了一套叫类型标注（type hinting）的类型声明设施，它最初基于更早的函数注解（annotation），灵感来自 JavaScript 的 TypeScript 方言。借助这些语法和模块扩展，可以为函数参数和结果、基于类的对象中的属性、甚至 Python 代码中的简单变量写出期望的对象类型，这些标注可以被 mypy 之类的外部类型检查器使用：`python >>> x: int = 1 # 可选标注：x 也许是个整数``>>> x = 'anything' # 但它不必须是！`但重要的是：Python 类型标注只用于文档和第三方工具。Python 语言本身不强制、也不使用类型声明，而且永远没有这样做的计划。因此，从大多数衡量标准看，类型标注很大程度上是学究气的，其用处不比程序内注释更大，而且古怪地与 Python 的核心思想相悖。它居然还是被提升为语言语法和复杂的子领域，这可以说对 Python 学习者和使用者都是一种伤害。尤其对初学者，这是一个可选的、边缘的话题，最好等你在 Python 动态类型的灵活性上游刃有余之后再学。本书只在第 6 章简短地研究它。需要明确的是，类型标注并不意味着 Python 不再动态类型化。事实上，一个要求类型声明的静态类型化 Python 根本不是 Python！一些习惯了约束性语言的程序员，可能会把写 Python 类型标注当作业已难改的习惯（或误入歧途的炫技），但好的程序员会转而聚焦于 Python 的多态。正如你将发现的，这才是“用 Python 的方式写 Python”。

深度理解

·核心概念：类型标注 = "可选"的元数据：`x: int = 1` 只是声明意图（文档），解释器完全不执行——`x = 'anything'` 照常通过，没有任何报错。它的消费者是 `mypy` 等外部静态检查工具。

·底层实现：标注在函数对象里存进 `annotations` 字典；`x: int = 1` 这类变量标注存在模块/函数级 `annotations` 里，解释器求值标注表达式后丢弃或存字典，不产生任何运行时类型检查字节码。所以它对运行行为零影响。

·设计原因：作者的态度非常鲜明：标注"违背 Python 核心思想"（因为核心思想是动态+多态），对初学者是有害的边缘话题。这与 TypeScript 不同：TS 的类型是编译期强制的一部分，而 Python 的类型标注是"借来的概念，不执行、不强制、不影响运行"。

·实际问题：大型团队/大型代码库会用 `mypy` 做静态检查、用类型标注当文档；但对学习阶段和小项目，动态类型 + 多态 + 测试才是 Python 的常规武器。想了解可以在第 6 章之后回来，本书只简短涉及。

·初学者误区：①以为 `x: int = 1` 之后赋字符串会报错——不会，运行期无感；②以为标注"让 Python 变成静态类型语言"——它永远是动态类型，标注只是贴纸；③花大量时间学类型体操而忽略多态——作者直言这是本末倒置。

代码分析

```
>>> x: int = 1                                # x
>>> x = 'anything'                             #
>>> x
'anything'
```

·解释器如何处理：x: int = 1 编译为普通赋值字节码（STORENAME x），唯一的差别是：标注元数据被存入当前作用域的 annotations 字典（{'x': <class'int'>}），然后没有然后——VM 不为标注生成任何检查指令。x = 'anything' 就是一次普通赋值，把字符串对象绑到 x——解释器既不查 annotations 也不报错。真正的检查发生在你主动运行的 mypy 等外部工具里（它在源码级别分析，不运行程序）。

·设计思想：标注 = “写给工具和人看的贴纸，不是写给解释器的命令”。这就是为什么作者说它“不比程序内注释更有用”——它本质上就是结构化的注释，而 Python 的运行时哲学依然如故：鸭子类型、多态、让代码作用于一切兼容接口。

4.11.5 用户自定义对象 (User-Defined Objects)

英文原文

We won't study object-oriented programming (OOP) in Python and its class statement until later in this book. In abstract terms, though, classes define new types of objects that extend the core set, so they merit a passing glance here. Suppose, for example, that you wish to have a kind of object that models workers in a company. Although there is no such specific core o

bject type in Python (it's not an HR language, after all), a user-defined class might fit the bill: python >>> class Worker: ... # 类体代码在此省略, 敬请期待第六部分.

.. Most of this code is omitted because it wouldn't make much sense at this point in the book, but such a class might define attributes of workers like name and pay, as well as behavior coded as custom methods. Calling the class would generate objects that are instances of our new type, and the class's methods would process them: python >>> sue = Worker('Sue Jones', 60000) # 创建两个新对象>>> bob = Worker('Bob Smith', 50000) # 每个都有名字和工资>>> sue.lastName() # 调用方法处理 sue'Jones'>>> bob.lastName() # 调用方法处理 bob'Smith'>>> sue.giveRaise(.10) # 更新 sue 的工资>>> sue.pay # 显示 sue 的工资66000.0 This is called object-oriented, because there is always an implied subject in functions within a class. Class-based objects ultimately use built-in objects internally, and we can always describe things like workers with Python's built-in objects instead, as we did with dictionaries and lists earlier. Classes, though, implement operations with meaningful names, add structure to your code, and come with inheritance mechanisms that lend themselves to customization by extension. In OOP, we strive to extend software by writing new classes, not by changing what already works. All of which is well beyond the bounds of this object preview, though, so we must stop short here. For full disclosure on user-d

efined object types coded with classes, you'll have to read on. Because classes build upon other tools in Python, they are one of the major destinations of this book's journey.

中文翻译

我们要到本书后面才会学习 Python 中的面向对象编程 (OOP) 及其 class 语句。但抽象地说，类定义的是扩展核心集的新对象类型，所以在这里值得瞥一眼。假设，例如，你想要一种模拟公司员工的对象。虽然 Python 里没有这样特定的核心对象类型（毕竟它不是 HR 语言），但用户自定义的类可能正合适：python >>> class Worker: ... # 类代码在此省略，敬请期待第六部分... 这段代码大部分被省略了，因为在这个时间点它还没什么意义，但这样的类可能会定义员工的属性 (attribute)，如 name 和 pay，以及写成自定义方法的行为 (behavior)。调用这个类会生成我们新类型的实例对象，类的方法会处理它们：python >>> sue = Worker('Sue Jones', 60000) # 创建两个新对象>>> bob = Worker('Bob Smith', 50000) # 每个都有名字和工资>>> sue.lastName() # 调用方法处理 sue'Jones'>>> bob.lastName() # 调用方法处理 bob'Smith'>>> sue.giveRaise(.10) # 更新 sue 的工资>>> sue.pay # 显示 sue 的工资66000.0 这被称为面向对象 (object-oriented)，因为类里的函数总是有一个隐含的主语。基于类的对象最终在内部使用内置对象，而且我们总可以用 Python 的内置对象来描述员工这类事物——就像我们前面用字典和列表做的那样。不过，类用有意义的名字实现操作、为你的代码增加结构，并且带有继承 (inheritance) 机制，非常适合

通过扩展来实现定制。在 OOP 中，我们努力通过编写新类来扩展软件，而不是修改已经能工作的东西。所有这些都是本次对象预览的范围之外了，所以我们必须在此打住。关于用类编写的用户自定义对象类型的完整披露，你得继续读下去。因为类建立在 Python 的其他工具之上，它们是本书旅程的主要目的地之一。

深度理解

·核心概念：类（class）= 自定义对象类型的"设计图纸"。
Worker(...) 制造实例对象；方法（lastName、giveRaise）是"隐含主语"的函数——sue.lastName() 里主语是 sue。类把"属性 + 行为"打包成新类型，并带来继承等扩展机制。

·底层实现：sue = Worker('Sue Jones', 60000) 的实际流程：调用 Worker 类型对象 → C 层分配新实例 → 调用其 init（本书第六部分讲的初始化方法）把'Sue Jones'和 60000 存成实例属性。类对象自身也是对象（type 的实例）；实例的属性和方法都通过名字空间字典查找。sue.giveRaise(.10) 内部把 sue 作为第一个参数传给类里的方法。

·设计原因：为什么有了字典还要类？字典能装数据，但类还提供：有意义的方法名（giveRaise vs rec['pay'] = 1.1）、代码结构（相关行为聚集）、继承（新类复用旧类行为，通过扩展定制而非修改）。OOP 的黄金律：加新类，不改旧代码。

·实际问题：字典表达数据（记录/JSON）、类表达"有行

为的数据"（领域对象）。真实项目两者并用：数据层用字典/JSON，业务逻辑用类。第 27 章会把本节的 Worker 完整实现并与字典/命名元组对照。

·初学者误区：①以为类"比字典高级"——它们是不同的工具，字典做数据、类做行为模型；②以为 `sue = Worker(...)` 是函数调用——它是"类型实例化"；③把"面向对象"误读成"用对象"——核心是"带行为 + 可继承的类型"，本书第六部分之前，字典+函数也够用。

代码分析

```
>>> class Worker:                                     #
...     pass
...
>>> sue = Worker('Sue Jones', 60000) # sue
>>> bob = Worker('Bob Smith', 50000) # bob
>>> sue.giveRaise(.10)                                # sue
>>> sue.pay                                             #
66000.0
```

·解释器如何处理：`class Worker:` 编译为 `BUILDCLASS` 指令：执行类体代码，把其中定义的名字收集成类命名空间，然后创建类对象并绑定到名字 `Worker`——类的定义本身是运行时执行的一条语句（这就是为什么 `class` 是语句）。`Worker('Sue Jones', 60000)` 是调用类对象：C 层走类型实例化协议——分配实例内存 → 调用 `Worker.init(sue, 'Sue Jones', 60000)` 初始化属性 → 返回实例。`sue.giveRaise(.10)` 的属性查找链：先在实例命名空间找 `giveRaise`，找不到再沿类型链找——类方法定义处找到后绑定实例调用。`sue.pay` 直接读实例属性字典。

·设计思想：实例 = "属性字典 + 类型指针"，方法 = "挂在类型上的共享函数"——这与内置对象（如列表实例 + list 类型上的方法）完全同构。换句话说：类不过是让你自制内置类型的语法。这就是作者说"类建立在其他工具之上"的含义——对象模型从第四章到 OOP 部分是一脉相承的。

4.11.6 以及其他一切 (And Everything Else)

英文原文

As mentioned earlier, everything you can process in a Python script is a type of object, so our object-type tour is necessarily incomplete. However, even though everything in Python is an "object," not everything is considered a part of Python's core toolset. Other object types in Python either are related to program execution (like functions, modules, classes, and compiled code), or are implemented by imported module functions, not language syntax. The latter of these also tend to have application-specific roles—text patterns, database interfaces, network connections, and so on. Moreover, keep in mind that the objects we've met here are objects, but not necessarily object-oriented—a concept that usually requires the Python class statement, which you'll meet again later in this book. Still, Python's core objects are the workhorses of all Python scripts you're likely to meet and are often the basis of larger noncore objects.

中文翻译

正如前面提到的，你在 Python 脚本里能处理的每一样东西都是某种对象，所以我们的对象类型巡游注定是不完整的。不过，即便 Python 里一切都是“对象”，也并不是每样东西都算 Python 核心工具集的一部分。Python 中其他对象类型，要么与程序执行相关（如函数、模块、类、已编译代码），要么由导入的模块函数实现，而不是由语言语法实现。后者往往还有应用特定的角色——文本模式、数据库接口、网络连接，等等。此外，请记住：我们在这里遇到的对象是对象（objects），但不一定是面向对象的（object-oriented）——后者通常需要 Python 的 class 语句，你会在本书后面再次遇到它。不过，Python 的核心对象是你可能遇到的所有 Python 脚本的主力军，而且常常是更大的非核心对象的基础。

深度理解

- 核心概念：区分两件事：“一切皆对象”（对象模型：所有实体都是对象）与“面向对象”（OOP：用 class 定义新类型）。核心对象不要求 class，但它是一切非核心对象（正则 pattern、socket、数据库游标、你的类）的地基。
- 底层实现：函数、模块、类、已编译代码（code object）都是“程序执行相关”的对象——它们住在运行时系统里，由 import/def/class 等机制产生；而 re 编译的模式对象、socket 连接对象则是由标准库模块的工厂函数（如 `re.compile()`、`socket.socket()`）创建。
- 设计原因：“一切皆对象”保证统一规则（引用、回收、传

参、属性访问) 通用于所有东西; "核心/非核心"的划分保证语言核心小而稳定、功能通过模块无限扩展——这正是 "标准库 + 扩展"生态的基础。

·实际问题: 看到 `type()` 输出 `<class'socket'>` 不要惊讶——这就是 "非核心但依然是对象" 的例子。学完本章, 你已经有了阅读任何 Python 对象的通用心智模型。

·初学者误区: 以为 "核心对象 = 所有对象"——表 4-1 只是核心子集; 以为 "对象" 和 "面向对象" 是一回事——它们的关系正如 "木头" 和 "木匠活": 前者是材料, 后者是工艺 (`class` 是工艺工具)。

4.12 章节小结 (Chapter Summary)

英文原文

And that's a wrap for our object tour. This chapter has previewed Python's core object types and the sorts of operations we can apply to them. We've studied generic operations that work on many object types (sequence operations such as indexing and slicing, for example), as well as type-specific operations available as method calls (string splits and list appends, for instance). We've also defined some key terms, such as immutability, sequences, and polymorphism. Along the way, we've learned that Python's core object types are more flexible and powerful than what is available in lower-level languages. For instance, Python's lists

and dictionaries can nest, grow and shrink, and contain objects of any type, and their space is automatically created and cleaned up as you go. We've also glimpsed the ways that strings and files work hand in hand to support binary and text data, peeked at the iteration protocol and OOP, discussed the perils of type hinting, and introduced the if and for statements we'll be using ahead. This chapter skipped many of the details in order to provide a first tour, so you shouldn't expect all of this chapter to have made sense yet. In the next few chapters, we'll start to dig deeper, taking a second pass over Python's object types that will fill in details omitted here and give you a deeper understanding. We'll start off the next chapter with an in-depth look at Python numbers. First, though, here is another quiz to review.

中文翻译

至此，我们的对象巡游就告一段落了。本章预览了 Python 的核心对象类型，以及我们能对它们施加的各类操作。我们研究了作用于许多对象类型的通用操作（例如索引、切片这样的序列操作），也研究了以方法调用形式提供的类型特有操作（例如字符串的 `split`、列表的 `append`）。我们还定义了一些关键术语，如不可变性（`immutability`）、序列（`sequence`）和多态（`polymorphism`）。一路走来，我们了解到 Python 的核心对象类型比低级语言里可用的东西更灵活、更强大。例如，Python 的列表和字典可以嵌套、可以增长和收缩、可以容纳任何类型的对象，而且它们的空间

会随着你的使用自动创建和清理。我们还瞥见了字符串与文件如何携手支持二进制和文本数据，瞄了一眼迭代协议和 OOP，讨论了类型标注的危险之处，并介绍了我们今后要用的 if 和 for 语句。为了提供第一次巡游，本章跳过了许多细节，所以你不应该指望本章的所有内容现在就能全部理解。在接下来的几章里，我们将开始挖得更深，对 Python 的对象类型做第二遍讲解，补上这里省略的细节，给你更深的理解。下一章我们将深入探讨 Python 数字。不过首先，这里还有一个小测验供你复习。

深度理解

·核心概念：本章是“全书地图册”：数字、字符串、列表、字典、元组、文件、集合 + 布尔/None/类型——每种对象给一个“快照”。关键词汇总：序列（位置有序）、映射（按键存储）、不可变（不能改）、多态（操作含义随对象而异）、迭代协议（按需产出）、垃圾回收（内存自动管理）。

·底层实现：这些概念在 CPython 里都有具体对应物：序列=位置数组、映射=哈希表、不可变=无修改槽位、多态=按类型分派的操作符槽位、迭代=iter/next 协议、垃圾回收=引用计数+分代回收。

·设计原因：作者有意“先给全貌、再补细节”——心理上的地图先建立，后面每章是“定点爆破”。所以本章不求全懂，只求“见过”。

·实际问题：作为读者，你应该能回答“学数字该去哪一章”“列表和字典的区别”这类问题了；后续第 5 章（数字）、第 6 章（动态类型与引用）、第 7 章（字符串）、第 8

章（列表与字典）、第 9 章（元组、文件）会把这张地图逐一放大。

·初学者误区：巡游章的常见误区是"都看懂了"或"都看不懂"——都别怕：细节还没来，前面的例子到后面章节都会重现并加深。

4.13 测验与答案 (Test Your Knowledge: Quiz / Answers)

英文原文 (Quiz)

We'll explore the concepts introduced in this chapter in more detail in upcoming chapters, so we'll just cover the big ideas here: 1. Name four of Python's core object types. 2. Why are they called "core" object types? 3. What does "immutable" mean, and which three of Python's object types are considered immutable? 4. What does "sequence" mean, which objects fall into this category, and how is "iterable" related? 5. What does "mapping" mean, and which core object type is a mapping? 6. What is "polymorphism," and why should you care?

中文翻译 (测验)

我们将在接下来的章节中更详细地探索本章引入的概念，所以这里只覆盖大思想：1. 说出 Python 的四个核心对象类型。2. 为什么它们被称为"核心"对象类型？3. "不可变"是什

什么意思？Python 的哪三种对象类型被认为是不可变的？4. "序列"是什么意思？哪些对象属于这一类别？"可迭代" (iterable) 与它有什么关系？5. "映射"是什么意思？哪个核心对象类型是映射？6. 什么是"多态"？你为什么应该关心它？

英文原文 (Answers)

1. Numbers, strings, lists, dictionaries, tuples, files, and sets are generally considered to be the core object types. Booleans, None, and types themselves are classified this way as well. Some of these types are really categories: there are multiple number types (integer, floating point, complex, fraction, and decimal) and multiple string types (text strings, byte strings, and mutable byte strings). 2. They are known as "core" object types because they are part of the Python language itself and are always available. To create other objects, you generally must call functions in imported modules. Most of the core objects have specific syntax for generating their objects: 'hack', for example, is an expression that makes a string and determines the set of operations that can be applied to it. Because of this, core objects are hardwired into Python's syntax. In contrast, you must call the built-in open function to create a file object, even though this is usually considered a core object type too. 3. An "immutable" object is an object that cannot be changed after it is created. Numbers, strings, and tuples in Python fall into this category. While you cannot change an immutable obj

ect in place, you can always make a new one by running an expression. bytearrays offer mutability for strings, but they only apply directly to text if it's a simple 8-bit kind (e.g., ASCII).

4. A "sequence" is a positionally ordered collection of objects. Strings, lists, and tuples are all sequences in Python. They share common sequence operations, such as indexing, concatenation, and slicing, but also have type-specific method calls. The related term "iterable" means either a physical sequence, or a virtual one that produces its items on request. Sequences are iterable, but so are generators, files, results of functions like range, and the dictionary object (which produces its keys when iterated, just like its keys method).

5. The term "mapping" denotes an object that maps keys to associated values. Python's dictionary is the only mapping among its core object types. Mappings retain insertion order (as of Python 3.7), and support access to data stored by key, plus type-specific method calls that enable key tests, iteration, and more.

6. "Polymorphism" means that the meaning of an operation (like a +) depends on the objects being operated on. This turns out to be a key idea (and perhaps the largest) behind using Python well—not constraining code to specific types makes that code automatically applicable to many types. This becomes more obvious when coding functions and classes in Python. While Python today has type hinting, it's not used by Python itself and is meant only for d

中文翻译 (答案)

1. 数字、字符串、列表、字典、元组、文件和集合通常被认为是核心对象类型。布尔值、None 和类型本身也被归入此类。其中有些类型其实是类别 (category)：有多个数字类型 (整数、浮点、复数、分数、十进制) 和多个字符串类型 (文本字符串、字节字符串、可变字节字符串)。2. 它们被称为"核心"对象类型，因为它们是 Python 语言本身的一部分、始终可用。要创建其他对象，你通常必须调用导入模块里的函数。大多数核心对象都有专门的语法来生成它们的对象：例如，'hack' 是一个制造字符串并决定可对其施加的操作集的表达式。正因如此，核心对象被硬接 (hardwired) 进了 Python 的语法。相比之下，即使文件通常也算核心对象类型，你也必须调用内置 open 函数才能创建文件对象。3. "不可变"对象是指创建之后不能被改变的对象。Python 中的数字、字符串和元组属于这一类别。虽然你不能就地改变不可变对象，但总是可以通过运行表达式造一个新的。bytearray 为字符串提供了可变性，但只有当文本是简单的 8 位类型 (如 ASCII) 时，它才直接适用于文本。4. "序列"是按位置有序排列的对象集合。Python 中的字符串、列表和元组都是序列。它们共享常见的序列操作，如索引、拼接和切片，但也有类型特有的方法调用。相关术语"可迭代" (iterable) 指的要么是物理序列，要么是按需产出条目的虚拟序列。序列是可迭代的，但生成器、文件、range 之类函数的结果、以及字典对象 (迭代它产生键，就像它的 keys 方法一样) 也都是可迭代的。5. "映射"指的是把键映射到关联值的对象。Python 的字典是其核心对象类型中唯

一的映射。映射保留插入顺序（自 Python 3.7 起），支持按键访问存储的数据，还有支持键测试、迭代等等的类型特有方法调用。6. "多态"意味着操作（如 +）的含义取决于被操作的对象。事实证明，这是用好 Python 背后的关键思想（也许是最重要的思想）——不把代码约束到特定类型，就能让代码自动适用于许多类型。在 Python 中编写函数和类时，这一点会更加明显。虽然 Python 如今有类型标注，但 Python 本身并不使用它，它只用于文档和工具。

本章总结

技术拓展 (Technical Expansion)

对象模型全景：对象是值的载体，变量是名字

·对象 (object) 是内存中真实存在的实体：值、类型、操作集都是它的属性；变量 (name) 只是指向对象的标签。a = [1, 2, 3] 做两件事：创建列表对象、把名字 a 绑定到它。理解"对象与名字分离"，就能推导出全部共享引用 (shared reference) 行为：b = a 让 b 与 a 指向同一个列表；可变对象经 b 修改，a 会看到变化；不可变对象（如 int、str）因为无法被修改，共享永远不会出问题——这就是不可变设计的根本价值。

·引用计数与垃圾回收：CPython 中每个对象头有一个引用计数；名字被重新赋值、离开作用域、对象被删时计数递减，归零即立即回收内存 (rec = 0 一执行，整棵嵌套记录树被释放)。引用计数覆盖不了"循环引用" (A 引用

B、B 引用 A) ，所以 CPython 另配一个分代循环垃圾收集器 (0/1/2 三代，按阈值定期扫描) 专门处理。sys.getrefcount(x) 和 gc.collect() 是观察这两个机制的探针。第 6 章将完整展开。

·小对象缓存：小整数 (-5~256) 与空元组、短字符串 (部分驻留) 是解释器级缓存，共享无风险；这是"为什么 a is b 对小整数为 True、对 257 为 False"的谜底——也是理解 is 与 == 区别的起点 (is 比身份、== 比内容) 。

核心对象一览 (与 Java/C++ 对照)

需求	Python	Java	C++	
变长有序集合	list (异构、动态)	ArrayList (需泛型声明)	std::vector (需模板+类型)	
键值查找	dict (内置字面量)	HashMap (需 import)	std::unordered_map	
不可变集合	tuple (内置语法)	List.of/不可变集合	const 容器	
大整数	int (任意精度)	BigInteger (需 import 且繁琐)	无内建 (GMP 等库)	
文本处理	str + 方法 (内置)	String 工具类	std::string + 算法	
去重/集合运算	set (&、\)	、- 运算符)	HashSet 手动运算	std::set_intersection 等
多类型容器	任意对象 (无需泛型)	需泛型或 Object	需模板	
内存管理	自动 (引用计数+GC)	GC	手动/RAII	

核心差别一句话：Python 把数据结构做成语法和内置，Java/C++ 把数据结构做成库和模板——所以同样"建一张表"，Python 一个字面量，C++ 可能要十行。

版本与历史背景

- Python 3.7 (2018) : 字典插入顺序成为语言保证 (PEP 567 前的实现巧合转正) ——本书第 6 版把这一点当作常态写进正文。
- Python 3.6 (2016) : f-string (PEP 498) 成为第三种格式化; 数字分隔符 (PEP 515) 登场。
- Python 3.11 (2022) : setintmaxstrdigits 登场, 给"病态大整数转十进制"设 4300 位上限, 抵御 DoS。
- Python 3.12 (2023) : f-string 允许嵌套同种引号——本章 `f'{296999.256:,.2f} | {'sapp'[1:]}'` 正是新语法。
- 历史脉络: 本书的 % 格式化源自 C 语言 printf; .format() 是 Python 2.6/3.0 引入的"现代"方案; f-string 是"最 Python 的"第三代。读老代码三种都得认识。

高级开发者需要掌握的相关知识

- isinstance 与鸭子类型 (duck typing) 的边界把握; 何时"破戒"做类型检查 (str/bytes 判别、边界序列化) 。
- 迭代协议与生成器的内存优势: for line in f 处理 GB 级文件; 生成器表达式替代中间列表。
- 哈希与相等: 可变对象不可哈希 → 不可作键; namedtuple (第 9 章) 与自定义 hash/eq 的关系。
- 文件资源管理: with open(...) as f (第 9 章) 取代裸 close; 编码显式化是跨平台工程纪律。
- rec = 0 式的引用释放技巧、gc.collect() 排查循环引用

导致的"内存不降"。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5 星) ——这是全书第一个"硬核"章节。对象模型、可变/不可变、序列/映射、多态四个概念，是理解第 5–9 章乃至 OOP 部分（第四部分）的公共基础。这里欠下的债，后面每一章都会加倍偿还。
- 应该掌握到什么程度：
- 能不看书写出每种核心对象的字面量与 2–3 个代表操作；
- 能用自己的话解释：动态类型 vs 强类型、不可变 vs 可变、序列 vs 映射、可迭代 vs 迭代器；
- 能在 REPL 里用 `dir/help/type` 自主探索陌生对象；
- 能预判 `S[0]='z'`（报错）、`L[99]=1`（报错）、`D['e']`（报错）与 `D.get('e', d)`（不报错）之间的差异；
- 理解"变量是名字、对象是值"并能解释 `b = a` 后共享引用的后果。
- 暂时不需要：掌握类型标注、生成器细节、`bytearray` 细节、操作符重载（这些本章只预告，后续章节深入）。
- 后续应该学习哪些相关内容：
- 第 5 章：数字的完整细节（本章数字节是它的预告片）；
- 第 6 章：动态类型、引用与共享引用、赋值模型、垃圾回收（本章"变量是名字"的完整展开）；
- 第 7 章：字符串的完整细节（正则、格式化全部语法）；

- 第 8 章：列表与字典深入（推导式、排序、哈希原理）；
- 第 9 章：元组、文件、with 语句、namedtuple；
- 第四部分（10–13 章）：语句与语法（if/for 的完整形态）；
- 第六部分：类与 OOP——届时你会发现类"复用"的正是本章的对象模型。
- 练习建议：用字典嵌套描述你身边一个实体（如"一本书"）；用一个 3×3 列表实践 [row[i] for row in M]；在 REPL 里对字符串、列表、字典各跑一遍 dir() + help()。